

中图分类号：TP316

论文编号：100006SY2306143

北京航空航天大学
硕士学位论文

面向云边端跨域分布式训练的存
储加速技术

作者姓名	谭升阳
学科专业	计算机科学与技术
指导教师	肖利民 教授
培养学院	计算机学院

Storage Acceleration Technology for Cloud-Edge-Device Cross-Domain Distributed Training

A Dissertation Submitted for the Degree of Master of Engineering

Candidate: Shengyang Tan

Supervisor: Prof. Limin Xiao

School of Computer Science and Engineering
Beihang University, Beijing, China

中图分类号：TP316

论文编号：100006SY2306143

硕 士 学 位 论 文

面向云边端跨域分布式训练的存储加速技术

作者姓名	谭升阳	申请学位级别	工学硕士
指导教师姓名	肖利民	职 称	教授
学科专业	计算机科学与技术	研究方向	计算机系统结构
一级学科	计算机科学与技术	学科方向	体系结构
学习时间自	2023 年 09 月 01 日	至	2026 年 06 月 30 日
论文提交日期	2026 年 04 月 14 日	论文答辩日期	2026 年 05 月 22 日
学位授予单位	北京航空航天大学	学位授予日期	年 月 日

关于学位论文的独创性声明

本人郑重声明：所呈交的论文是本人在指导教师指导下独立进行研究工作所取得的成果，论文中有关资料和数据是实事求是的。尽我所知，除文中已经加以标注和致谢外，本论文不包含其他人已经发表或撰写的研究成果，也不包含本人或他人为获得北京航空航天大学或其它教育机构的学位或学历证书而使用过的材料。与我一同工作的同志对研究所做的任何贡献均已在论文中作出了明确的说明。

若有不实之处，本人愿意承担相关法律责任。

学位论文作者签名：_____

日期： 年 月 日

学位论文使用授权书

本人完全同意北京航空航天大学有权使用本学位论文（包括但不限于其印刷版和电子版），使用方式包括但不限于：保留学位论文，按规定向国家有关部门（机构）送交学位论文，以学术交流为目的赠送和交换学位论文，允许学位论文被查阅、借阅和复印，将学位论文的全部或部分内容编入有关数据库进行检索，采用影印、缩印或其他复制手段保存学位论文。

保密学位论文在解密后的使用授权同上。

学位论文作者签名：_____

日期： 年 月 日

指导教师签名：_____

日期： 年 月 日

摘 要

随着产业数智转型的不断深入，云数据中心、边缘节点与终端设备构成的三层异构计算拓扑逐步形成，云边端跨域计算需将分散于不同网络域的算力与存储统一纳入。课题组据此研发了云边端跨域虚拟存储空间系统，打通了云边端间的基本数据访问通路。

然而，云边端环境中的广域链路存在带宽受限、传播延迟显著等问题，使得训练全流程的 I/O 效率面临严峻考验。具体而言，元数据查询需频繁穿越跨域链路，开销成倍放大，严重拖慢训练冷启动；训练样本读取需从云边端远端节点拉取，路径长、耗时长，计算单元容易因等待数据而空转；检查点写入与恢复时，同步写入时间长，容易阻塞训练并导致算力闲置；同时，边缘节点状态波动较大，快速恢复也面临严峻挑战。

因此，本文面向云边端跨域分布式训练中的 I/O 性能瓶颈，围绕元数据预取、数据预热和检查点写入与恢复三条路径展开存储层面的加速研究，旨在提升云边端跨域分布式训练的效率。本文的具体研究工作和主要成果如下：

(1) 针对元数据跨域访问开销过大的问题，设计并实现面向跨域训练的元数据批量预取与缓存方法。首先，在数据集初始化阶段，通过结合异步递归目录树与请求聚合，在单轮内完成全量元数据的拉取；其次，通过三级缓存架构，驻留预取到的元数据，消除后续轮次重复的网络请求。实验结果表明，该方法显著降低了元数据访问开销，缩短了冷启动时间，也明显提升了后续轮次的元数据访问效率。

(2) 针对跨域样本加载拖慢训练节奏的问题，设计并实现基于异步预取调度的数据预热技术。首先，利用训练框架随机种子，确定可预知性访问序列，确定对应文件访问顺序；其次，预取引擎根据多因子优先级策略生成预取任务，在带宽配额约束下将数据异步搬运至本地缓存；最后，训练执行过程中，以预取窗口始终领先于训练消费进度的方式持续搬运数据。实验结果表明，该机制有效地将计算和加载流水线并行化，缓解数据等待引起的算力浪费。

(3) 针对检查点快速写入和恢复的问题，设计并实现基于状态感知分配的检查点分级写入技术。首先，依据存储容量、实时带宽和拓扑距离等维度，构建边缘节点的动态资源画像；其次，依据检查点分片的不同关键程度，划分优先等级，定制对应匹配差异化副本策略，并利用异步机制完成写入；最后，在故障恢复时，依据优先等级就近取副

本，以最低传输代价恢复训练状态。实验结果表明，该机制能有效减少写入阻塞，同时能提升边端故障场景下的恢复效率。

综上，本文面向云边端跨域分布式训练中的关键 I/O 性能瓶颈，设计并实现了元数据批量预取与缓存方法、基于异步预取调度的数据预热技术以及基于状态感知分配的检查点分级写入技术，有效优化了跨域数据访问通路，提升了云边端跨域分布式训练效率。

关键词：跨域分布式训练，云边端协同，分布式存储系统，跨域数据访问，广域网络

Abstract

As industrial digital and intelligent transformation deepens, a three-tier heterogeneous computing topology comprising cloud data centers, edge nodes, and terminal devices has gradually taken shape, and cloud-edge-device cross-domain computing needs to integrate compute and storage resources scattered across different network domains. Accordingly, our research group developed a cloud-edge-device cross-domain virtual storage system that establishes a basic data access path among the three tiers.

However, wide-area links in cloud-edge-device environments suffer from limited bandwidth and significant propagation delay, posing severe challenges to I/O efficiency throughout the training workflow. Specifically, during metadata queries, requests must frequently traverse cross-domain links, causing overhead to multiply and significantly delaying training cold start; during training sample reading, data must be fetched from remote cloud-edge-device nodes over long paths with high latency, leaving compute units prone to idling while waiting for data; during checkpoint writing, prolonged synchronous writes tend to block training and waste compute resources, while the high volatility of edge node status also poses severe challenges for fast recovery.

Therefore, targeting the I/O performance bottlenecks in cloud-edge-device cross-domain distributed training, this thesis conducts storage-level acceleration research along three paths—metadata prefetching, data prewarming, and checkpoint writing and recovery—aiming to improve the efficiency of cloud-edge-device cross-domain distributed training. The specific research work and main results are as follows:

(1) To address the excessive overhead of cross-domain metadata access, a metadata batch prefetching and multi-level caching mechanism for training cold start is designed and implemented. First, during the dataset initialization phase, full metadata is retrieved in a single round by combining asynchronous recursive directory tree traversal with request aggregation. Second, a three-level cache architecture retains prefetched metadata and eliminates redundant network requests in subsequent epochs. Experimental results show that this mechanism

significantly reduces metadata access overhead and cold start time, and notably improves metadata access efficiency in subsequent epochs.

(2) To address the problem of cross-domain sample loading slowing down training, a data prewarming mechanism based on asynchronous prefetch scheduling is designed and implemented. First, the predictable access sequence is determined using the training framework's random seeds to establish the corresponding file access order. Second, the prefetch engine generates prefetch tasks according to a multi-factor priority policy and asynchronously transfers data to a local cache under bandwidth quota constraints. Finally, during training execution, data is continuously transferred with the prefetch window always staying ahead of training consumption progress. Experimental results show that this mechanism effectively parallelizes the computation and loading pipelines, alleviating compute waste caused by data waiting.

(3) To address the problem of fast checkpoint writing and recovery, a state-aware tiered checkpoint writing technique is designed and implemented. First, a dynamic resource profile of edge nodes is constructed based on dimensions such as storage capacity, real-time bandwidth, and topological distance. Second, checkpoint shards are classified into priority tiers according to their criticality, matched with differentiated replication policies, and written using asynchronous mechanisms. Finally, during failure recovery, replicas are retrieved from the nearest nodes according to priority tiers, restoring training state with minimal transfer cost. Experimental results show that this mechanism effectively reduces write blocking and improves recovery efficiency in edge-device failure scenarios.

In summary, targeting the key I/O performance bottlenecks in cloud-edge-device cross-domain distributed training, this thesis designs and implements a metadata batch prefetching and multi-level caching mechanism, a data prewarming mechanism based on asynchronous prefetch scheduling, and a state-aware tiered checkpoint writing technique, effectively optimizing the cross-domain data access path and improving cloud-edge-device cross-domain distributed training efficiency.

Keywords: Cross-Domain Distributed Training, Cloud-Edge-Device Collaboration, Distributed Storage System, Cross-Domain Data Access, Wide-Area Network

目 录

第一章	绪论	1
1.1	课题来源	1
1.2	研究背景与意义	1
1.3	研究目标与内容	3
1.3.1	研究目标	3
1.3.2	研究内容	4
1.4	主要贡献与创新之处	5
1.5	论文组织结构	7
第二章	国内外相关研究	9
2.1	元数据访问优化研究现状	9
2.1.1	面向云边端的优化方法	9
2.1.2	面向传统云集群的访问架构优化	10
2.1.3	面向传统云集群的服务性能优化	11
2.2	数据加载优化研究现状	11
2.2.1	面向数据缓存的优化方法	12
2.2.2	面向数据预取的优化方法	12
2.3	检查点写入与恢复加速研究现状	13
2.3.1	面向检查点存储的优化方法	13
2.3.2	面向检查点生成的优化方法	14
2.4	本章小结	15
第三章	面向跨域训练的元数据批量预取与缓存方法	16
3.1	引言	16
3.2	问题分析与研究思路	16
3.3	详细方案设计与实现	17
3.3.1	符号描述	17
3.3.2	整体方案设计	17
3.3.3	基于批量聚合的目录元数据预取	18
3.3.4	客户端递归遍历与多级缓存	19
3.3.5	整体执行流程分析	22

3.4	实验与验证	23
3.4.1	实验环境	23
3.4.2	多级缓存持续加速能力验证	26
3.4.3	批量聚合效果与参数敏感性实验分析	27
3.4.4	跨域网络延迟下的加速优势实验分析	29
3.4.5	批量聚合的流量优化效果实验分析	30
3.4.6	网络丢包敏感性实验分析	31
3.5	本章小结	32
第四章	基于异步预取调度的数据预热技术	33
4.1	引言	33
4.2	问题分析与研究思路	33
4.3	详细方案设计与实现	33
4.3.1	符号描述	34
4.3.2	整体方案设计	34
4.3.3	基于确定性种子的数据访问序列预计算	36
4.3.4	基于动态优先级的预取任务生成与滑动窗口合并	37
4.3.5	基于带宽感知的预取任务调度与执行	39
4.3.6	数据预热执行与流水化加载机制	40
4.3.7	整体执行流程分析	42
4.4	实验与验证	43
4.4.1	实验环境与参数配置	43
4.4.2	本地缓存容量对预热收益的影响	45
4.4.3	并行消费规模边界分析	46
4.4.4	真实训练中的计算与 I/O 重叠验证	46
4.5	本章小结	47
第五章	基于状态感知分配的检查点分级写入技术	49
5.1	引言	49
5.2	问题分析与研究思路	49
5.3	详细方案设计与实现	49
5.3.1	符号描述	49
5.3.2	整体方案设计	50

5.3.3	动态资源模型构建	53
5.3.4	基于重要性分级的检查点分片与动态分配	54
5.3.5	基于异步传输通道的检查点写入与副本创建	55
5.3.6	基于分层恢复阶梯的检查点故障恢复	57
5.3.7	整体执行流程分析	59
5.4	实验与验证	60
5.4.1	实验环境与参数配置	60
5.4.2	端到端检查点写入与恢复方法对比	61
5.4.3	异步写入阻塞与检查点规模关系分析	62
5.4.4	跨域检查点异步上传性能分析	63
5.4.5	存储容量感知降级实验分析	64
5.4.6	检查点分级恢复效果实验分析	65
5.4.7	自动路由恢复韧性实验分析	67
5.4.8	带宽条件对分级恢复的影响分析	68
5.5	本章小结	68
第六章	云边端跨域虚拟存储空间原型系统与集成验证	70
6.1	引言	70
6.2	云边端跨域存储空间架构	70
6.2.1	物理资源层	70
6.2.2	系统支撑层	70
6.2.3	用户层	72
6.3	云边端跨域虚拟存储空间部署与工作机制	73
6.3.1	各层部署方案	73
6.3.2	训练过程中的基础访问链路	73
6.3.3	加速技术优化后的访问链路	76
6.4	实验与验证	76
6.4.1	实验环境与参数配置	76
6.4.2	带宽敏感性实验分析	77
6.4.3	缓存受限带宽敏感性实验分析	80
6.4.4	RTT 敏感性实验分析	81
6.4.5	检查点保存频率实验分析	82
6.4.6	稳定性实验分析	84

6.5 本章小结	85
总结与展望	86
参考文献	88
攻读硕士学位期间取得的成果	95
致谢	96

图清单

图 1.1	云边端智算生态	1
图 1.2	云边端跨域分布式训练的数据流转过程	2
图 1.3	本文研究内容与研究目标	4
图 3.1	面向跨域训练的元数据批量预取与缓存方法整体架构	17
图 3.2	批量目录读取处理流程	18
图 3.3	深度优先与广度优先遍历策略对比	20
图 3.4	多级级联缓存架构	22
图 3.5	多 Epoch 缓存元数据遍历耗时对比	26
图 3.6	批量预取与逐条请求耗时对比	28
图 3.7	批量块大小 β 对 ReadDir 和 GetAttr 加速比的影响	29
图 3.8	RTT 对 ReadDir 和 GetAttr 性能的影响与加速比	30
图 3.9	不同批量块大小 β 下的协议总流量对比	31
图 3.10	不同丢包率下两种模式元数据遍历耗时对比	31
图 4.1	基于异步预取调度的数据预热技术整体架构	34
图 4.2	基于确定性种子的数据访问序列生成与注入流程	36
图 4.3	动态优先级计算与滑动窗口合并机制	39
图 4.4	预取调度器调度流程与带宽控制	40
图 4.5	异步流水化数据加载时序示意图	42
图 4.6	基于异步预取调度的数据预热技术端到端执行流程	42
图 4.7	缓存容量变化对完整训练吞吐的影响	45
图 4.8	并行消费规模变化对聚合吞吐与加速比的影响	46
图 4.9	真实单卡 GPU 训练中异步预取的重叠收益	47
图 5.1	基于状态感知分配的检查点分级写入技术整体架构	51
图 5.2	优化后检查点分级写入与恢复流程	51
图 5.3	检查点分片动态分配决策流程	55
图 5.4	探测优先策略示意图	57
图 5.5	基于状态感知分配的检查点分级写入技术端到端执行流程	60
图 5.6	四种检查点保存方法的写入阻塞时间与恢复时间对比	62
图 5.7	本地异步写入阻塞时间与检查点大小的关系	63

图 5.8	跨域检查点上传时间对比	63
图 5.9	Edge-A 存储占用率对检查点分片分发路径的影响	65
图 5.10	不同 RTT 下三种恢复策略的训练可恢复时间对比	66
图 5.11	训练可恢复时间对比与缩短比例	66
图 5.12	不同节点状态与丢包率下的检查点恢复时间	67
图 5.13	不同带宽下的训练可恢复时间及其缩短比例	68
图 6.1	云边端跨域存储空间总体架构图	70
图 6.2	云边端跨域存储空间部署图	73
图 6.3	常规数据加载训练流程	75
图 6.4	常规检查点写入与恢复流程	75
图 6.5	各存储方案稳态吞吐量随带宽变化	78
图 6.6	首个 Epoch 总数据加载时间随带宽变化	79
图 6.7	缓存有限下稳态吞吐量随带宽变化	80
图 6.8	缓存有限下首个 Epoch 数据加载耗时随带宽变化	80
图 6.9	稳态吞吐量随 RTT 变化	81
图 6.10	数据加载占比随 RTT 变化	81
图 6.11	训练总时间随检查点间隔变化	83
图 6.12	训练阻塞率随检查点间隔变化	83
图 6.13	稳态吞吐量随丢包率变化	84
图 6.14	数据加载 P99 延迟随丢包率变化	84

表清单

表 3.1	面向跨域训练的元数据批量预取与缓存方法的相关符号描述表	17
表 3.2	第三章实验的硬件与软件环境	24
表 3.3	第三章实验的节点部署拓扑	24
表 3.4	第三章实验的网络模拟配置	25
表 3.5	第三章实验的合成数据集	25
表 3.6	第三章实验的关键评测指标	26
表 3.7	第三章实验的关键参数配置 (Full 模式)	26
表 4.1	基于异步预取调度的数据预热技术的相关符号描述表	34
表 4.2	天数智芯三节点集群的硬件与软件环境	43
表 4.3	天数智芯三节点集群的节点部署拓扑	44
表 4.4	天数智芯三节点集群跨域链路的默认网络模拟参数	44
表 4.5	第四章实验的关键配置	44
表 5.1	基于状态感知分配的检查点分级写入技术相关符号描述表	50
表 5.2	检查点内容重要性分级与基础副本默认配置	54
表 5.3	第五章关键评测指标	61
表 5.4	第五章实验关键参数配置	61
表 6.1	第六章实验的对比存储方案	77
表 6.2	第六章实验使用的数据集	77
表 6.3	第六章实验的关键评测指标	78
表 6.4	第六章实验的关键配置	78

第一章 绪论

1.1 课题来源

本论文依托国家重点研发计划项目《面向云边端协同的芯粒结构与系统软件》(项目编号: 2023YFB4503100) 开展研究。该项目旨在解决云边端跨域计算环境下算网存资源的聚合与协同问题, 为智算应用提供高效支撑。论文在此框架下, 聚焦云边端跨域分布式训练中的存储加速技术。

1.2 研究背景与意义

在国家战略的推动下, 制造、电网、交通等关键产业正加速数智化转变^[1]。本文所研究的“云边端环境”指由云端数据中心、边缘计算集群和终端设备构成的三层异构计算拓扑^[2,3]。云边端协同跨域计算作为新兴模式, 通过整合云边端环境中的异构资源, 统筹不同网络域 的节点, 协同完成计算任务, 有效降低数据处理延迟和成本, 保护数据隐私安全, 这一模式为实现高效、高质的数字化转型注入了强大动力^[4,5]。如图 1.1 所示, 云边端智能计算生态涵盖智能应用、计算框架、存储资源、异构通信网络以及异构智算硬件等多个层面, 为分布式训练提供了丰富的基础设施支撑^[6,7]。



图 1.1 云边端智算生态

在产业落地层面, 制造、电网、交通等行业普遍采用“模型下发、数据就近、边缘微调”的训练范式: 云端预训练的基础模型下发至边缘集群, 由边缘节点利用持续产生的业

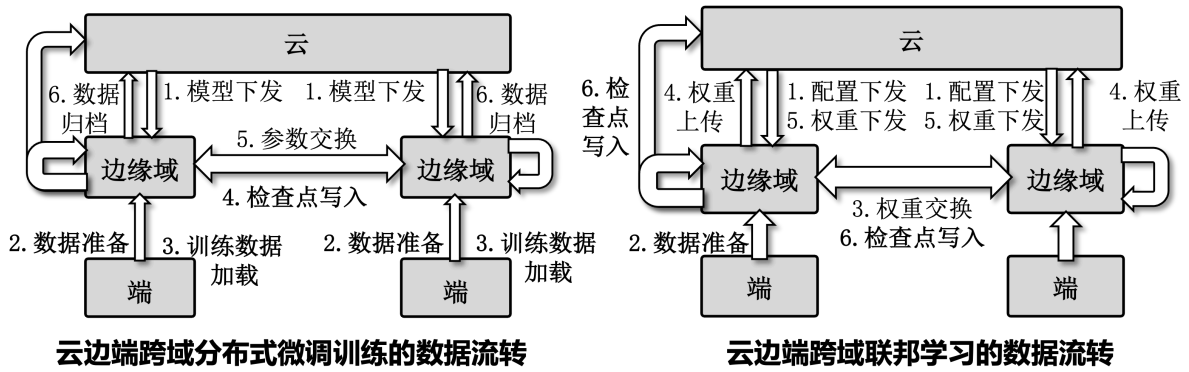


图 1.2 云边端跨域分布式训练的数据流转过程

务数据进行微调以适应特定场景^[8]。该范式在边侧执行具有多重必然性：产线质检图像、电网巡检影像等数据涉及工艺参数与关键基建安全，受合规约束不宜集中上传至云端；边侧持续产生的图像与传感器数据可达 GB 量级，实时全量回传的传输成本难以承受；缺陷检测、设备巡检等任务对模型更新时效性要求高，远程上云训练再下发难以满足实时响应。

另一方面，单一边缘域的本地数据在分布与覆盖上往往不全，边缘微调仍需跨域获取其他域的差异化样本——产线缺陷类型、输电线路故障形态在不同区域分布差异显著，罕见样本须从其他边缘域或云端补充^[9,10]。边侧微调所面向的多为参数量在百万至千万级的轻量模型（如 ResNet-18/50^[11]），在云边带宽受限条件下仍具可行性^[12]；但训练样本分散在跨域多跳的存储与链路上，每个迭代均需完成批次数据的跨域读取与就绪，端到端训练时间往往由数据路径主导。

在云边端环境中，跨域分布式训练的过程包含多阶段数据流转，如图 1.2 所示。以分布式微调训练场景为例：训练前，云为边下发微调模型，边访问端元数据，进行数据准备；训练中，边从端加载训练数据，边边节点间交换模型参数，同时，定期执行检查点写入；训练后，边将训练结果上传至云进行归档。在联邦学习场景中，也存在类似的多阶段跨域数据流转^[13,14]，涵盖配置下发、数据准备、权重交换、检查点写入等环节。

在这一数据流转过程中，云边端跨域分布式训练呈现出三类突出的现象。其一，训练数据通常以原始文件形式分散存放在边端与端侧节点，文件数量多、单文件体积小，导致数据集初始化和训练迭代中需要频繁处理元数据请求。其二，训练数据分布在云、边、端多类节点上，实际读取路径跨越不同网络域，容易因跨域访问链路较长、节点状态异构而形成慢速读取路径。其三，检查点文件集中承载模型参数与优化器状态，体量显著大

于单次训练样本读取规模，而边缘节点的出口带宽和本地存储吞吐能力又较为有限，使检查点持久化与恢复更容易成为训练过程中的长时阻塞点。

这些现象表明，云边端跨域分布式训练的存储瓶颈并不集中于单一环节，而是分别表现为元数据访问开销高、训练数据读取路径慢以及检查点写入与恢复代价大。然而，现有存储优化策略大多面向单一场景或单一阶段设计，尚未充分对接云边端环境中多阶段数据流转的需求。因此，本文从上述现象出发，进一步凝练出以下关键科学问题，以支撑云边端环境中的模型训练任务：

(1) 如何加速训练数据加载的元数据访问：现有面向云边端的优化方法在应对元数据频繁变化方面适应性不足，难以满足跨域元数据访问需求；现有面向传统云集群的优化方法决策与维护开销较大，在边缘资源受限条件下难以取得同等效果。因此，亟需研究加速云边端训练数据加载相关的元数据访问技术，缩短数据集初始化与迭代中的元数据等待，提升整体训练效率。

(2) 如何加速训练数据加载的数据读取：现有面向数据缓存的优化会打散加载顺序，干扰训练效果；面向数据预取的优化未充分适应云边端环境中数据加载路径的复杂性和分散性，未能充分考虑云边端数据节点的运行状态。因此，亟需研究加速云边端训练数据加载的数据读取技术，降低训练数据加载的时间。

(3) 如何加速训练检查点的数据写入与恢复：现有面向数据中心的检查点写入与恢复优化方法需要充裕的本地存储与稳定网络，难以直接套用于资源受限的边缘环境；而面向检查点压缩的方法在编码过程中引入的额外计算开销同样不适合算力紧张的边缘节点。因而，云边端场景迫切需要一种轻量化的加速方案，在不额外加重 CPU 的前提下缩短中小规模检查点的持久化与恢复时间。

综上所述，为解决上述关键问题，亟需构建面向云边端跨域分布式训练的存储加速方法，研究面向跨域训练的元数据批量预取与缓存、数据预热和检查点分级写入技术，形成云边端跨域存储系统，满足云边端跨域分布式训练存储系统需求，充分聚合与利用云边端算网存资源，支撑云边端环境中的模型训练任务，提升模型训练效率。

1.3 研究目标与内容

1.3.1 研究目标

鉴于元数据管理效率偏低、数据加载耗时过长、检查点写入与恢复对训练阻塞严重等瓶颈，本文将研究面向云边端跨域分布式训练的存储加速技术，具体包括面向跨域训

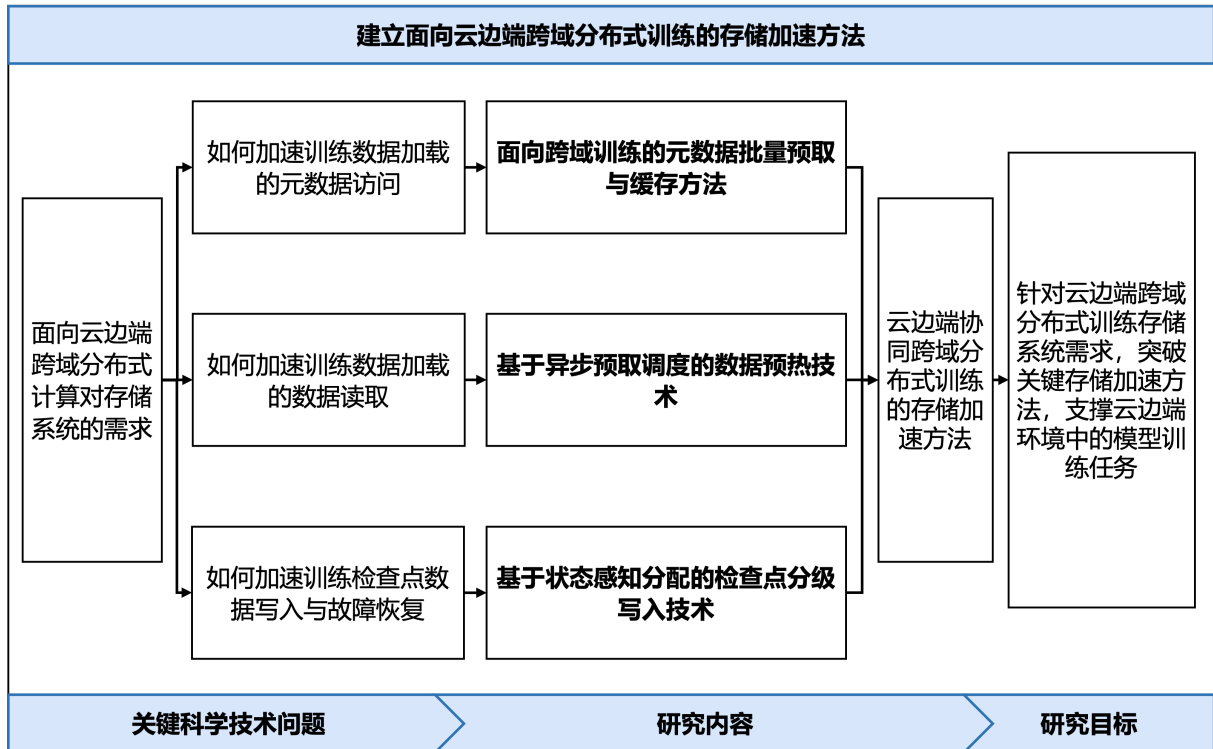


图 1.3 本文研究内容与研究目标

训练的元数据批量预取与缓存方法、基于异步预取调度的数据预热技术、以及基于状态感知分配的检查点分级写入技术。最终目标是构建一套面向云边端跨域分布式训练的存储系统，有效聚合和利用分散在云、边、端各层的存储资源，同时提升跨域分布式训练效率。本文的研究内容与技术框架如图 1.3 所示。

1.3.2 研究内容

本论文针对云边端跨域分布式训练场景，从元数据预取、数据预热、检查点写入与恢复这三个切入点研究跨域数据访问优化技术，提升云边端跨域存储系统的整体效能。论文的研究内容和工作如下所述：

(1) 面向跨域训练的元数据批量预取与缓存方法

云边端环境中，分布式跨域训练进行数据加载时，各计算节点需从分散的存储节点中获取相关元数据信息，这可能涉及大量的云边跨域访问和边边跨域访问，该过程较为耗时，显著增加了数据集初始化的启动时延。同时，同一节点的多个训练进程访问相同的数据集目录结构，产生大量重复跨域请求，导致边端侧有限的出口带宽被冗余流量占用。现有元数据管理方法缺乏对训练数据集目录结构的批量预取能力，也未充分利用多进程间元数据的复用性和训练访问序列的可预测性。

针对上述问题, ①批量目录预取: 预取引擎 (Client Prefetch Engine) 在后台异步递归遍历目录树, 将同层多个目录聚合为批量请求, 通过元数据服务器的批量 RPC 接口在单次请求中获取多个目录内容, 大幅减少跨域通信次数; ②多级缓存: 在客户端侧, 预取结果自动填充至三级缓存, 同节点多进程直接复用, 消除冗余的跨域请求; 在元数据服务器侧, 域内元数据缓存本域文件与目录属性, 命中时可避免反复访问底层持久化存储; 转发元数据缓存转发查询的远程条目属性, 命中时可避免对同一路径的重复跨域转发。

(2) 基于异步预取调度的数据预热技术

云边端环境中, 分布式跨域训练进行数据加载时, 数据成批形式组织, 在每轮迭代中均被访问一次, 会导致大量随机读取操作, 带来频繁的云边端跨域数据访问, 尤其在云边端网络带宽受限的环境下, 这种跨域数据加载会导致时延显著增加, 使得边端宝贵算力闲置。现有方法缺乏对云边端各类复杂路径的加载适配, 未能考虑云边端数据存取节点的实际状态, 导致数据加载效率低下。

针对上述问题, ①构建本地多级数据缓存, 配置容量与淘汰策略管理, 使训练读取优先命中本地快速路径; ②利用训练访问序列的可预测性, 在带宽与并发约束下, 设计前瞻窗口与任务优先级, 将远端数据提前搬入缓存; ③通过异步预取调度, 重叠跨域数据搬运与训练计算, 提升加载连续性, 缓解边侧带宽瓶颈。

(3) 基于状态感知分配的检查点分级写入技术

云边端环境中, 检查点数据在分发时可能需经历本节点与域内、跨域及云端等多级存储路径; 在边缘带宽与存储能力受限的条件下, 大体积检查点若采用统一写入与副本策略, 仍易出现较长的写入完成时间、恢复路径不确定以及存储资源利用不均衡等问题。对于边缘节点而言, 存网状态波动较大, 副本放置与冗余管理更加复杂。现有检查点写入、恢复或生成方法多面向数据中心环境, 难以充分适配边端存算约束, 也较少结合云边端多层拓扑对检查点进行差异化处理, 灵活性不足。

针对上述问题, ①构建资源模型: 基于拓扑距离分级与实时状态采集, 对边侧存储节点进行动态建模, 以反映存网状态变化; ②检查点分片与动态分配: 按重要性对检查点内容分级, 结合容量与距离等因素完成分片到节点的映射, 并在带宽或容量紧张时降级策略保持写入可控; ③副本创建与故障恢复: 关键数据优先同步落盘至就近节点并异步向远端归档, 恢复时按由近及远的阶梯尝试, 同时结合延迟感知等手段缩短恢复时间、提高资源利用效率。

1.4 主要贡献与创新之处

本论文的主要贡献与创新点如下：

1、面向云边端协同环境的跨域虚拟存储空间系统

现有通用分布式文件系统大多将底层存储视作黑盒，导致上层训练框架的读写模型与云边端跨域环境的高延迟、弱连接特征产生严重错配。本文打破了计算与存储的隔离，提出并构建了面向云边端协同环境的跨域虚拟存储空间系统 (CEDFS)。该架构将云边端跨域分布式训练全生命周期的 I/O 特征与底层跨域多级存储组件深度融合，将被动触发式的数据访问，升级为主动协同式的定制化数据供给，为云边端环境下的小模型边缘微调与协同计算提供了高效、透明的系统底座。

2、面向跨域训练的元数据批量预取与缓存方法

在云边端结构下，传统元数据访问方法未能有效应对跨域目录发现的高 RTT 放大效应。针对在云边端跨域训练启动场景，多节点、多计算进程对同一目录树的并发初始化极易引发“元数据风暴”，产生海量重复请求并持续拥塞边侧有限的出口带宽。本文深入剖析了训练启动阶段元数据访问规律，提出了面向跨域训练的元数据批量预取与缓存方法。其关键创新在于将目录树展开前移至训练启动前，通过动态批量聚合策略减少跨域请求次数，同时构建涵盖云边端多级缓存驻留架构，减少了后继轮次冗余的网络流量。实验证明，该机制突破了云边端训练冷启动的元数据访问瓶颈，不仅在弱网下有效加速了冷启动，还实现了跨 Epoch 的持续复用，减少了后续轮次的重复跨域开销。

3、基于异步预取调度的数据预热技术

现有面向深度学习的数据预取与缓存机制多基于单集群稳定互联假设，未能全面适应云边端环境中广域链路的高时延与带宽波动特征。特别是在多节点、多计算进程并发执行数据加载时，现有方法缺乏跨域视角的全局协同，极易引发底层脆弱带宽的无序竞争与拥塞，远端数据供给落后于 GPU 消费节奏。本文通过随机种子与采样策略，确定边侧各分散节点的训练序列。基于此，提出了基于异步预取调度的数据预热技术，构建动态预取窗口实现跨域加载与前台计算的精准流水化重叠；并引入带宽感知的多因子调度算法，在受限云边端链路下动态平衡资源并优先保障关键数据搬运。实验证明，该技术有效抹平了跨域高时延对训练吞吐的负面影响，在边侧有限缓存条件下，依然能够稳定地将预取命中率转化为全局训练吞吐收益，在边侧多卡与小规模协同训练场景中展现出显著优势。

4、基于状态感知分配的检查点分级写入技术

在云边端架构中，边缘节点存网状态高频波动，传统全量同步检查点方案因忽视了检查点内容的非对称重要性及跨域恢复路径的差异，极易引发长时阻塞。本文解耦了检查点内部状态，揭示了不同分片在恢复必要性与重建成本上的显著差别，并构建了云边端场景下涵盖容量、带宽与拓扑距离的多维动态资源模型。基于此，提出了基于状态感知分配的检查点分级写入技术，其关键机制在于根据数据重要性实施差异化副本策略与本地优先的异步持久化，在极端存储压力下实现低优先级数据的优雅降级；同时针对长尾故障恢复，设计了五级阶梯与动态探测优先的跨域恢复机制。实验证明，该技术有效兼顾了保存与恢复的速度，将保存阻塞时间压缩至亚秒级，并在复杂单点故障场景下实现了透明路由，稳定缩减约 60% 的训练可恢复时间。

1.5 论文组织结构

本论文共分为七章，各章内容安排如下：

第一章为绪论，阐述了论文的课题来源、研究背景与意义、研究目标与具体内容、主要贡献与创新点以及全文组织脉络。

第二章为国内外相关研究，分别围绕元数据访问优化、数据加载优化和检查点写入与恢复加速三个方向梳理国内外研究现状，并总结现有工作的适用边界与不足。

第三章介绍面向跨域训练的元数据批量预取与缓存方法。该章聚焦于云边端跨域训练，分析逐条元数据请求引起初始化耗时过长的瓶颈，设计了批量目录预取配合多级缓存的优化机制：异步递归遍历加请求聚合压缩了跨域元数据交互次数，多级缓存确保预取结果高效驻存并支持跨进程复用。考虑到该方法主要作用于训练冷启动阶段、与后续两项技术在系统层面深度耦合，其端到端实验评估与系统集成效果统一在第六章中给出。

第四章介绍基于异步预取调度的数据预热技术。该章围绕跨域数据加载与训练计算串行导致 GPU 空闲的问题展开，设计了基于确定性访问序列预计算和带宽感知调度的数据预热技术：本地内存与磁盘两级缓存加上异步预取任务调度，将数据加载与模型计算解耦为流水并行。章末以实验量化了该技术对训练吞吐量的提升效果。

第五章介绍基于状态感知分配的检查点分级写入技术。该章围绕边缘节点存网状态波动造成的写入阻塞与恢复路径不可靠问题展开，设计了状态感知的检查点分级写入方法：先构建动态资源模型，再按重要性对检查点内容实施分级并配以差异化副本策略，通过容量感知与距离优先的动态分配完成分片到目标节点的映射，并辅以分层恢复阶梯

来压缩故障恢复开销。章末以实验检验了该技术在写入效率与恢复可靠性两方面的改进效果。

第六章为云边端跨域虚拟存储空间原型系统与集成验证。该章介绍 CEDFS 原型系统的设计目标与总体架构，阐述系统部署方式及基础访问链路，将前三章提出的三项关键技术统一集成到 CEDFS 原型中，并通过端到端实验验证集成系统在带宽、时延、检查点频率与链路稳定性等多种条件下的整体效果。

第七章为总结与展望，总结全文工作与贡献，并对未来研究方向进行展望。

第二章 国内外相关研究

本章围绕云边端跨域分布式训练的存储加速技术需求，分别从元数据访问优化、数据加载优化以及检查点写入与恢复加速三个方向对国内外代表性研究进行梳理，提炼各类方法的适用边界与尚存的不足之处，为后续各章的具体方案设计提供研究背景与参照依据。

2.1 元数据访问优化研究现状

在云边端跨域分布式训练中，数据集目录与文件属性分散在多个域，目录遍历与元数据查询往往伴随大量跨域交互，显著影响训练初始化与迭代阶段的效率；已有代表性工作仍以云内分布式元数据服务与扩展性优化为主线^[15,16]，面向云边端跨域训练访问模式的专门讨论仍在发展之中。目前面向元数据访问优化的研究可归纳为两类：一类是面向云边端场景的优化方法，另一类是面向传统云集群的优化方法。前者侧重缩短跨域查询路径、在边侧组织索引与缓存以提升就近访问效率，后者侧重命名空间划分、负载均衡及元数据服务吞吐与一致性，可进一步分为访问架构优化与服务性能优化两条线索。

2.1.1 面向云边端的优化方法

面向云边端的元数据优化方法主要关注如何在云边协同环境下缩短元数据访问路径、降低跨域查询开销，并通过边侧缓存与索引机制提升高频访问命中率^[17]。在存储资源管理方面，Liu 等人^[18]研究了分层云边协同存储中的成本与延迟优化策略，Chen 等人^[19]提出了安全的云边容错存储方案，Roy 等人^[20]探讨了多层边缘云环境下的协作存储管理机制。上述工作为云边端存储资源的协同管理提供了多维度参考，但主要聚焦于通用存储服务的可靠性与效率，尚未针对分布式训练的元数据访问模式进行专门优化。

PathFS^[21]为云边端存储节点构建树状目录结构，设计了对应文件系统，将元数据从端推送至边，再由边推送至云，能有效降低边端的访问时延，但是进行边端元数据组织时会产生同步开销，当边端数据写入频繁时，边端侧需要暂停以等待元数据在层次间的传递。PhatKV^[22]提出了一种新的两级哈希索引结构，通过控制索引结构在 SSD 和内存间的移动，实现了元数据读写性能的提升，适用于边侧元数据管理。EDIndex^[23]基于计数布隆过滤器构建索引结构，为边侧快速数据查询提供了支持，提高了元数据访问性能，然而，当边缘侧产生大量元数据时，索引的更新频率随之增加，这导致布隆过滤器出现伪真的概率增大，进而增加处理开销。COIN^[24]基于网络传输成本将每个边缘节点映射到

二维坐标系中的点，构建对应权重图，以决定数据项与边缘节点之间的关系，并利用贪婪路由策略在边缘节点之间转发数据索引请求，确保了数据索引的高效放置和检索。然而，在云边端场景中，云边间存在稳定连接的情况下，COIN 的边边转发策略可能不再是最优解。

综合来看，现有面向云边端的优化方案在边侧元数据组织与就近访问方面已建立了一定基础，但对跨域训练场景的适配仍然不够充分：其一，针对目录遍历等高频访问模式的批量化请求能力不足，难以有效压缩跨域交互次数；其二，面向同节点多进程与多轮训练的缓存复用机制不完善，重复元数据请求仍较突出^[25,26]；其三，在边侧负载与网络状态波动条件下，相关机制的访问稳定性与维护开销平衡仍需进一步优化。

2.1.2 面向传统云集群的访问架构优化

云内访问架构优化方法主要关注如何组织命名空间并在多个元数据服务器间分配访问负载，以提升系统可扩展性。该类方法的主要实现方式包含静态空间分割和动态空间分割^[27]。

静态空间分割方法：基于路径名、子树大小等结构信息对元数据进行分割，不考虑实际负载变化。NFS^[28]、AFS^[29] 和 Sprite^[30] 使用静态子树分区技术，并依托于分布式文件系统将文件系统的元数据分布在不同的服务器上，将对应数据和元数据划分为多个区域，各区域采用自治进行管理。GDSS^[31] 使用静态子树分区技术将系统划分为多个逻辑域进行管理。元数据服务建立在 LDAP 协议上，每个逻辑域中拥有一个目录服务器存储文件信息，并创建一个全局命名根服务器管理所有域中的目录服务器。Gfarm^[32] 为跨域多云集群构建了全球级别的分布式存储系统，提供全局一致的文件视图，各云集群仅将其子树挂载到全局端点，未实际产生元数据的流动。此外，Tectonic^[33] 与 DIESEL^[34] 在大规模集群文件系统中引入了分层元数据与缓存架构，大幅提升了深度学习场景下的存储扩展能力^[35,36]。静态空间分割方法实现简单，具有一定的可伸缩性，不需要保证元数据一致性，但是在面对资源有限的边设备时，容易导致存储空间使用不均的问题，同时也容易产生访问热点，导致边侧网络拥堵加剧。

动态空间分割方法：根据工作负载的变化动态地在元数据服务器集群中重新分配元数据。Ceph^[37] 按照目录层次结构划分为子树，并动态地分配给元数据服务器。Ceph 的元数据服务器根据流量信息识别频繁访问的子树，并通过两阶段事务将元数据转移到非繁忙节点。Lunule^[38] 在 CephFS 的基础上，设计元数据负载均衡器，通过不平衡因子模型和工作负载感知的迁移规划器选择适合的子树并完成迁移，但是 Lunule 需要为较多元数据

服务器维护分配迁移指数,对应的信息记录开销较高。此外,IndexFS^[39]和LocoFS^[40]等工作证明了采用无状态缓存与松耦合服务可有效应对极大规模命名空间的元数据瓶颈。HopsFS^[41]、InfiniFS^[42]通过KV型数据库构建元数据管理,通过动态感知负载完成KV条目迁移,实现了海量小文件场景下的元数据弹性扩容和负载均衡。动态空间分割方法能有效平衡容量和访问负载,但是进行动态空间分割会在动态决策、副本迁移、一致性维护等方面产生一定开销,需要进一步调整以适配边端环境。

简言之,面向传统云集群的访问架构优化方法所解决的主要是通用文件系统层面的扩展性与负载均衡问题,其优化目标更多聚焦于命名空间组织和元数据服务能力本身;与分布式训练过程中的访问序列、阶段性瓶颈和训练时序耦合相对较弱,因此难以直接覆盖训练初始化、迭代加载与多轮次复用等训练关键环节的专项需求。

2.1.3 面向传统云集群的服务性能优化

云内服务性能优化方法主要关注于如何提高元数据服务性能的技术,包括元数据的缓存和复制,降低元数据同步、锁的开销等。CFS^[43]采用分层组织元数据,将命名空间与文件属性分离,并实现了单片原子原语,放松中间状态实现最终正确性,降低事务协调的开销。FlatFS+^[44]将路径名查找与其他复杂的系统操作分离,缩短了路径遍历,同时基于Br树管理平面命名空间中可变大小的索引键,加速元数据范围操作。PancakeFS^[45]利用LSM-Tree确保了高效的持久化介质写入性能,同时实现了基于子树分区分配算法,减少了路径解析访问时间。SingularFS^[46]采用无日志元数据操作,降低了保证崩溃一致性的开销,同时实现分层并发控制,使用读写锁同步inode操作,减少锁开销,提升了共享目录元数据访问的吞吐量。面向云内元数据服务的性能优化方法虽然在提升元数据访问效率和降低同步开销方面取得了积极进展,但在云边端环境下未考虑云边端结构下的元数据访问特征,未充分满足到边端计算节点对元数据访问的动态性和局部性的优化需求。

综上所述,在适用场景上,现有元数据优化方法大多建立在相对稳定的网络条件或资源条件较好的部署环境之上。其主要优势在于通过管理架构优化、负载迁移、缓存与并发控制等机制提升元数据访问效率。面向云边端跨域训练场景时,其不足主要体现在对批量元数据访问、跨进程缓存复用、跨域高时延链路适配与动态负载波动的支撑仍不充分,相关机制在资源受限环境下的维护开销与收益平衡仍需进一步优化。

2.2 数据加载优化研究现状

在云边端跨域分布式训练中，训练样本分散在云边端三层拓扑的多级存储节点上，跨域读取易与训练计算形成串行瓶颈^[47]；已有代表性工作仍以单集群或数据中心内高速缓存与相对稳定后端为主要设定^[48]。目前面向训练数据加载的优化可归纳为两类：一类是面向数据缓存的优化方法，另一类是面向数据预取的优化方法。前者侧重在高速缓冲中复用已读样本以降低对远端或慢速存储的访问，后者侧重依据对未来访问的预测将数据提前搬至快速层级以摊薄读延迟。

2.2.1 面向数据缓存的优化方法

面向数据缓存的优化方法通过在高速缓存中复用已读样本，减少对远端或慢速存储的访问，从而缩短数据就绪时间。SOLAR^[49] 依据预确定的索引列表优化全局访问顺序与缓冲驱逐策略，提高重用率与命中率，对数据并行场景较为有效，但难以推广到张量并行、模型并行等访问模式更为复杂的训练形态^[50]。SHADE^[51] 依据样本级重要性动态调整缓存与访问顺序以提升复用，但会改变既定数据管线下的访问次序，带来与原始训练分布不一致的风险，不利于严格可复现的训练。Lobster^[52] 通过性能建模与启发式缓解 I/O 与预处理倾斜，并结合重用距离优化缓存管理。DIESEL^[34] 采用任务粒度的节点内缓存与分块洗牌等策略，在控制对训练干扰的前提下提升访问效率。

上述工作的共同局限可概括为两点：一是部分方法依赖重排访问顺序以换取缓存收益，与固定随机种子下的可复现训练需权衡；二是多数工作仍以数据中心内高速缓存与相对稳定后端为主，对云边端跨域链路与边端资源受限场景考虑不足^[53,54]。

2.2.2 面向数据预取的优化方法

面向数据预取的优化方法通过未来访问的预测，将数据从慢速层级提前搬至快速层级，以摊薄读延迟。Fluid^[55] 以全局数据集进行抽象，支持预加载，将远程数据拉至邻近计算节点，进行分布式缓存，接近全量或大规模集中式搬运；当数据集规模远超边端可用空间或仅需部分样本时，适用性会变差。CrossPrefetch^[56] 通过内核态与用户态运行时的协同改善缓存可见性与预取路径，侧重通用用户态数据加载路径上的缓存协同，对跨域存储拓扑与数据平面传输策略的刻画相对间接。NoPFS^[57] 利用随机种子还原访问序列，并结合本地存储层次与分布式内存组织预取空间，与本文采用确定性序列驱动预取在思路相近；其设计语境仍以集群内相对稳定的后端与节点资源为主，对云边端多跳跨域链路、边侧水位与带宽波动下的调度约束，以及客户端两级缓冲与训练流水异步重叠等机制，仍需结合场景进一步扩展^[58]。

总之, 现有面向数据预取的优化方法在利用存储层次、缩短读路径等方面已取得积极进展, 但对云边端跨域训练场景的适配仍不充分: 其一, 全量或粗粒度预放置难以在边端容量受限条件下长期维持; 其二, 单集群或相对稳定后端假设下的预取决策, 对多跳跨域链路、远端节点负载与带宽波动缺乏细粒度感知; 其三, 与训练消费进度对齐的异步调度、跨进程共享同一缓冲视图等能力, 仍需与系统栈深度协同方能稳定发挥作用^[59]。

综上所述, 在适用场景上, 现有数据加载优化方法大多建立在单集群或相对稳定的数据访问路径假设之上。其主要优势在于通过缓存复用或提前搬运降低训练过程中的数据访问等待时间。面向云边端跨域训练场景时, 其不足主要体现在对多层级跨域路径、边端缓冲容量与链路波动联合约束的针对性不足, 对访问序列可复现性与加载优化目标之间平衡的刻画仍不充分, 以及预取与训练异步流水、本地分级缓冲协同方面仍有待系统化, 相关机制在资源受限环境下的收益与维护成本平衡仍需进一步优化。

2.3 检查点写入与恢复加速研究现状

在云边端跨域分布式训练中, 检查点的保存与恢复需跨节点与跨链路, 而已有代表性工作仍以数据中心环境为主要设定^[60,61]。目前面向检查点写入与恢复优化的研究可归纳为两类: 一类是面向检查点存储路径与写入调度的优化方法, 另一类是面向检查点生成体积压缩的优化方法。前者侧重减少写入与恢复路径开销、降低对训练吞吐的干扰, 后者侧重在一定精度损失下进行检查点体积压缩。

2.3.1 面向检查点存储的优化方法

面向检查点存储的优化方法主要面向的是如何与层级式存储系统进行交互, 以高效地保存和恢复训练状态^[62,63]。Gemini^[64] 提出了两个核心写入优化策略, 一方面提出近乎最优的检查点放置策略, 最大化从 CPU 内存中恢复失败的概率; 另一方面是检查点流量调度算法, 最小化写入流量对模型训练吞吐量的干扰。FastPersist^[65] 利用 NVMe SSD 的特性进行写入优化, 使用异步 I/O 库提高写入性能, 并将检查点操作与独立的训练计算重叠, 以隐藏检查点创建的延迟。DataStates-LLM^[66] 针对数据并行、张量并行和流水线并行模式来分割模型和优化器状态, 预分配并重用一定大小的主机内存缓冲区, 同时合并了从 GPU 到主机内存的模型和优化器片段的传输, 并使用分布式共识机制来实现检查点的异步写入, 从而最小化对训练过程的干扰。ByteCheckpoint^[67] 实现了一种不受特定并行配置限制的检查点存储机制, 从而允许在不同的并行环境设置中, 能够自动进行检查点数据的重新分片。Amber^[68] 针对大语言模型训练中检查点体积庞大的问题, 通

过增量检查点机制仅持久化相比上次保存发生变化的参数子集，显著降低了单次写入的数据量。CheckFreq^[69] 采用了两阶段的检查点机制：第一阶段在每次迭代后进行内存快照，再异步将快照写入持久存储，并结合计算过程的流水线，减少了写入时间；第二阶段通过在线分析训练作业的特征，自动调整检查点频率，以适应不同的模型和硬件环境。GoCkpt^[70] 提出了跨多个训练步重叠检查点保存的方法，将检查点状态分散在多个训练步中逐步传输至 CPU 侧，并传输部分梯度数据用于在 CPU 上恢复一致性版本，同时引入传输优化策略以最大化 GPU-CPU 带宽利用率和 SSD 持久化吞吐。该方法依赖于本地存储能力，且需要训练步长稳定，在边缘环境下网络波动频繁、存储介质异构的条件下适配性受限。FlowCheck^[71] 将检查点保存操作从训练计算流程中完全解耦，使检查点的生成与写入以独立数据流方式异步执行，避免对前向一反向计算的阻塞干扰。Universal Checkpointing (UCP)^[72] 将检查点结构与并行训练策略和硬件配置解耦，设计了基于模式匹配的重配置流水线，支持在训练过程中灵活变更并行策略，具备良好的弹性和灵活性。然而，UCP 侧重于解决并行策略变更时兼容问题，未涉及检查点在异构存储层级间的差异化放置。此外，Xu 等人^[73] 探讨了面向大规模机器学习训练的高效检查点系统设计，通过优化写入路径减少持久化开销，但同样以数据中心的高性能存储基础设施为前提^[74,75]。

总之，现有面向检查点存储的优化方法在异步写入、与计算重叠及并行解耦等方面已取得积极进展，但对云边端跨域训练场景的适配仍不充分：其一，多数方案隐含高性能本地存储与相对稳定带宽，边端介质与跨域链路条件下收益可能收窄^[76]；其二，放置与调度多以集群内吞吐与干扰最小化为目标，对多层级存储与节点状态差异的联合约束考虑不足^[77]；其三，面向并行策略变更的兼容性工作与面向跨域恢复路径的系统性设计往往分属不同脉络，仍需在真实部署约束下统筹^[78]。

2.3.2 面向检查点生成的优化方法

面向检查点生成的优化方法主要面向的是如何在保持模型性能几乎不受损失的前提下，通过压缩减少检查点自身体积，以降低训练过程中的存储开销。ExCP^[79] 框架通过计算相邻检查点的残差来提取稀疏信息，实现高压缩比，同时提出了权重-动量联合收缩方法，利用模型和优化器信息，去除冗余参数，保留关键信息以确保最优性能，并采用非均匀量化进一步压缩存储空间，有效减少了 LLM 训练检查点的大小。Inshrinkerator^[80] 根据模型权重对压缩的敏感性将权重分为全精度保留、修剪和量化三类，采用基于草图的近似 K-Means 聚类进行非均匀量化，同时动态搜索机制自动调整量化配置以适应权重

变化，并采用差分编码方案重排权重提高编码效率。QD-Compressor^[81] 先根据每层参数的值分布和权重熵自适应地确定量化器和量化比特数，然后在量化后的整数网络上计算增量数据，利用无损压缩技术进一步压缩增量数据，显著提高模型压缩比。

上述工作的共同局限可概括为两点：一是编解码与量化引入额外计算，边端算力与能耗受限时需与压缩收益权衡；二是评价多聚焦于体积与单次写入，对跨域传输与恢复时延等约束的讨论相对有限^[82]。

综上所述，在适用场景上，现有检查点优化方法大多建立在数据中心级存储与网络条件或相对稳定的后端假设之上。其主要优势在于通过异步写入与调度、放置与重配置优化或压缩编码，降低训练被打断的时间与存储占用。面向云边端跨域训练场景时，其不足主要体现在对异构节点能力、跨域链路波动与恢复路径选择的针对性仍不充分，压缩与系统级写入路径协同方面仍有权衡空间，相关机制在资源受限环境下的复杂度与实际收益平衡仍需进一步优化。

2.4 本章小结

本章围绕元数据优化、数据加载优化和检查点写入与恢复优化三条研究主线，系统梳理了国内外代表性工作，并从适用场景、核心机制和云边端适配边界三个维度进行了归纳分析。总体来看，现有方法在各自目标场景下已形成较成熟的优化思路，但在云边端跨域训练环境中，针对跨域高时延链路、边侧资源受限与动态波动、以及训练全流程协同优化等问题仍存在改进空间。基于上述分析，后续章节将分别从元数据批量预取与缓存、异步预取调度数据预热、状态感知分配的检查点分级写入三个方向展开设计与实现。

第三章 面向跨域训练的元数据批量预取与缓存方法

3.1 引言

在云边端跨域分布式训练中，首轮冷启动需要遍历分散于多个域的训练样本元数据。逐条跨域查询产生的大量 RPC 调用严重制约了训练启动效率。因此，必须优化元数据访问策略、减少跨域请求次数并构建高效的访问路径。

3.2 问题分析与研究思路

在云边端跨域分布式训练中，元数据访问性能瓶颈源于 PyTorch 训练框架的数据集初始化机制与跨域存储环境之间的错配。训练开始前，框架必须先完成目录遍历和样本索引构建；在本地文件系统中，这一路径开销通常较小，但在跨域环境中，同样的目录访问会被放大为大量远程元数据查询，直接拉长训练冷启动时间。

以 PyTorch 常用的 ImageFolder 和 DatasetFolder 为例，数据集初始化主要包含类别发现和样本索引构建两个阶段：前者通过 `os.scandir()` 扫描根目录并筛选类别目录，后者对各类别目录依次执行 `os.walk()` 展开目录树并枚举样本文件。以 ImageNet 数据集为例，本地文件系统上这类调用通常只需数秒即可完成；然而在 CEDFS 的基础访问链路中，每一次 `readdir` 或 `listdir` 都可能经由 FUSE 转化为一次跨域 gRPC 查询，若目标元数据不在本域，还需由元数据服务器继续转发至其他域。以包含 1 万个文件的数据集子集为例，若跨域往返延迟为 100 ms，仅元数据遍历就可能消耗约 400 秒，扩展到完整 ImageNet 时初始化耗时将达到数小时。

这一问题在云边端分布式训练场景下会进一步放大。一方面，数据集初始化发生在 DistributedSampler 切分样本之前，每个 rank 都需要独立完成对完整目录树的遍历，导致同节点多进程对同一目录结构发起重复请求；另一方面，在同数据集多任务训练和多轮次训练场景下，不同任务和不同轮次虽然访问顺序可能变化，但所依赖的目录结构、文件属性和路径映射基本保持不变，现有方法难以利用这种稳定的元数据复用机会。

基于上述分析，本章聚焦三个核心问题：①如何将逐目录的跨域元数据查询压缩为少量批量请求，以减少初始化阶段的网络往返；②如何在 PyTorch 目录遍历真正开始前利用后台时间窗口，尽可能提前完成目录树元数据拉取；③如何使首次获取的元数据在多进程、多任务和多轮次之间持续复用，避免重复跨域访问。下文据此围绕批量目录聚合、客户端递归预取和多级缓存驻留三个方面展开方案设计与实现。

3.3 详细方案设计与实现

3.3.1 符号描述

本章所描述的面向跨域训练的元数据批量预取与缓存方法涉及对云边端跨域环境中元数据访问过程的建模和分析。为简化后续模块设计和算法描述中的参数推导过程，本章方法所涉及数据特征及其描述如表 3.1 所示。

表 3.1 面向跨域训练的元数据批量预取与缓存方法的相关符号描述表

符号	定义
D	数据集，包含所有训练样本文件的集合
D_r	数据集根目录路径
N	数据集中样本文件总数， $N = D $
$P = \{p_1, p_2, \dots, p_m\}$	数据集包含的目录路径集合， m 为目录总数
M	客户端连接的元数据服务器
d	预取深度，控制递归目录遍历的最大层数
β	批量请求块大小，每次 BatchReadDir RPC 最多包含的目录数
γ	最大并发预取任务数，控制同时活跃的异步遍历任务上限
B_j	第 j 个批量请求块，包含至多 β 个目录路径
RTT_{cross}	跨域请求的平均往返延迟

3.3.2 整体方案设计

基于前文对元数据访问瓶颈的分析，面向跨域训练的元数据批量预取与缓存方法整体上由客户端预取控制链路和服务端批量访问支撑链路协同完成，如图 3.1 所示。客户端侧负责触发预取、组织目录遍历并维护本地分层缓存；元数据服务器侧提供单目录与批量目录读取原语，并结合服务端缓存降低本域查询与跨域转发开销。整个方案的目标是在数据集初始化阶段尽可能一次性完成元数据拉取，并将结果持续复用于后续轮次。

整体工作流程：训练任务初始化数据集时，Python SDK 在后台触发预取引擎，由客户端核心递归调用目录读取接口获取目录树元数据并写入本地分层缓存。随后，PyTorch 基类的目录遍历优先命中本地缓存完成数据集初始化，未命中时再回退至常规跨域路径。首次预取完成后，后续轮次以及同节点多进程共享同一份缓存结果，无需重复发起大规模跨域元数据请求。

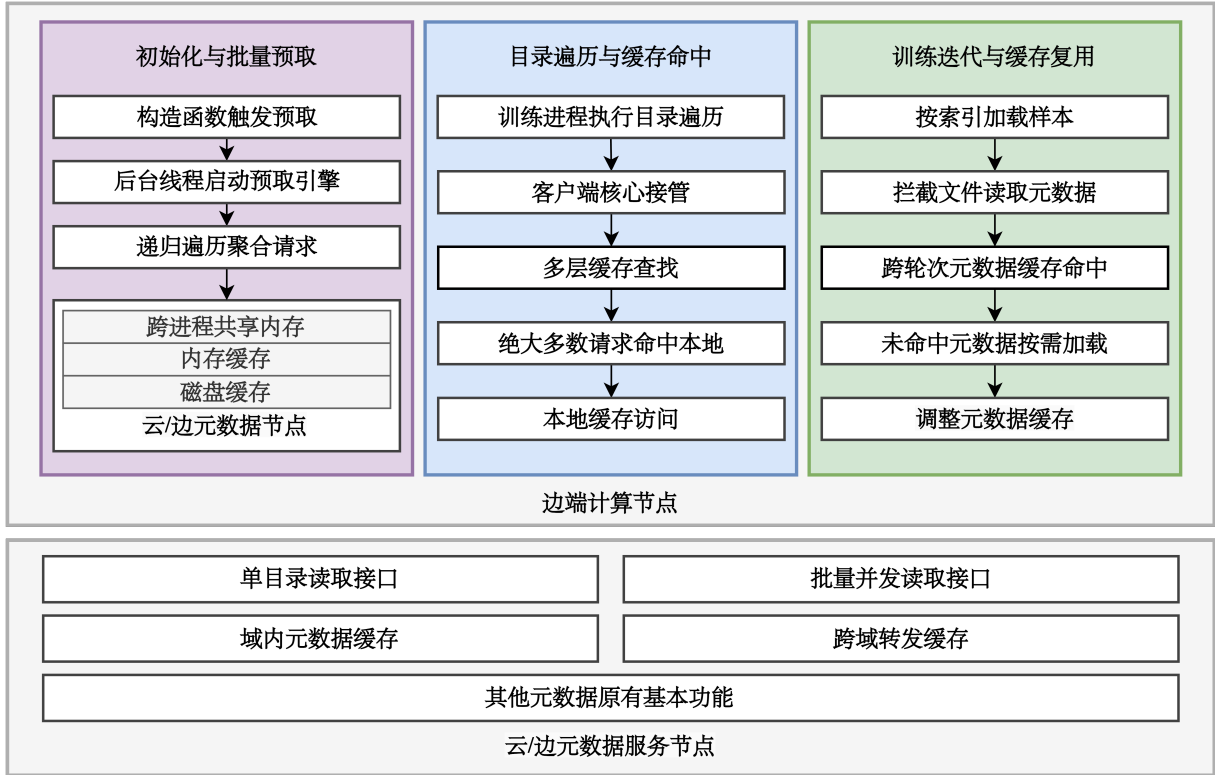


图 3.1 面向跨域训练的元数据批量预取与缓存方法整体架构

客户端侧设计：客户端侧由 Python SDK 接入端、预取引擎与分层缓存组成，负责触发预取、组织递归遍历并管理本地缓存驻留；具体的遍历策略、缓存层次与命中流程见第 3.3.3 节和第 3.3.4 节。

元数据服务器侧设计：元数据服务器侧提供 GetDir 与 BatchReadDir 两类接口，并利用域内缓存和转发缓存协同降低本域查询与跨域转发开销；服务端批量处理与缓存机制的具体实现见后续小节。

3.3.3 基于批量聚合的目录元数据预取

在基础的元数据访问模式下，每个目录的读取需要一次独立的 GetDir RPC 调用。对于包含 m 个目录的数据集，目录遍历阶段需发起 m 次独立的跨域 RPC 调用，总延迟为 $O(m \cdot \text{RTT}_{\text{cross}})$ 。为减少跨域通信次数，本方法在客户端核心引入批量目录聚合机制，将同一遍历层级内的多个目录路径打包为一个批量请求，通过元数据服务器的 BatchReadDir 接口在单次 RPC 中完成多个目录的元数据获取。图 3.2 展示了客户端与服务端端的批量目录读取处理流程。

批量读取模块接收一组目录路径 $P_{\text{level}} = \{p_1, p_2, \dots, p_l\}$ 后，按照批次大小 β （默认值 $\beta = 50$ ）将路径集合划分为多个批次块 $B_1, B_2, \dots, B_{\lceil \frac{l}{\beta} \rceil}$ ：

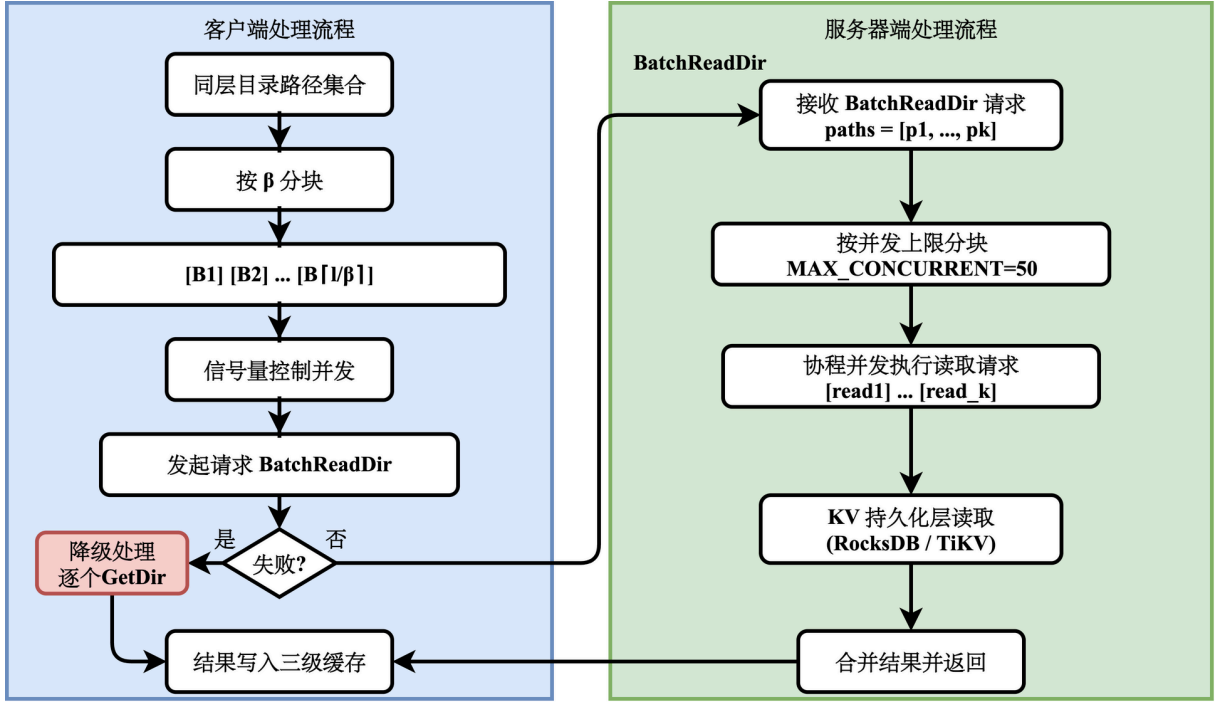


图 3.2 批量目录读取处理流程

$$B_j = \{p_{(j-1) \cdot \beta + 1}, p_{(j-1) \cdot \beta + 2}, \dots, p_{\min(j \cdot \beta, l)}\}, \quad j = 1, 2, \dots, \left\lceil \frac{l}{\beta} \right\rceil \quad (3.1)$$

每个批次块 B_j 作为一次 BatchReadDir gRPC 请求发送至元数据服务器 M 。这一分块策略将原本 l 次独立的 RPC 调用压缩为 $\left\lceil \frac{l}{\beta} \right\rceil$ 次批量请求，通信次数降低为原来的 $\frac{1}{\beta}$ 。

在并发控制方面，客户端使用信号量机制控制同时进行的批量 RPC 数量：每个批次块在发送前需获取信号量许可，确保并发请求数不超过预设上限，避免对元数据服务器造成过大的瞬时压力。每次批量 RPC 调用均通过重试机制包裹，采用指数退避策略处理网络抖动和服务端暂时过载的情况。

在元数据服务器侧，BatchReadDir 请求处理模块接收到包含多个目录路径的批量请求后，将各路径分配给并发执行的异步任务，每个任务独立完成单个目录的读取操作，即从 KV 持久化层获取该目录下的所有条目并返回。服务端的并发处理使得批量请求的延迟近似于单个目录读取的延迟，而非各目录延迟之和，实现了服务端的请求级并行。

当某个批次块的 BatchReadDir RPC 调用失败时，客户端自动降级为对该块中的每个目录路径逐个发起独立的 GetDir 请求，确保预取过程的容错性。该降级策略在网络不稳定的云边端环境中尤为重要：即使批量接口暂时不可用，系统仍能通过单路径方式完成元数据获取，仅性能有所下降而不影响正确性。

3.3.4 客户端递归遍历与多级缓存

客户端递归遍历与多级缓存环节是元数据预取方法的核心执行机制。预取引擎在客户端侧驱动目录树的递归遍历，通过异步并发机制高效获取各层级目录的元数据，并将结果自动填充至多级缓存，供后续的 PyTorch 目录遍历和训练过程中的文件访问直接命中。

预取引擎的遍历策略由数据集类型决定，系统针对不同类型内置了对应的预设策略：ImageFolder 策略采用深度优先遍历，预取深度 $d = 2$ ，最大并发任务数 $\gamma = 100$ ，适用于标准图像分类数据集的两层目录结构（类别目录—样本文件）；DatasetFolder 策略同样采用深度优先，预取深度可由用户指定；BreadthFirst 策略采用广度优先遍历，优先利用批量请求接口，适用于目录层数较浅但每层目录数量较多的数据集。图 3.3 以 ImageFolder 类型的示例目录结构为例，对比展示了两种策略的遍历过程。

在深度优先模式下，预取引擎从数据集根目录 D_r 开始递归遍历。对于当前目录，引擎首先通过信号量获取并发许可，然后调用元数据服务器的目录读取接口获取该目录下的所有条目，将条目区分为文件条目和子目录条目。对于获取到的子目录集合，引擎释放当前持有的信号量许可后，对每个子目录创建独立的异步任务并行处理：每个任务克隆预取引擎的状态（包括元数据获取器、信号量引用和统计计数器），递归调用深度优先遍历函数。信号量初始容量为 γ ，控制同时活跃的遍历任务数不超过上限，避免当目录树规模庞大时产生过多并发网络请求。当遍历深度超过预设的最大深度 d 或检测到超时信号时，引擎停止向更深层递归，保证预取过程有界可控。

在广度优先模式下，预取引擎维护当前层级的目录集合 L_{curr} ，初始为 $L_0 = \{D_r\}$ 。对于每一层级，引擎检查该层的目录数量：若数量大于 1 且底层支持批量请求，则将 L_{curr} 中的目录路径按批次大小 $\beta = 50$ 分组，每组通过 BatchReadDir 批量接口一次性获取全部内容；若批量请求失败，自动降级为使用信号量控制的并发单目录读取。将每个目录读取结果中的子目录收集到下一层级集合 L_{next} 中，然后令 $L_{\text{curr}} = L_{\text{next}}$ ，深度加 1，重复该过程直至达到深度限制 d 或当前层级无子目录。

预取引擎获取的元数据通过客户端与服务器的多级缓存协同管理，覆盖从元数据服务器到客户端进程的完整访问链路。

在客户端侧，缓存层采用两级协同架构。底层包含内存缓存与磁盘持久化缓存，二者共同构成进程内两层元数据缓存（TieredMetadataCache）：第一级内存缓存以路径字符串为键存储包含属性、数据块位置和目录条目列表的统一元数据条目，提供微秒级访问延迟；第二级磁盘持久化缓存以相同键结构将内存中的元数据溢出至本地磁盘，进程重

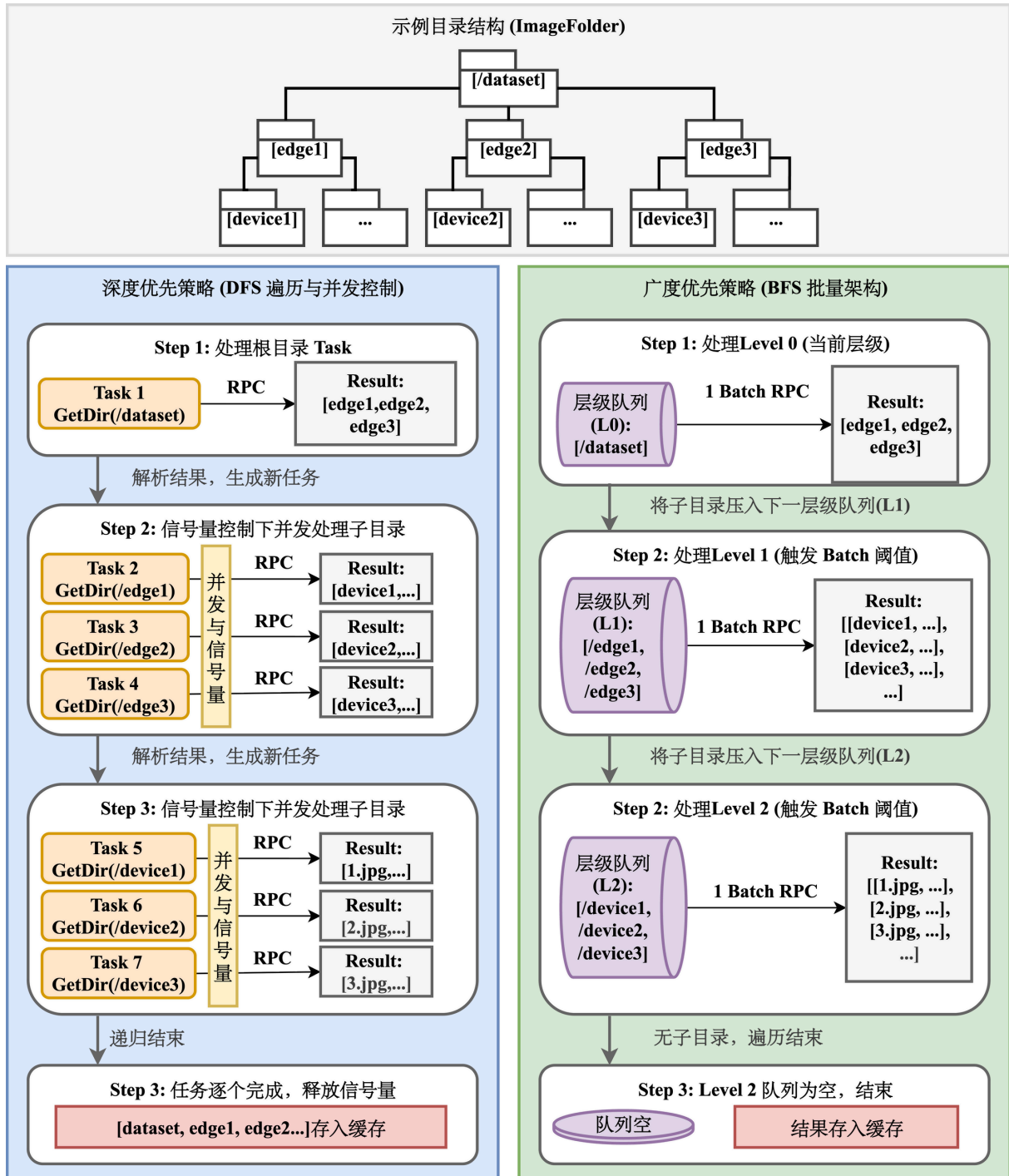


图 3.3 深度优先与广度优先遍历策略对比

启后仍可保留, 一次查找即可直接获取完整元数据信息, 同时维护以 inode 编号为键的辅助索引以支持按 inode 反查路径。最上层为跨进程 IPC 接入层, 通过命名共享内存连接独立的缓存服务进程, 使同节点多个训练进程可直接复用同一份缓存数据, 有效避免多进程重复发起跨域请求的问题, 该共享缓存同时服务于元数据和训练数据的缓存。当预引擎调用元数据服务器的 GetDir 接口获取某目录内容时, 客户端核心首先在缓存层中执

行查找（首选命中最快的第一级内存缓存，当内存满时依次溢出至第二级磁盘），命中则直接返回，未命中则通过 gRPC 发起实际请求，获取成功后通过异步任务将结果写入各级缓存，写入操作不阻塞当前预取流程。缓存层实现了基于 LRU 策略的淘汰机制，并提供主动失效接口，当客户端执行文件移动、重命名等写操作时主动清除涉及路径和 inode 的缓存条目，保证缓存一致性。

在元数据服务器侧，两级缓存加速了元数据的读取和跨域转发过程。第一级为域内元数据缓存（MetaCache），以路径为键在内存中缓存本域文件和目录的属性及数据块位置信息，采用基于时间戳的过期策略和 LRU 淘汰机制，后台清理任务定期扫描并移除已过期条目。对于高频访问的热点元数据（如数据集根目录和类别目录的属性），域内缓存避免了重复读取 KV 持久化层的事务开销。第二级为转发元数据缓存（TransferCache），用于缓存跨域转发操作的结果。在聚合目录树中，当 GetDir 操作遍历目录遇到远程条目（TypeRemoteFile 或 TypeRemoteDirectory）时，需要通过 TransferOp 连接目标域的元数据服务器获取其属性。转发缓存将跨域获取的结果缓存在本地内存中，按类型设置差异化的 TTL：路由缓存的 TTL 较长，属性和块位置缓存的 TTL 较短以保证文件变更时及时失效。后台周期性任务负责清理已过期条目，当缓存超过内存配额时通过 LRU 策略淘汰。转发缓存使得 GetDir 操作处理远程条目时，可并发查询转发缓存，命中则直接使用缓存的远程属性，未命中才发起跨域 RPC 并将结果回填缓存，大幅减少了聚合目录树场景下的跨域转发次数。客户端侧与元数据服务器侧的多级级联缓存架构如图 3.4 所示。

3.3.5 整体执行流程分析

综合第 3.3.2 节至第 3.3.4 节的设计，本方法的端到端执行流程可分为三个阶段。

第一阶段为数据集初始化与元数据预取。训练任务创建数据集实例时，构造函数在调用 PyTorch 基类之前在后台线程中触发元数据预取（第 3.3.2 节）。预取引擎根据数据集类型选择遍历策略（第 3.3.4 节），通过深度优先或广度优先方式递归遍历目录树，利用批量聚合机制（第 3.3.3 节）减少跨域 RPC 次数。遍历获取的元数据自动写入客户端分层元数据缓存和跨进程共享内存缓存；在元数据服务器侧，域内缓存和转发缓存协同加速本域与跨域的元数据查询。整个预取过程与 PyTorch 的其他初始化操作并行执行，若因超时或异常未完成则优雅降级而不阻断训练流程。

第二阶段为 PyTorch 目录遍历与缓存命中。预取引擎启动后，PyTorch 基类执行目录遍历所发起的 `os.scandir()` 和 `os.walk()` 等 POSIX 调用通过 FUSE 转发至客户端核心，在分层缓存中查找元数据。如第 3.3.2 节所分析，绝大多数目录遍历请求直接命中本地内存缓

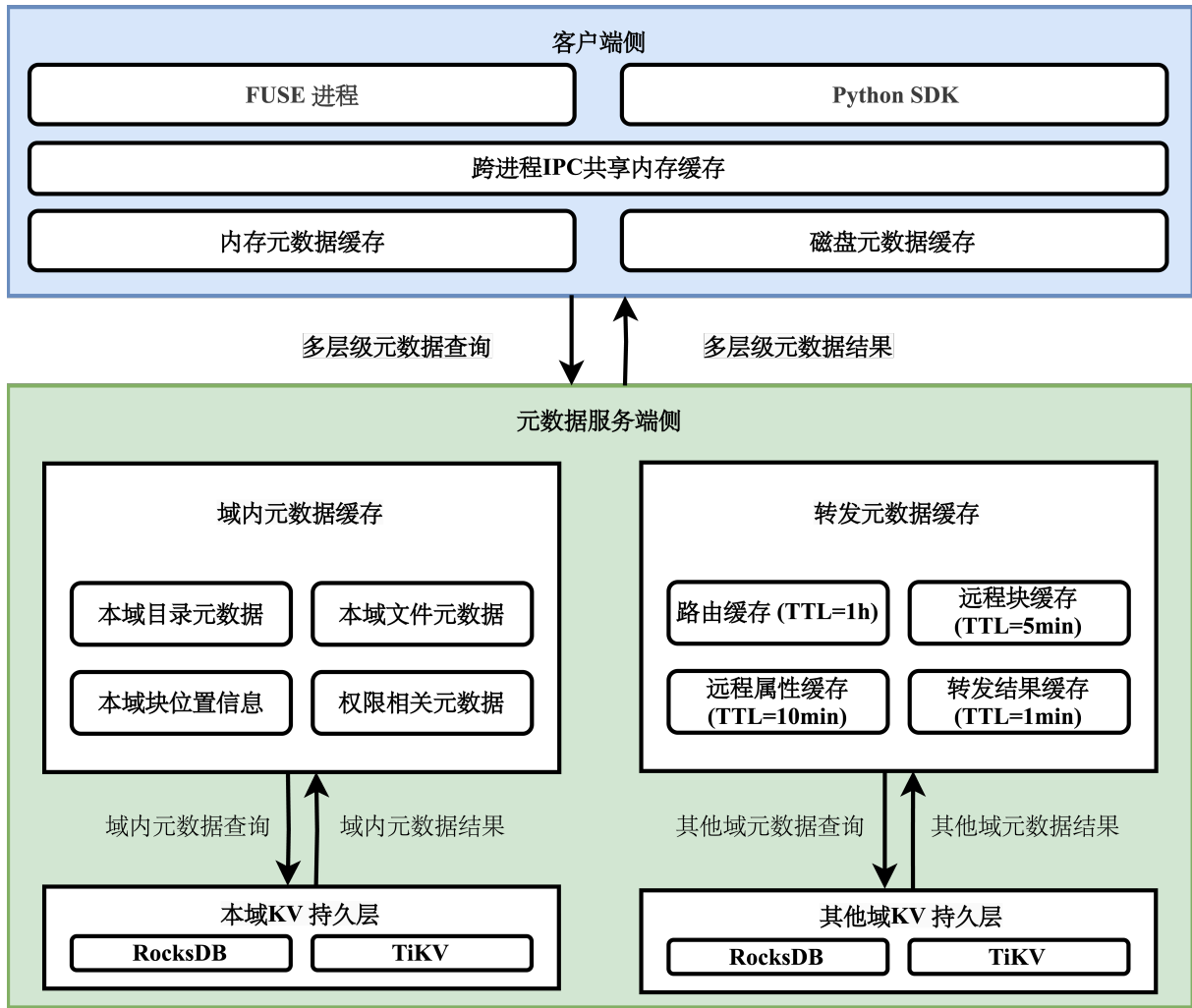


图 3.4 多级级联缓存架构

存，无需发起跨域网络请求。同节点的多个训练进程通过 IPC 共享内存缓存复用同一份预取结果，消除了多进程重复预取的通信开销。

第三阶段为多轮次训练中的缓存持续复用。数据集的元数据在整个训练生命周期内保持不变，首个轮次预取的缓存结果可在后续所有轮次中持续命中。客户端缓存通过可配置的长 TTL 实现元数据长期驻留，磁盘持久化层支持进程重启后的缓存保留（第 3.3.2 节），使得后续轮次的目录遍历均直接命中本地缓存，元数据访问延迟从跨域 RPC 的毫秒级降至本地缓存查找的微秒级。

3.4 实验与验证

本节通过系统的实验验证元数据批量预取与缓存方法的有效性。

3.4.1 实验环境

硬件配置：本章实验在阿里云弹性计算服务（ECS）集群上进行，集群由 10 台虚拟机组成。硬件配置如表 3.2 所示。

表 3.2 第三章实验的硬件与软件环境

配置项	参数
处理器	Intel Xeon Platinum（经济型 e 共享实例）
节点规格	云端 8vCPU/16GB；边缘元数据 4vCPU/8GB；训练客户端节点 2vCPU/2~4GB
磁盘	40GB ESSD Entry 云盘
网络	VPC 内网带宽上限：同 VPC 延迟~0.17ms，跨 VPC 延迟~0.6ms
操作系统	Ubuntu 24.04.4 LTS
内核版本	6.8.0
Rust 版本	1.82.0 (nightly)
Python 版本	3.12.3
PyTorch 版本	2.10.0+cu128

表 3.3 第三章实验的节点部署拓扑

节点	规格	内网 IP	VPC	所属域	角色
PM-1	8C16G	172.31.147.125	VPC-A	Cloud	Cloud Meta、Cloud Data
PM-2	4C8G	172.31.147.130	VPC-A	Edge-A	Edge Meta-A、Data-A1
PM-3	2C4G	172.30.219.57	VPC-B	Edge-A	Data-A2、Client-1（主客户端）
PM-4	2C2G	172.31.147.126	VPC-A	Edge-B	Edge Meta-B
PM-5	2C4G	172.30.219.56	VPC-B	Edge-B	Data-B1、Client-2
PM-6	2C4G	172.30.219.59	VPC-B	Edge-A	Client-3（多进程测试）
PM-7	2C4G	172.30.219.58	VPC-B	Edge-B	Data-B2、Client-4（跨域客户端）
PM-8	2C2G	172.31.147.127	VPC-A	Edge-A	Client-5（轻量端设备）
PM-9	2C2G	172.31.147.129	VPC-A	Edge-B	Client-6（跨域纯客户端）
PM-10	2C2G	172.31.147.128	VPC-A	Edge-B	Client-7

网络部署拓扑：实验采用三域部署模型模拟云边端跨域环境，如表 3.3 所示。该拓扑与真实云边端场景对齐：云端资源充裕，承担全局元数据服务和数据存储；边缘域资源受限，各自部署边缘元数据子服务器以减少跨域查询；训练客户端分散于不同边缘域，需跨域访问元数据和数据。10 台节点分布在两个 VPC 中，VPC 间通过阿里云对等连接互通，基础跨 VPC 延迟约 0.6ms，同 VPC 延迟约 0.17ms。云端（Cloud Zone）部署 1 台节点承担云端元数据根服务器（Cloud Meta）和数据服务器（Cloud Data）角色；边缘域 A（Edge-A）部署 4 台节点，包含边缘元数据子服务器（Edge Meta）、数据服务器（Data）

和训练客户端节点 (Client); 边缘域 B (Edge-B) 部署 5 台节点, 包含边缘元数据节点、数据服务器和训练客户端节点。

表 3.3 中各角色含义如下: Cloud Meta 为云端全局元数据根服务器, 负责跨域目录分区与路由; Edge Meta 为边缘域本地元数据子服务器, 缓存并服务本域元数据请求; Data 为数据服务器, 负责存储和提供文件数据块; Client 为训练客户端, 运行 CEDFS FUSE 客户端与训练框架。

跨域网络延迟通过 Linux 内核流量控制工具 tc 与网络模拟器 netem^[83] 模拟, 采用 HTB^[84] (Hierarchical Token Bucket) 队列规则进行带宽整形, netem 进行延迟和抖动注入。网络模拟配置如表 3.4 所示。

表 3.4 第三章实验的网络模拟配置

链路	延迟	带宽上限
域内 LAN (同 Zone 节点间)	1ms	400 Mbps
Cloud - Edge WAN	25ms±10ms	80 Mbps
跨边缘域 Edge-A - Edge-B WAN	50ms±20ms	40 Mbps
同机 Loopback	~0ms	不限

测试方法: 冷缓存基准测试: 每轮测试前, 强制终止所有 CEDFS 服务进程并等待资源释放, 随后按序重启元数据服务器和数据服务器, 确保服务端内存缓存、转发缓存和客户端各级缓存 (包括 Linux 内核页缓存, 通过 `echo 3 > /proc/sys/vm/drop_caches` 清除) 均被完全清空。每组配置重复运行 5 次取均值。

对照组设置: 实验设两种客户端配置。Full 配置 (完整方案) 启用批量目录预取、BFS 遍历和三级缓存 (内存、磁盘、IPC 共享内存); Bypass 配置作为基线对照, 关闭所有预取和缓存, 每次操作均通过独立 gRPC 请求逐条发送, 代表未优化的元数据访问模式。

表 3.5 第三章实验的合成数据集

数据集	类别数	每类文件	目录数	文件总数	用途
syn-100c-50f	100	50	101	5,000	标准消融 (5K 规模)
syn-1000c-50f	1,000	50	1,001	50,000	大规模压力测试 (50K 规模)

tc/netem 网络模拟: 跨域延迟通过 tc 在各节点网络接口上配置 HTB^[84] 根队列实现, 按目标 IP 的 u32 过滤器将流量分入不同 class, 每个 class 挂载 netem^[83] 叶子队列注入延

迟和带宽限制。延迟采用正态分布抖动模拟 WAN 链路波动。RTT 敏感性实验中通过 tc qdisc change 动态调整延迟参数，无需重启服务。

数据集说明：本章元数据预取实验采用合成数据集进行消融分析，如表 3.5 所示。

之所以采用合成数据集而非 ImageNet 等真实图像数据集，基于以下考虑：一方面，元数据预取实验仅涉及目录结构遍历和文件属性查询，不涉及文件内容读取，数据集中文件的实际内容不影响实验结果；另一方面，合成数据集可精确控制类别数和文件数，消除真实数据集中类别分布不均匀对结果的干扰。合成数据集采用标准 ImageFolder 两层目录结构，与真实数据集的目录布局一致，能忠实反映元数据访问模式。syn-100c-50f（101 个目录、5,000 个文件）模拟中等规模场景，syn-1000c-50f（1,001 个目录、50,000 个文件）模拟 ImageNet-val 量级的大规模目录遍历场景。

关键评测指标：本章实验涉及的关键评测指标及其含义如表 3.6 所示。

表 3.6 第三章实验的关键评测指标

指标	含义
ReadDir 耗时	执行目录列举操作（readdir）的总耗时
GetAttr 耗时	对数据集所有文件执行属性查询（stat/getattr）的总耗时
加速比	Bypass 基线耗时与 Full 优化方案耗时之比
遍历耗时	客户端完成整个数据集目录树遍历的端到端耗时
Epoch 遍历耗时	单个训练轮次中元数据遍历的总耗时

实验参数配置：CEDFS 各组件启动时通过 TOML 配置文件注入运行参数，影响实验结果的关键参数如表 3.7 所示。

3.4.2 多级缓存持续加速能力验证

本组实验验证多级缓存机制在多轮次训练中是否按预期工作，即首轮预取后后续轮次能够持续命中本地缓存。

为此在 syn-1000c-50f 数据集上采用默认 BFS 遍历策略，连续执行 5 个 Epoch，分别测量 Full 模式和 Bypass 模式每轮的元数据遍历耗时，如图 3.5 所示。

图 3.5 显示 Full 模式在 Epoch 0 耗时 5,110ms，Epoch 1 起稳定在约 430ms，冷热切换加速约 12 倍；Bypass 模式各轮恒定在约 89 秒，每轮均重复发起全部 gRPC 请求。Full 模式首轮即获得约 17.4 倍加速，后续轮次获得约 208 倍加速，表明 Epoch 0 完成批量预取后，目录条目和文件属性已持久化至本地缓存，后续轮次直接命中内存缓存而无需网络请求，三级缓存架构按预期实现了跨轮次复用。

表 3.7 第三章实验的关键参数配置 (Full 模式)

参数	描述	默认值
cache.enabled	全局缓存开关	true
cache.batch_metadata.enabled	批量元数据请求开关	true
cache.batch_metadata.batch_size	BatchReadDir 每批最大目录数 (β)	50
cache.batch_metadata.concurrency	最大并发批量操作数	50
cache.batch_metadata.timeout_ms	批量请求聚合超时	100
cache.tiered_metadata.enabled	分层元数据缓存开关	true
cache.tiered_metadata.memory_capacity	内存元数据缓存配额	512MB
cache.tiered_metadata.attr_ttl_secs	属性条目 TTL	30s
cache.tiered_metadata.entries_ttl_secs	目录条目 TTL	15s
cache.ipc.enabled	IPC 共享内存缓存开关	true
MAX_CONCURRENT_BATCH_READDIR	服务端 ReadDir 批量并发上限	50
MAX_CONCURRENT_BATCH	服务端 GetAttr 批量并发上限	100
TRANSFER_CONNECT_TIMEOUT	跨 Meta 节点连接超时	10s
TRANSFER_REQUEST_TIMEOUT	跨 Meta 节点请求超时	30s

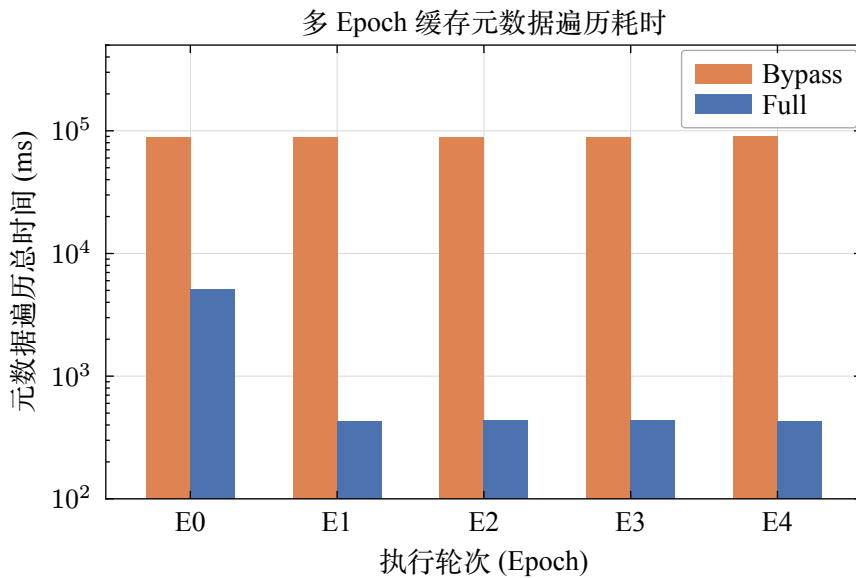


图 3.5 多 Epoch 缓存元数据遍历耗时对比

3.4.3 批量聚合效果与参数敏感性实验分析

本组实验验证批量预取的核心加速能力及批量块大小 β 的参数敏感性。

为了量化批量聚合相比逐条请求的性能提升幅度，在 Edge-A 本地部署下分别对 ReadDir 和 GetAttr 两类操作进行对比测试，设置 $\beta = 100$ ，使用 syn-100c-50f 和

syn-1000c-50f 两种规模的数据集, 通过对比 Full 模式与 Bypass 模式的耗时差异来衡量批量聚合的加速效果, 如图 3.6 所示。

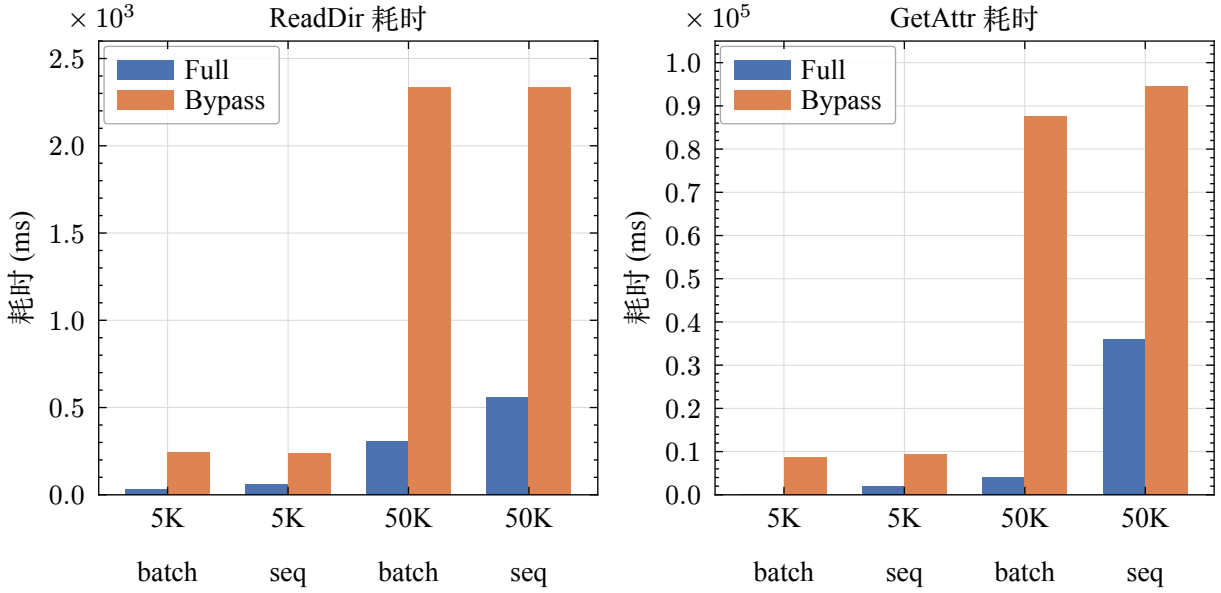


图 3.6 批量预取与逐条请求耗时对比

图 3.6 直接反映了批量聚合对两类元数据操作的加速幅度。对于 ReadDir 操作, Full 模式在 5K 文件规模下耗时 31.35ms, 而 Bypass 为 243.72ms, 加速约 7.8 倍; 50K 文件规模下耗时 307.25ms, Bypass 为 2337.59ms, 加速约 7.6 倍。对于 GetAttr 操作, 加速效果更为显著: 5K 文件规模下 Full 模式仅需 9.39ms, Bypass 高达 8598.29ms, 加速约 916 倍; 50K 文件规模下耗时 4006.30ms, Bypass 为 87508.40ms, 加速约 21.8 倍。该图间接说明了两类操作加速差异的根源: ReadDir 单次 RPC 返回完整目录条目列表, 操作本身已具有一定聚合性; GetAttr 为每个文件发起独立的属性查询, Bypass 模式下 50,000 个文件需发起 50,000 次独立 gRPC 请求, 批量聚合将其压缩为少量批次, 因此加速效果远高于 ReadDir。

为了探究批量块大小 β 对加速效果的影响规律, 在冷缓存条件下扫描 $\beta \in \{10, 20, 50, 100, 200\}$ 五组取值, 分别测量 ReadDir 和 GetAttr 操作的加速比, 通过观察加速比随 β 变化的趋势来确定推荐的参数区间, 如图 3.7 所示。

图 3.7 直接展示了 β 与加速比的关系曲线。ReadDir 操作的加速比在 5K 文件规模下于 $\beta = 20$ 时达峰值 8.32 倍, 随后维持在 7.2~8.2 倍; 50K 文件规模下趋势类似, $\beta = 10$ 时最高为 7.96 倍, 大 β 值下略降至 5.4~5.5 倍。GetAttr 操作则呈相反趋势: 加速比随 β 单调递增, 5K 文件规模下从 3.01 倍上升至 4.72 倍, 50K 文件规模下从 2.82 倍上升至 4.22 倍。该图间接揭示了两类操作的聚合收益差异: ReadDir 在较小 β 即可充分减少 RPC 次数, 过大的 β 增加单次响应时间但收益递减; GetAttr 每文件一次查询, β 越大, 单次批量能聚合的

查询越多, 边际收益持续为正。值得注意的是, β 并非越大越好, 原因有二: 一方面是过大的 β 会导致服务端单次批量请求的处理延迟显著增加, 元数据服务器需要在一次 RPC 中处理更多子目录查询并聚合响应, 尾延随之上升; 二是 β 增大使单次 gRPC 响应的数据量成比例增长, 在云边端带宽受限的跨域链路上传输时间增加, 同时服务端需为更大批次分配更多内存来聚合响应, 当 β 超过单层实际目录数量时, 额外的资源开销无法换来 RPC 次数的进一步减少。

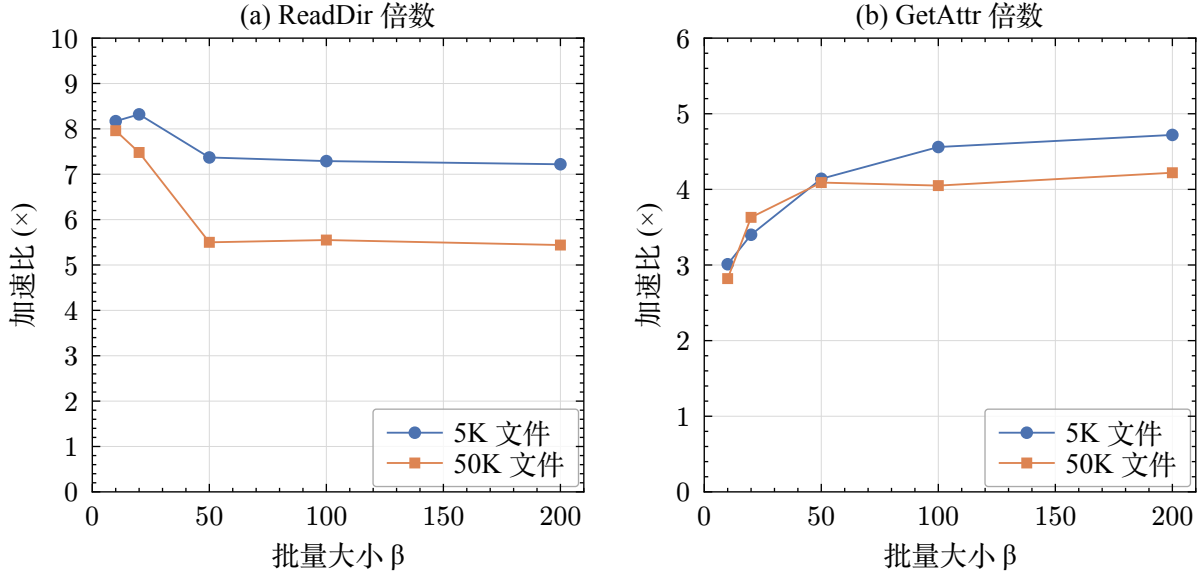


图 3.7 批量块大小 β 对 ReadDir 和 GetAttr 加速比的影响

批量预取显著降低元数据访问延迟, ReadDir 操作获得 7~8 倍加速, GetAttr 操作在大规模数据集上获得约 22 倍加速, 验证了批量聚合机制的核心加速能力。 β 的最优取值与操作类型和数据规模相关, ReadDir 在较小 β 即趋于饱和, GetAttr 则随 β 增大持续提升, 不存在普适的固定最优值, 实际部署时应结合目标工作负载的操作构成和网络条件进行针对性调优。

3.4.4 跨域网络延迟下的加速优势实验分析

本组实验验证批量预取在云边端高延迟跨域环境下的加速优势是否随延迟增大而增强。

为了分析网络延迟对批量聚合加速效果的影响, 通过 tc/netem 动态调整 Cloud-Edge 链路 RTT, 在 0ms 至 200ms 范围内扫描多组延迟值, 设置 $\beta = 100$ 且带宽不做限制以隔离延迟因素, 分别记录 ReadDir 和 GetAttr 操作的耗时与加速比, 如图 3.8 所示。

图 3.8 直接展示了加速比随 RTT 增长的变化趋势。对于 ReadDir 操作, 逐条请求耗时与 RTT 呈线性增长, 从 0.226 秒增至 31.06 秒, 而批量请求耗时增长缓慢, 从 0.025 秒

增至 1.018 秒，加速比从 RTT=0ms 时的 8.88 倍持续上升至 RTT=200ms 时的 30.51 倍。对于 GetAttr 操作，加速比在本地即 RTT=0ms 时极高达 1,241 倍，RTT 增加至 5ms 后稳定在约 50 倍。该图间接揭示了批量聚合的延迟缩减机制：RTT 每增加 1ms，逐条请求的额外延迟惩罚为 $N \times 1\text{ms}$ ，其中 N 为请求次数，而批量请求的额外惩罚仅为 $\lceil \frac{N}{\beta} \rceil \times 1\text{ms}$ ，延迟放大因子从 N 缩减至 $\lceil \frac{N}{\beta} \rceil$ ，在 $\beta = 100$ 时缩减约 100 倍。GetAttr 在 RTT 大于 5ms 后稳定在约 50 倍加速，是因为 RPC 次数从 5,000 次降至约 50 次即 $\lceil \frac{5000}{100} \rceil$ ，延迟惩罚被等比缩减。

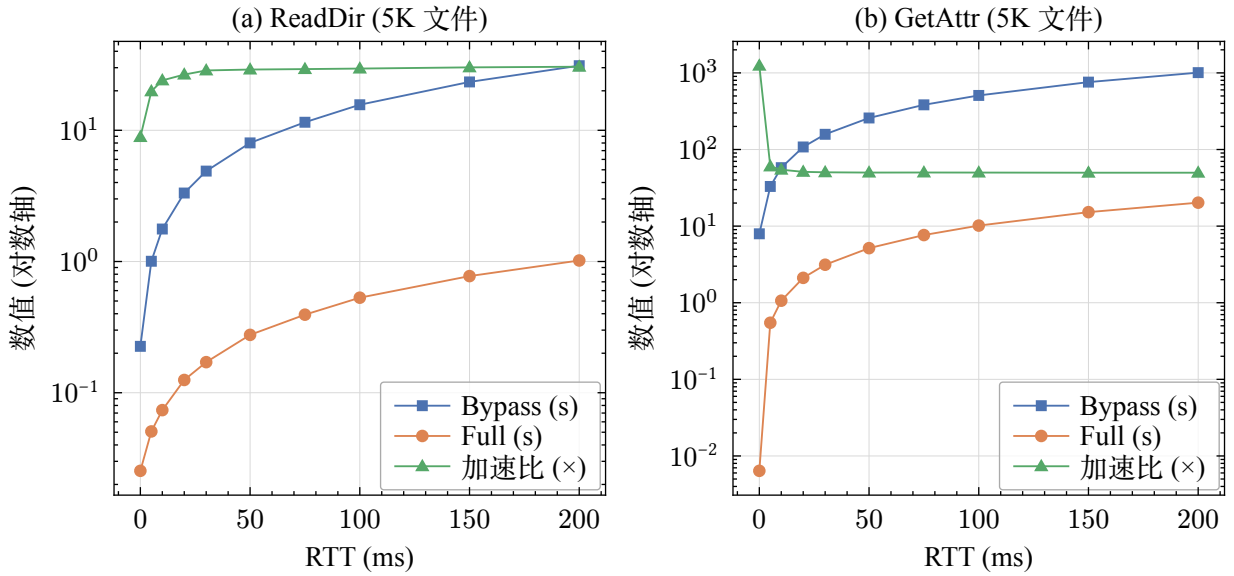


图 3.8 RTT 对 ReadDir 和 GetAttr 性能的影响与加速比

批量预取在高 RTT 环境下优势更加突出。ReadDir 加速比从本地的 8.88 倍提升至 RTT=200ms 时的 30.51 倍，GetAttr 在 RTT 大于 5ms 后稳定保持约 50 倍加速。RTT 越高，批量聚合对延迟惩罚的缩减效果越显著，验证了本方法对云边端跨域高延迟网络环境的良好适应性。

3.4.5 批量聚合的流量优化效果实验分析

本组实验验证批量聚合机制在减少协议层网络流量方面的效果。

为了量化批量聚合对网络流量的优化作用，在本地部署环境下分别设置 $\beta = 1$ (等效逐条请求)、 $\beta = 50$ 和 $\beta = 200$ 三组配置，对 syn-1000c-50f 数据集执行完整的元数据遍历，记录客户端与元数据服务器之间的协议总流量，如图 3.9 所示。

图 3.9 显示逐条请求模式 ($\beta = 1$) 的协议总流量为 7.14 MB，而 $\beta = 50$ 和 $\beta = 200$ 时均降至约 4.16 MB，流量减少约 42%。该结果揭示了批量聚合的流量优化机理：逐条模式下每次 gRPC 请求和响应均携带独立的协议头、序列化信封和连接管理开销，50,000 个文

件的逐条查询产生大量重复的协议包装开销；批量聚合将多个查询打包为单次 RPC，协议头和信封开销被多个查询分摊，净载荷比例显著提高。值得注意的是 β 从 50 增至 200 后流量几乎不变，说明协议开销的缩减在较小 β 即趋于饱和，进一步增大 β 的边际流量收益极为有限。

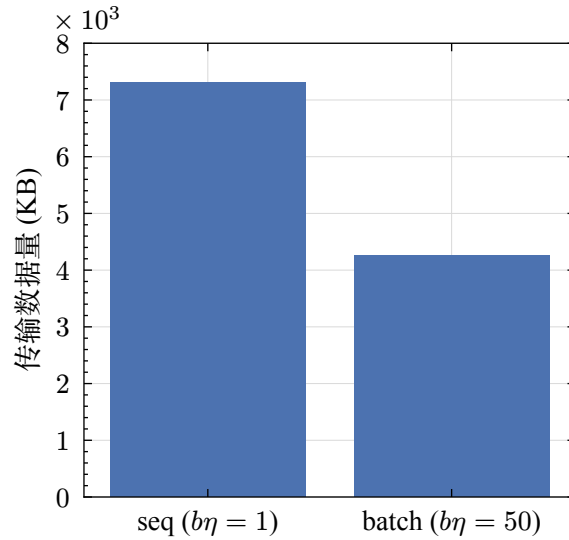


图 3.9 不同批量块大小 β 下的协议总流量对比

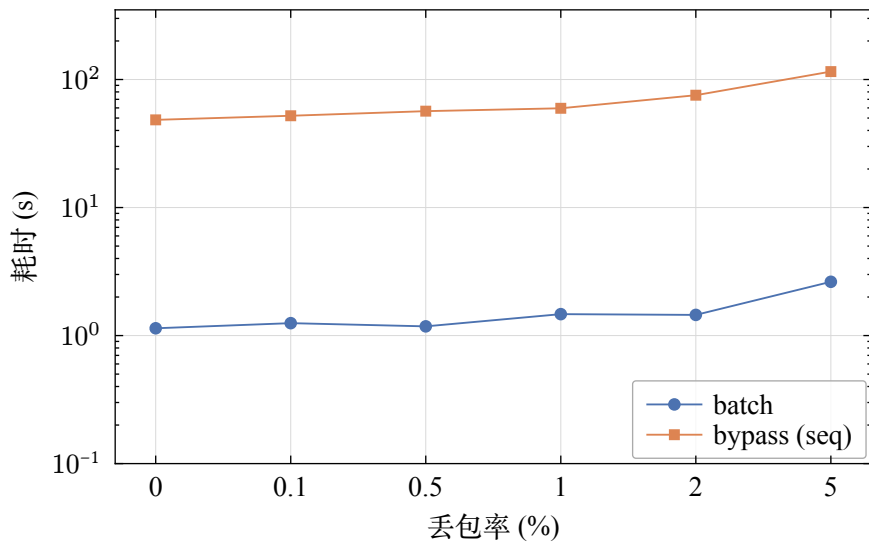


图 3.10 不同丢包率下两种模式元数据遍历耗时对比

批量聚合减少了约 42% 的协议流量，在云边端带宽受限的跨域链路上有助于降低传输延迟和网络拥塞风险，且该收益在较小 β 值下即可获得。

3.4.6 网络丢包敏感性实验分析

本组实验验证批量预取在网络丢包条件下的性能稳健性。

为了分析网络丢包对批量预取和逐条请求两种模式的差异化影响，通过 tc/netem 在 Cloud-Edge 链路上注入 0% 至 5% 的随机丢包率，固定 $RTT=20ms$ 、 $\beta = 100$ ，在 syn-1000c-50f 数据集上分别测量 Full 模式和 Bypass 模式的元数据遍历耗时，每组重复 3 次取均值，如图 3.10 所示。

图 3.10 显示 Full 模式在丢包率从 0% 增至 5% 的过程中，耗时从 1.14 秒温和上升至 2.63 秒，增幅约 2.3 倍；Bypass 模式则从 48.5 秒急剧恶化至 115.3 秒，增幅约 2.4 倍，且绝对耗时始终高出一到两个数量级。该结果反映了两种模式对丢包敏感性的根本差异：Bypass 模式发起 50,000 次独立 gRPC 请求，每次丢包均触发 TCP 重传和指数退避，重传开销随请求总数线性叠加；Full 模式将请求聚合为约 500 次批量 RPC，丢包触发重传的次数从数万次级别降至百次级别，即使 5% 丢包率下，绝对重传次数仍保持在可控范围。在所有丢包率下，Full 模式的加速比均稳定在 40~52 倍，未出现因丢包加剧而导致加速比收窄的现象。

批量预取在网络丢包条件下展现出良好的性能稳健性。即使在 5% 的高丢包率环境下，Full 模式耗时仅温和增长，而逐条请求模式耗时急剧恶化，验证了批量聚合通过减少 RPC 次数有效抑制了丢包导致的重传开销累积。

3.5 本章小结

本章针对云边端跨域训练中元数据访问往返多、冷启动慢的问题，提出了面向跨域训练的元数据批量预取与缓存方法。该方法以批量目录聚合、客户端递归遍历和多级缓存驻留为核心：在客户端侧通过异步递归遍历提前发起预取，并将多次目录请求聚合为单次批量 RPC；在元数据服务器侧通过本域缓存和跨域转发缓存协同降低查询代价；在结果驻留环节借助分层元数据缓存与跨进程共享内存缓存，使首轮获取的元数据可在多进程、多任务与多轮次之间持续复用。

整体设计将原本散布于初始化、迭代和多任务复用各阶段的逐条元数据访问，压缩为冷启动阶段的少量批量请求加上后续轮次的本地命中，从机制层面回应了引言提出的元数据访问瓶颈。本章实验在合成数据集上验证了批量聚合与多级缓存的核心加速能力、参数敏感性、跨域延迟下的加速优势、协议流量优化以及丢包条件下的稳健性；该方法与后续两项技术在数据加载、检查点等环节的系统级耦合效应，将在第六章 CEDFS 的端到端集成实验中进一步体现。

第四章 基于异步预取调度的数据预热技术

4.1 引言

在云边端跨域分布式训练中，通常每个 batch 的训练数据需从分散节点加载，与训练计算串行执行，导致 GPU 算力闲置。现有优化方法多针对数据中心内的高速存储，缺乏对边缘环境异构链路的适配，也未充分利用训练序列的可预测性。因此，本章提出面向云边端环境的数据预热技术，通过异步预取实现数据加载与计算的流水化解耦。

4.2 问题分析与研究思路

在云边端跨域训练中，数据加载性能瓶颈源于训练框架按需消费样本的执行方式与跨域异构存储环境之间的错配。训练计算只有在当前 batch 的数据到达后才能继续，因此数据加载延迟会直接转化为 GPU 等待；在本地高速存储中，这一开销往往可被缓存和带宽掩盖，但在跨域场景下会成为训练吞吐的关键限制因素。

以 PyTorch 为例，DataLoader 按照 Sampler 生成的样本顺序逐批请求训练数据。对于分布式训练，不同 rank 虽然消费不同的数据分片，但每个 rank 在给定随机种子下都遵循确定性的访问序列。这意味着训练数据的未来访问顺序在轮次开始前是可预测的；然而默认的数据读取路径仍采取边读边等的同步方式，使跨域数据传输与前台训练计算难以形成有效重叠。

这一问题在云边端实际部署中进一步放大。智能制造和智能电网等场景中的训练数据往往分散在多个端侧节点，文件数量多而单文件尺寸较小，同一数据集还会被多任务比较训练和周期性重训反复访问。若缺乏本地缓冲与复用机制，跨域搬运开销不仅会在单次训练内阻塞数据消费，还会在多任务和多轮次场景中重复累积，持续占用边侧有限带宽。

基于上述分析，本章聚焦三个核心问题：①如何在边侧有限的内存与磁盘预算下构建本地两级缓存，将重复访问尽量转化为本地命中；②如何利用训练访问序列的可预测性，提前生成与实际消费顺序一致的预取任务；③如何在不挤占前台训练 I/O 通路的前提下，使后台数据搬运与训练计算形成稳定重叠。下文围绕两级数据缓存、访问序列预计算和异步预取调度三个方面展开设计。

4.3 详细方案设计与实现

4.3.1 符号描述

本章所描述的基于异步预取调度的数据预热技术涉及对云边端跨域环境中训练数据加载过程的建模和分析。为简化后续模块设计和算法描述中的参数推导过程，本章方法所涉及数据特征及其描述如表 4.1 所示。

表 4.1 基于异步预取调度的数据预热技术的相关符号描述表

符号	定义
S	训练数据集，包含所有训练样本文件的集合
N_s	数据集中样本文件总数， $N_s = S $
E	训练总轮次（轮次数量）
e	当前轮次（Epoch）编号， $e \in \{0, 1, \dots, E - 1\}$
r	当前训练进程的 rank 编号（分布式训练中的进程标识）
W	分布式训练的总进程数（World Size）
σ_e	第 e 个轮次的样本访问序列， $\sigma_e : \{0, \dots, N_s - 1\} \rightarrow S$
s_{seed}	训练框架的全局随机种子，用于确定性洗牌
ω	预取窗口大小，控制提前预取的样本数量
ω_{merge}	滑动窗口合并的前瞻窗口大小
Q	预取任务优先级队列，基于最大堆实现
w_d	域延迟权重，跨域高延迟数据节点 $w_d > 1$ ，本地节点 $w_d = 1$
β	文件大小权重系数，控制大文件优先级提升幅度
BW_{max}	预取最大带宽限制（MB/s）
BW_{quota}	Advisory 服务分配的动态带宽配额（MB/s）
BW_{eff}	有效带宽上限， $BW_{\text{eff}} = \min(BW_{\text{quota}}, BW_{\text{max}})$
BW_{curr}	当前预取带宽使用量（bytes/s）
α	带宽使用阈值比例
γ_{conc}	最大并发预取任务数
B	数据块大小
θ_{cross}	跨轮次预取触发进度阈值
η_{warmup}	warmup 目标覆盖率
t_{warmup}	warmup 最大等待时间（秒）

4.3.2 整体方案设计

基于异步预取调度的数据预热技术的整体架构如图 4.1 所示。客户端侧主要包含序列管理、预取调度和本地分层缓存三部分；服务端侧提供高吞吐数据传输与节点状态上

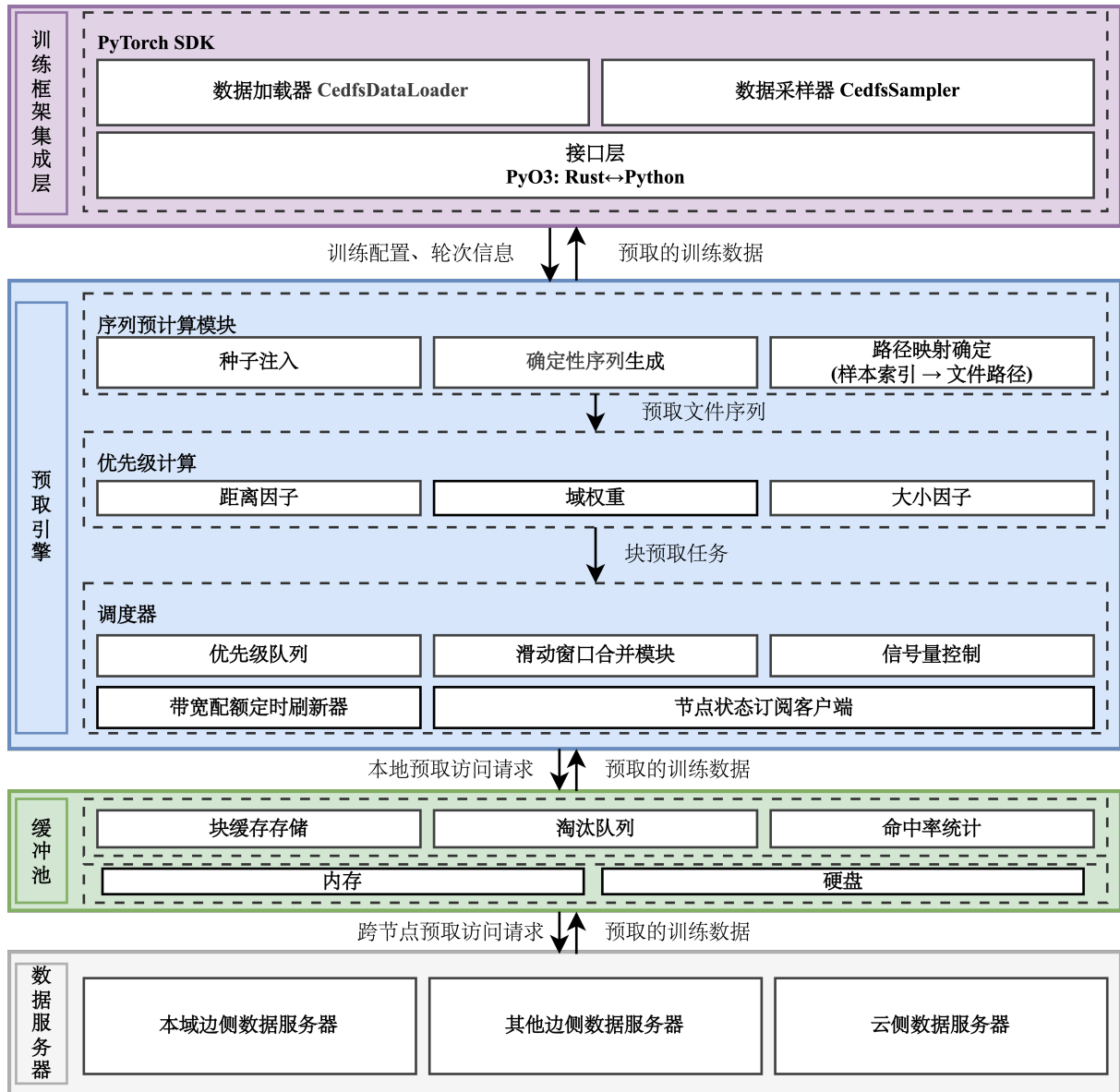


图 4.1 基于异步预取调度的数据预热技术整体架构

报能力，两侧协同完成数据预热与本地命中路径构建。前者解决“预取什么、何时预取”的问题，后者提供“从哪里取、以什么状态取”的执行支撑。

整体工作流程：从时间顺序看，训练前客户端先注入访问序列并触发预热；训练中，数据读取优先命中本地缓存，后台预取与前台计算并行推进；具体的序列预计算、任务生成、调度控制和执行机制见第 4.3.3 节至第 4.3.6 节。

客户端侧设计：客户端侧重点包括序列管理、任务调度和两级数据缓存三部分，分别负责维护访问顺序、组织预取执行和承接本地命中路径，构成训练侧的数据供给主链路。

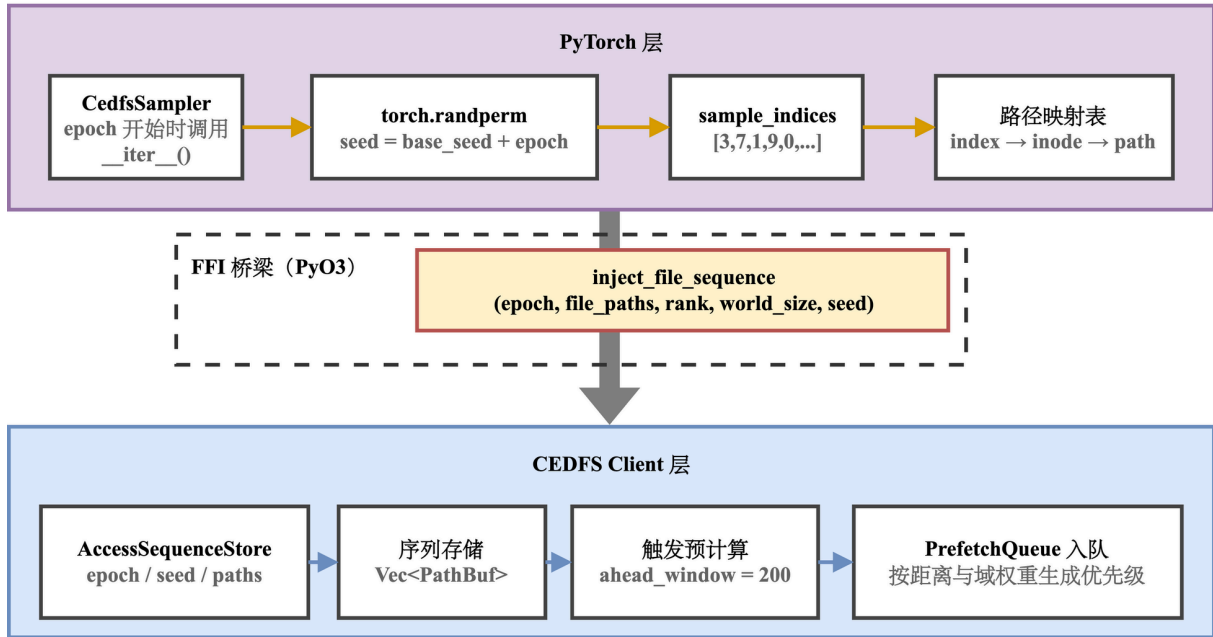


图 4.2 基于确定性种子的数据访问序列生成与注入流程

数据服务器侧设计：服务端侧提供基于 QUIC 的数据块传输通道和节点状态上报接口，为客户端的异步搬运和调度决策提供支撑。围绕这一总体框架，后续各节进一步说明访问序列生成、任务编排和预取执行机制。

4.3.3 基于确定性种子的数据访问序列预计算

训练数据预热的前提是准确知道训练框架将要访问的数据顺序。在 PyTorch 分布式训练中，每个轮次的数据访问顺序由 CedfsDistributedSampler 通过随机种子控制的洗牌操作确定。本方法利用训练框架固定的随机种子，在训练执行前预计算所有轮次的精确数据访问序列，为预取引擎提供确定性的预取目标。整体流程如图 4.2 所示。

在序列生成方面，采用训练框架侧负责洗牌、客户端核心负责序列缓存的协作策略。洗牌过程在 CedfsDistributedSampler 中完成，以训练随机种子为基础，结合轮次编号推导出当前的洗牌种子：

$$s_e = s_{\text{seed}} + e \quad (4.1)$$

式中， s_{seed} 为基础随机种子（一般 42）， e 为轮次编号。该公式确保相同种子下每个轮次的洗牌序列完全确定、跨运行可重现。对于分布式训练场景，各 rank 间的数据隔离并非通过种子区分，而是通过分片采样（striding）实现：CedfsDistributedSampler 先以 s_e

为种子调用 `torch.randperm` 生成全局洗牌序列 π ，再按 rank 间隔采样获取当前 rank 的数据子集：

$$\sigma_{e,r} = \pi[r :: W] \quad (4.2)$$

即从全局序列 π 中每隔 W 个元素取一个，起始偏移为 r 。该方式精确复现了 PyTorch 原生 `DistributedSampler` 的行为，保证各 rank 的数据子集互不重叠且联合覆盖整个数据集。

在序列注入方面，`CedfsDataLoader` 在每个轮次的迭代器初始化时，首先从 `Sampler` 获取当前轮次的洗牌后样本索引序列，再结合初始化阶段从 `Dataset` 中预提取并缓存的完整文件路径列表，完成索引到路径的映射，生成按训练访问顺序排列的文件路径列表，最后通过 `inject_file_sequence(epoch, file_paths, rank, world_size, seed)` 接口将该文件路径列表注入客户端核心。`FileSequenceManager` 以 $(epoch, rank)$ 为键将序列存入分层缓存，同时记录文件数量、随机种子和 `world_size` 等轻量元数据，供后续预取任务生成和覆盖率计算使用。

在分布式训练的序列分片方面，`CedfsDistributedSampler` 进一步处理了 PyTorch `DistributedSampler` 的 padding 和 truncation 行为：当数据集样本总数 N_s 不能被 W 整除时，对洗牌后的全局序列进行尾部填充（padding）使其长度为 $\lceil \frac{N_s}{W} \rceil \times W$ ，再按 rank 间隔采样，确保各 rank 获得等长的数据子集。由于洗牌操作完全在 Python 侧借助 `torch.randperm` 完成，Rust 侧仅接收和缓存最终的文件路径列表，不涉及洗牌算法的重新实现，从根本上避免了因随机数生成器差异导致的访问顺序不一致问题。

在序列缓存与复用方面，`FileSequenceManager` 采用分层缓存策略管理历史轮次的序列数据：内存层保留最近若干 epoch 的序列，磁盘层持久化更多轮次的历史序列，保留数量和磁盘配额均可配置。当训练因异常中断并恢复时，已缓存的序列数据仍可从磁盘层加载，无需 Python 侧重新注入，加速了训练恢复过程。

需要说明的是，上述序列预计算与注入环节仅涉及样本级别的文件访问顺序，batch 的形成（即将连续的若干样本索引分组、由 Worker 进程逐一加载后经 `collate_fn` 拼装为 batch 张量）仍由 PyTorch `DataLoader` 的标准机制完成。CEDFS 的数据预热介入点在于 Worker 进程执行 `dataset[index]` 触发文件 I/O 时，由 CEDFS 客户端拦截并从本地缓存优先返回数据，而非改变 batch 的组织与形成方式。

4.3.4 基于动态优先级的预取任务生成与滑动窗口合并

预取引擎在获取到当前轮次的文件访问序列后，需要将其转换为可调度的预取任务。本环节的核心是确保预取任务的执行顺序与训练框架的数据访问顺序总体一致，同时结合云边端环境中数据分布的异构特征动态调整优先级，使跨域高延迟和大体积的数据文件获得更早的预取机会，以吸收其额外的传输延迟。

在任务生成方面，预取引擎遍历注入的文件访问序列 $\sigma_e = \{f_0, f_1, \dots, f_{N_s-1}\}$ ，为每个文件 f_i 创建一个 PrefetchTask 结构（包含文件路径、优先级、轮次编号、置信度等属性），优先级计算如下：

$$\text{priority}(f_i) = (N_s - i) \times w_d(f_i) \times \left(1 + \beta \cdot \log_2 \left(1 + \frac{\text{size}(f_i)}{B_{\text{base}}}\right)\right) \quad (4.3)$$

其中， $N_s - i$ 为基础顺序优先级，保证序列首部的文件（即最先被训练访问的数据）获得最高基础优先级； $w_d(f_i)$ 为文件 f_i 所在数据节点的域延迟权重； $\text{size}(f_i)$ 为文件大小， B_{base} 为用于归一化的基准大小（如 1 MB）， β 为文件大小权重系数。整个优先级值越大，在最大堆优先级队列中越先出队。

域延迟权重 w_d 的获取采用两级策略确定：①客户端在预取引擎初始化时，尝试通过 gRPC 接口向元数据服务器上部署的 Advisory 服务发起 GetDataLocationHints 请求，该服务维护数据位置索引（DataLocationIndex），复用聚合目录树中 TypeRemoteFile 条目的域归属信息，通过文件路径前缀匹配确定每个文件所属的数据域，并返回对应的距离分级权重。该权重直接反映了数据搬运的跨域延迟差异，使跨域高延迟的文件在同等访问位置下获得更高的优先级以提前搬运。②当 Advisory 服务不可用时，预取引擎自动降级为静态推算模式：根据文件路径前缀与客户端配置的挂载点映射关系识别数据域，查表获取默认延迟权重，功能不受影响。此外，文件大小信息可由客户端从第三章所述的元数据缓存中获取——在训练数据集初始化阶段，元数据预取机制已将文件的目录项信息（含文件大小属性）缓存于本地 IPC 共享内存中，预取引擎在任务生成时直接查询该缓存即可获得文件大小，无需额外的远端元数据访问。

上述多因子优先级策略的效果是：在单个 epoch 内，访问序列靠前的文件仍然优先被预取（保证预取与训练消费进度的一致性），但同一访问区间内跨域高延迟、大体积的文件会获得优先级提升，使其更早开始传输以吸收网络延迟波动；在跨轮次任务共存时，不同轮次的任务通过优先级值自然排序，当前轮次的任务整体优先于下一轮次的任务。当域延迟权重和文件大小信息不可用时，系统退化为纯基于访问顺序的优先级（ $w_d = 1$ ， $\beta = 0$ ），等价于 FIFO 顺序，保证功能正确性。

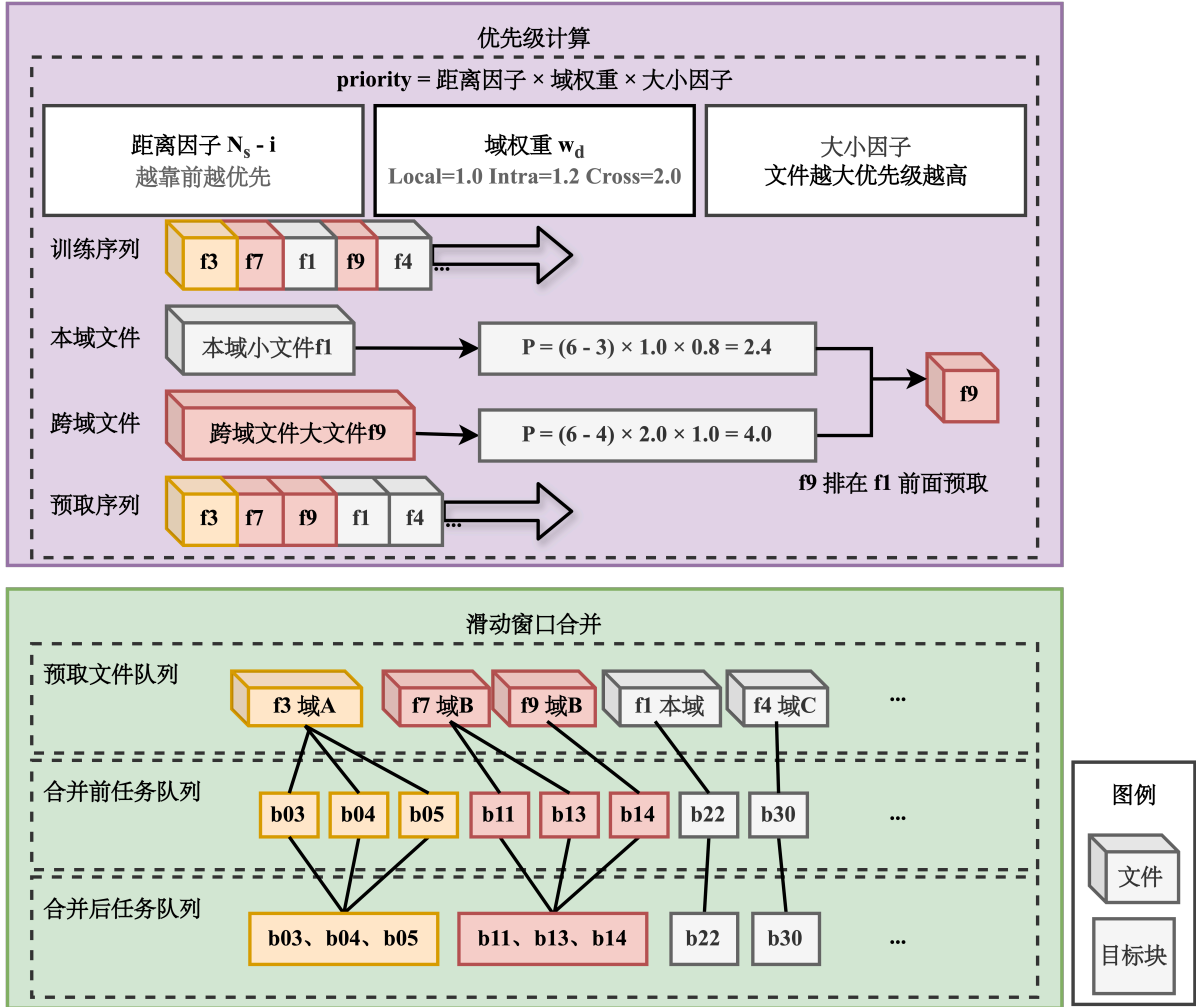


图 4.3 动态优先级计算与滑动窗口合并机制

在任务合并方面，当数据集规模较大时，逐文件生成预取任务会导致任务数量过多，增加调度器的调度开销。为此，设计基于滑动窗口的任务合并机制：预取引擎在生成任务时，使用前瞻窗口 ω_{merge} 扫描序列，在窗口 $[i, \min(i + 1 + \omega_{\text{merge}}, N_s))$ 内检测可合并的相邻文件——若连续文件位于同一目录下或属于同一文件的相邻字节范围，则将其合并为一个覆盖多文件或多范围的合并任务，合并任务的优先级取被合并文件中的最高优先级，以保持预取顺序不变。该合并机制可降低任务粒度，减少队列操作和调度开销。图 4.3 展示了动态优先级计算与滑动窗口合并机制。

在任务入队方面，生成的预取任务通过 PrefetchQueue 的批量入队接口 `push_batch()` 一次性压入优先级队列。队列容量可配置，当队列已满时返回错误，防止内存溢出。队列同时支持按 epoch 过滤已入队任务和清除过期 epoch 的任务，为跨轮次任务管理提供基础。

4.3.5 基于带宽感知的预取任务调度与执行

预取调度器从优先级队列中取出任务后，需要在资源约束下控制任务的并发执行，避免预取 I/O 过度占用网络带宽而影响前台训练的数据读取。

在调度循环方面，预取调度器以异步任务形式在后台持续运行。每次调度循环中，调度器首先尝试通过信号 γ_{conc} 获取并发许可，若无可用许可则跳过本轮；随后从优先级队列 Q 中弹出最高优先级任务 T ，若队列为空则短暂休眠后重试；最后，在分发任务前执行带宽检查，若当前带宽使用量超过限制则将任务放回队列并短暂等待。

在带宽控制方面，调度器采用两级带宽管理策略：①动态带宽配额：客户端定时向元数据 Advisory 服务发起 RequestBandwidthQuota 请求，Advisory 服务的带宽配额管理器 (BandwidthQuotaManager) 根据当前集群的整体网络负载和各节点的可用带宽，为该客户端的预取任务分配动态带宽配额 BW_{quota} 。配额分配遵循优先级权重机制：当检查点恢复与数据预取并发进行时，恢复任务的优先级权重为 2，预取任务的权重为 1，确保故障恢复场景下恢复流量获得至少 $\frac{2}{3}$ 的可用带宽。②静态限速兜底：当 Advisory 服务不可用时，调度器回退至静态带宽限制 BW_{max} 。

在两级策略下，调度器通过滑动窗口计算当前预取带宽。每完成一个预取任务，将其传输字节数记入带宽统计器；带宽控制模块读取最近 t_{window} 秒（默认 5 秒）内的累计传输字节数，计算当前带宽 BW_{curr} 。调度器在分发任务前检查：

$$BW_{\text{curr}} < \alpha \times BW_{\text{eff}} \quad (4.4)$$

其中 $\alpha = 0.9$ 为预留余量系数， $BW_{\text{eff}} = \min(BW_{\text{quota}}, BW_{\text{max}})$ 为有效带宽上限，取动态配额与静态限制的较小值。该检查确保预取流量不超过有效带宽限制的 90%，为前台读取预留网络余量。图 4.4 展示了预取调度器的调度流程与带宽控制机制。

在任务执行方面，通过带宽检查的任务被分发至异步执行线程。执行线程通过 QUIC 协议向目标数据服务器发起块读取请求，将读取到的数据写入本地分层缓存（优先写入内存层，内存层满时溢出至磁盘层），并向指标收集器上报传输字节数和执行耗时。执行完成后释放信号量许可，使调度器可以分发新任务。

在 warmup 覆盖率追踪方面，调度器维护当前预热轮次的已完成任务数，每完成一个属于预热轮次的任务即递增计数器，实时更新覆盖率：

$$\eta = \frac{\text{completed}_{\text{tasks}}}{N_s} \quad (4.5)$$

供预取引擎的 warmup 阻塞逻辑判断是否达到目标覆盖率。

4.3.6 数据预热执行与流水化加载机制

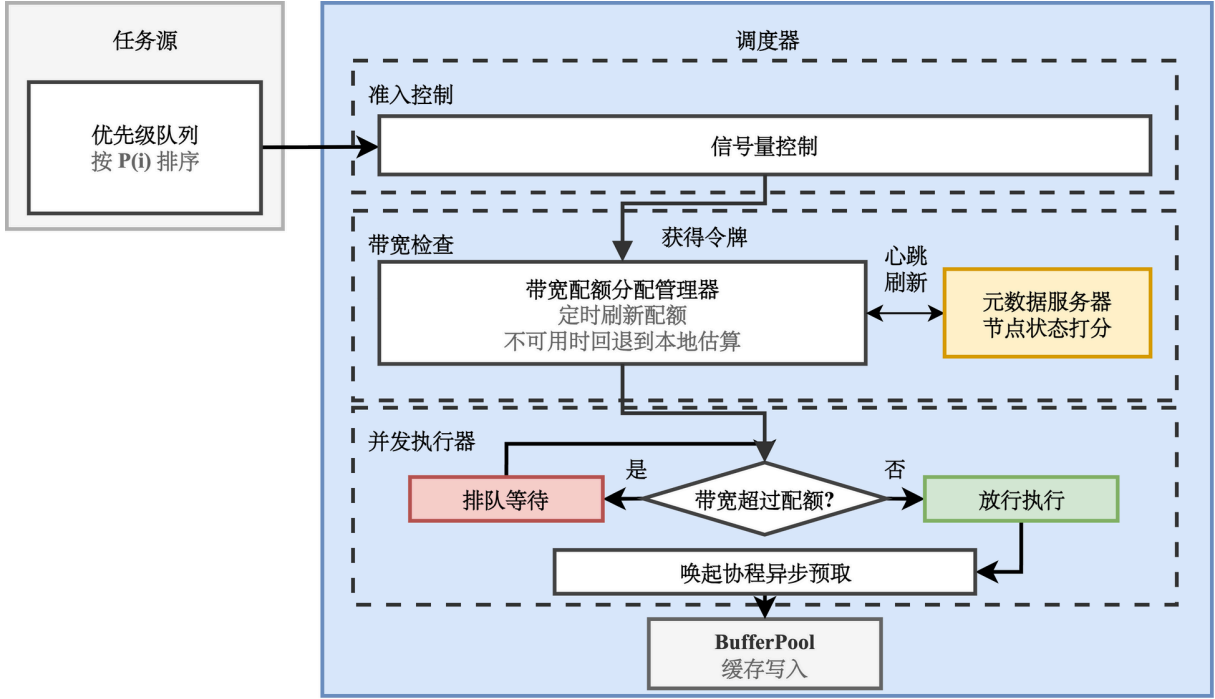


图 4.4 预取调度器调度流程与带宽控制

数据预热在训练生命周期的不同阶段采用不同的工作模式：首个轮次训练启动前，预取引擎以同步阻塞方式批量搬运数据至本地缓存，确保训练开始时有足够数据可用；后继 epoch 中，预取引擎以异步方式持续运行，与训练计算在时间维度上形成流水化重叠，同时通过跨轮次预取触发机制实现轮次间的无缝衔接。

同步阻塞预热模式：针对训练首个轮次的冷启动问题，预取引擎在训练开始前以同步阻塞方式执行数据预热，写入时优先填充内存层以支撑即时读取需求，当内存层容量不足时溢出至磁盘层暂存。

在阻塞等待方面，预取引擎定期轮询当前 warmup 覆盖率 η ：

$$\eta = \frac{\text{completed}_{\text{tasks}}}{N_s} \quad (4.6)$$

轮询直至覆盖率 $\eta \geq \eta_{\text{warmup}}$ （默认 0.70）或等待时间超过 t_{warmup} （默认 300 秒）后放行训练启动。当 warmup 超时时覆盖率仍未达标，系统采用优雅降级策略：以当前缓存状态放行训练，后续未命中缓存的数据读取自动降级为同步跨域访问，不中断训练流程。

异步流水化加载模式：训练启动后，预取引擎切换为异步模式在后台持续运行，如图 4.5 所示，并与训练计算并行工作。预取调度器根据预取窗口 ω 控制提前搬运的数据范围：在当前训练已消费至第 i 个样本时，预取调度器确保第 i 至第 $i + \omega$ 个样本的数据已被搬运至缓存中。

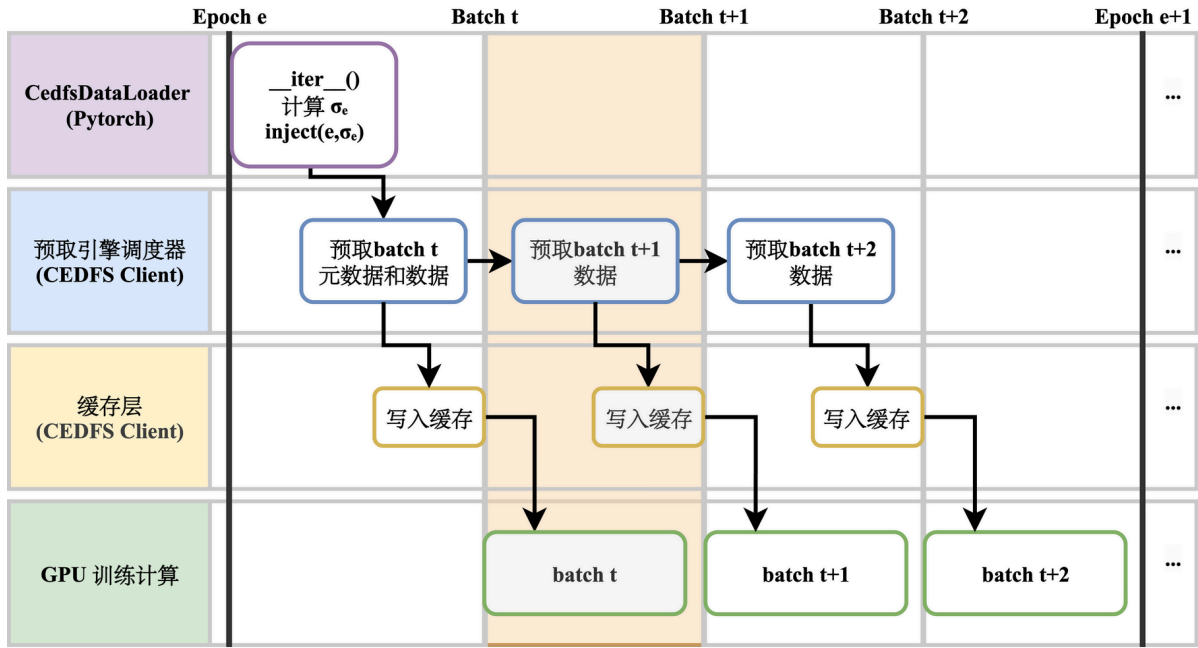


图 4.5 异步流水化数据加载时序示意图

在跨轮次预取触发方面，当轮次进度比例 $p = \frac{i}{N_s}$ 达到阈值 θ_{cross} （默认 0.80）时，预取引擎自动生成下一轮次的预取任务并压入队列，使轮次过渡时缓存中已有部分数据就绪，实现无缝衔接（详见第 4.3.6 节）。

在多进程场景下，预取调度器仅需为同一数据搬运一次，同节点上的多个训练进程通过共享内存 IPC 机制访问同一份缓存数据，避免重复预取，有效消除了多进程重复预取的网络开销。

4.3.7 整体执行流程分析

综合第 4.3.2 节至第 4.3.6 节的设计，本方法的端到端执行流程可分为四个阶段，如图 4.6 所示。

第一阶段为数据访问序列预计算与注入：Sampler 根据随机种子和轮次编号生成确定性文件序列，CedfsDataLoader 将其注入客户端核心缓存（第 4.3.3 节），分布式场景下各 rank 注入各自的数据子集。

第二阶段为预取任务生成与同步预热：预取引擎按多因子优先级策略生成预取任务（第 4.3.4 节），调度器在带宽配额约束下将数据异步搬运至本地两级缓存（第 4.3.5 节），并阻塞等待缓存覆盖率达标后放行训练启动（第 4.3.6 节）。

第三阶段为训练执行与异步流水化加载：预取引擎切换为后台异步模式，以预取窗口 ω 始终领先于训练消费进度持续搬运数据；训练的数据读取优先命中本地内存或磁

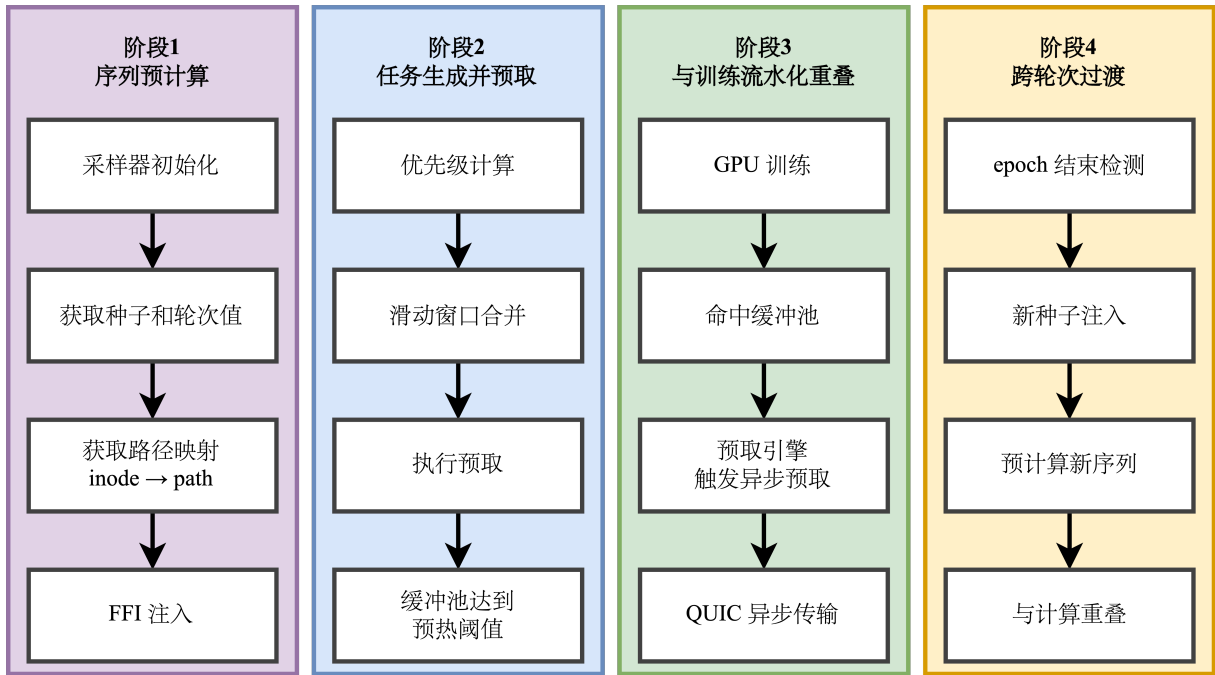


图 4.6 基于异步预取调度的数据预热技术端到端执行流程

盘缓冲，未命中时降级为同步跨域访问，计算与 I/O 在时间维度上形成流水化重叠（第 4.3.6 节）。

第四阶段为跨轮次过渡：当训练进度达到阈值 θ_{cross} 时，预取引擎提前触发下一轮次的任务生成，轮次切换时缓存中已有部分数据就绪，实现无缝衔接（第 4.3.6 节）。

表 4.2 天数智芯三节点集群的硬件与软件环境

配置项	参数
节点数量	3 台物理服务器
处理器	2× Intel Xeon Gold 6330（56 核/112 线程）
内存	503 GiB DDR4
GPU	n33: 16× BI-V150（32 GiB）；n192/n197: 8× BI-V100（32 GiB）
存储	NVMe SSD 3.6~3.7 TiB
操作系统	Ubuntu 20.04.6 LTS
Python	3.10
PyTorch	2.7.1

4.4 实验与验证

本节通过系统性实验验证第四章提出的数据预热机制。

4.4.1 实验环境与参数配置

硬件与网络部署：本章实验在天数智芯三节点物理集群上完成，硬件与软件环境如表 4.2 所示。

三节点构成云边端三域拓扑，训练节点为 n33，远端训练数据默认放置在 n192，保证训练数据访问稳定经过广域链路。节点部署如表 4.3 所示。

表 4.3 天数智芯三节点集群的节点部署拓扑

节点	角色	IP	CEDFS 角色
n33	Cloud / 训练节点	10.31.10.210	Cloud Meta / Data / Client
n192	Edge-A	10.31.10.192	Edge Meta / Data
n197	Edge-B	10.31.10.197	Edge Meta / Data

跨域链路由 tc/netem 注入延迟、抖动和带宽约束，默认配置如表 4.4 所示；带宽、RTT、抖动和丢包敏感性实验每次只修改其中一个变量，其余参数保持不变。

表 4.4 天数智芯三节点集群跨域链路的默认网络模拟参数

链路	单边延迟	RTT	带宽	模拟场景
n33 ↔ n192	2 ms ± 1 ms	~5 ms	80 Mbps	Cloud ↔ Edge-A
n33 ↔ n197	2 ms ± 1 ms	~5 ms	80 Mbps	Cloud ↔ Edge-B
n192 ↔ n197	5 ms ± 2 ms	~10 ms	40 Mbps	跨边缘域

表 4.5 第四章实验的关键配置

配置项	完整训练场景
训练模型	ResNet-50
批大小	64
训练轮数	5
并行方式	单机多卡 DDP
数据集名称	Imagenette2 全分辨率版本
数据集规模	10 类，9,469 张训练图像
数据集大小	1.5 GB
启动前预热覆盖率	25%
启动前最长等待时间	120 秒
本地数据缓存	256 MiB
缓存容量扫描范围	32/64/96/128/160/192/256/320/384/448/512 MiB
并行进程数扫描范围	1/2/3/4/6/8

本章实验沿用上述三节点弱网环境，使用 Imagenette2 全分辨率版本，并以 JuiceFS-cached 作为对照。

关键配置：表 4.5 展示完整训练实验中的关键设置，调度器等相关参数在各组实验中保持固定，因此不再逐项展开。

4.4.2 本地缓存容量对预热收益的影响

本组实验在固定 80 Mbps 带宽与约 4 ms RTT 条件下，使用约 1.5 GB 的 Imagenette2 数据集扫描 32~512 MiB 本地缓存容量，并比较 CEDFS-full 与 JuiceFS-cached 的平均吞吐量。

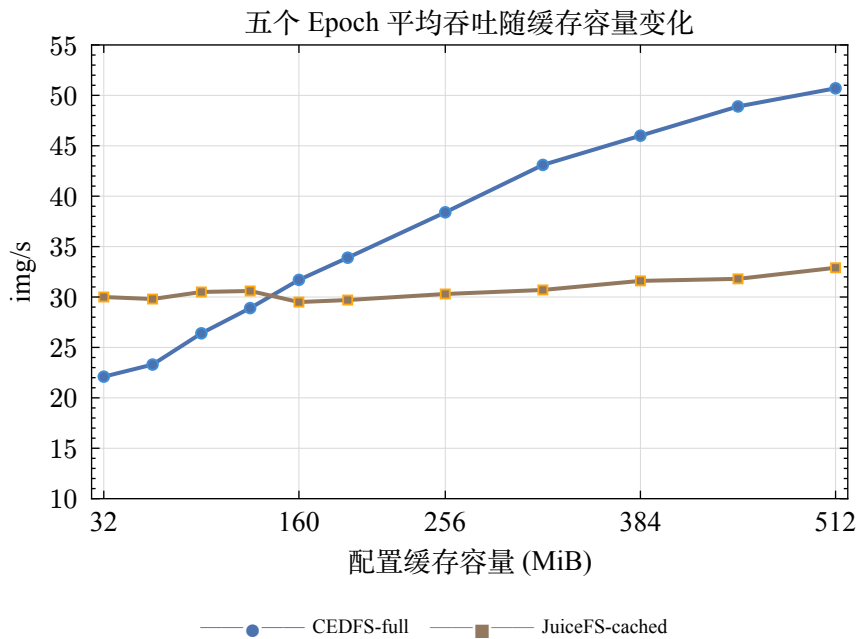


图 4.7 缓存容量变化对完整训练吞吐的影响

图 4.7 表明，在 32~128 MiB 的中小缓存区间内，CEDFS-full 落后于 JuiceFS-cached，随着缓存容量增加，差距逐渐缩小。该区间缓存容量较小，预取调度虽然能够提前搬运数据，但由于缓存空间不足，预热数据很快就被后续数据覆盖，同时预取占用了正常的数据访问流量，反而预取成了阻碍。

当缓存容量继续扩大到 160 MiB 以上时，CEDFS-full 开始稳定超过 JuiceFS-cached。这说明预取机制充分发挥作用，预热数据能够在训练访问时驻留在缓存中，从而持续提升训练吞吐。

在云边端训练中，本地缓存通常远小于数据集规模，该组实验表明，CEDFS-full 在缓存进入有效工作区间后，能高效地将同等规模缓存转化为实际训练吞吐收益。

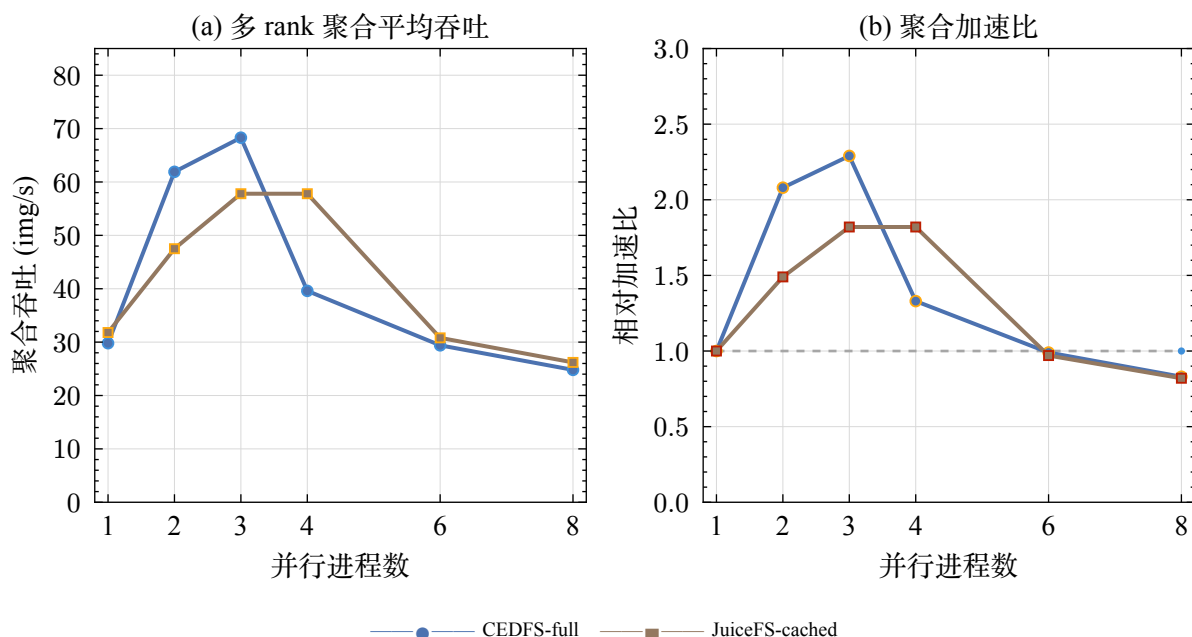


图 4.8 并行消费规模变化对聚合吞吐与加速比的影响

4.4.3 并行消费规模边界分析

本组实验选取 256 MiB 本地数据缓存作为默认工作点，在固定 80 Mbps 带宽与约 4 ms RTT 条件下扫描并行进程数为 1、2、3、4、6、8，并比较 CEDFS-full 与 JuiceFS-cached 的聚合吞吐量和相对加速比。

图 4.8 表明，随着并行度的提高，总吞吐呈现先上升后下降的趋势。在单进程条件下，CEDFS-full 与 JuiceFS-cached 的差距并不大；但当并行进程数提升到 2~3 时，CEDFS-full 转而成为更高聚合吞吐的一侧，但随着并行进程数继续增加，两种方案的聚合吞吐和相对加速比都开始回落，说明当并行度变大时，多卡间会产生更多存网资源竞争，预取优势被大大削弱。

综合本组实验可以看到，CEDFS-full 能提升中小并行度下的整体供给效率，这一特征更贴近边侧多卡训练与小规模协同训练的实际需求。

4.4.4 真实训练中的计算与 I/O 重叠验证

本组实验在与前文一致的弱网配置下，采用真实单卡 GPU 训练环境，对按需加载基线与 CEDFS-full 进行对比，并分别选取 ResNet-50 与 MobileNetV3 作为计算强度较高和较低的代表模型，统计 Epoch 0 的平均数据等待与训练吞吐。需要说明的是，full 模式在训练启动前仅阻塞至 warmup 覆盖率达到预设阈值，而非等待 Epoch 0 的全部样本完成预取；训练开始后，尚未覆盖的剩余样本仍由后台继续搬运。

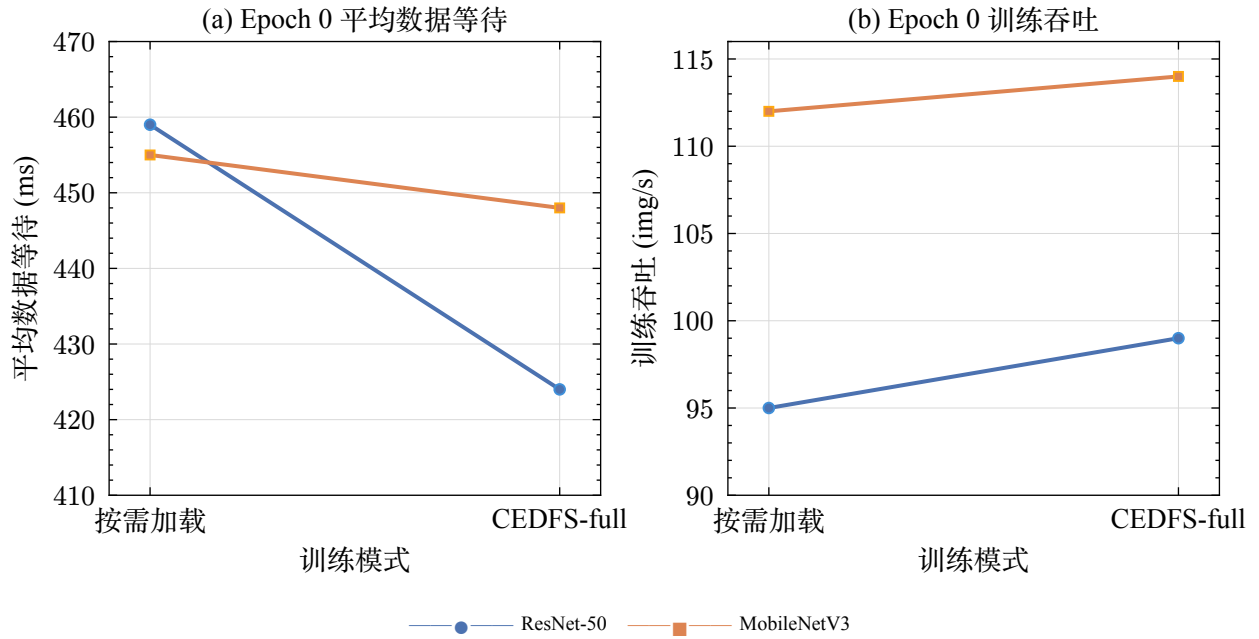


图 4.9 真实单卡 GPU 训练中异步预取的重叠收益

图 4.9 左图表明, CEDFS-full 在真实训练中能够降低 Epoch 0 的前台数据等待, 但改善幅度与模型计算强度密切相关。对于计算更重的 ResNet-50, 平均数据等待由 459 ms 降至 424 ms, 降幅为 7.6%; 而对于计算更轻的 MobileNetV3, 仅由 455 ms 降至 448 ms, 降幅为 1.5%。其在于训练开始后仍有一部分样本需要边训练边预取: 当单批计算更重、前向与反向传播持续时间更长时, 后台预取在相邻批次之间拥有更大的推进空间; 而当模型较轻、样本消费速度更快时, 这一重叠窗口就会缩小。

图 4.9 右图给出的 Epoch 0 吞吐变化与上述趋势一致。ResNet-50 的训练吞吐由 95 img/s 提升至 99 img/s, 增幅为 4.2%; MobileNetV3 仅由 112 img/s 提升至 114 img/s, 增幅为 1.8%。这表明在当前 warmup 仅覆盖部分样本的设定下, 模型计算阶段提供的时间窗口确实会影响后台预取能否持续跟上前台消费, 从而决定前台等待下降能在多大程度上转化为实际吞吐收益。

需要强调的是, 本组实验的目的在于验证 epoch 内计算与 I/O 的可重叠性, 而非证明端到端冷启动总时长必然更短。以 ResNet-50 为例, full 模式一次运行中达到约 70.5% 的 warmup 覆盖率仍需约 80 s, 而同次运行的 Epoch 0 训练阶段约为 87.5 s, 有一定启动代价。

4.5 本章小结

本章针对云边端跨域训练中数据加载与训练计算串行导致的算力空转问题，提出了基于异步预取调度的数据预热机制。该机制以前瞻式访问序列预计算为基础，结合两级本地缓存、优先级任务调度和异步预取执行，将远端数据搬运提前到训练消费之前。

综合实验结果可以看出，该机制的收益与可用缓存空间、并行消费规模以及模型计算强度密切相关：当本地缓存进入有效工作区间后，CEDFS-full 可以稳定把有限缓存转化为训练吞吐收益并超过对比方案；在小到中等并行度下，该机制的聚合供给能力更具优势，说明其更适合边侧多卡和小规模协同训练场景；在真实单卡 GPU 训练中，该机制能够在 warmup 之后减轻训练阶段的前台 I/O 阻塞，但这一收益仍显著依赖模型计算阶段所提供的重叠窗口。

第五章 基于状态感知分配的检查点分级写入技术

5.1 引言

在云边端跨域训练中，生成大型检查点文件常导致边缘网络的传输阻塞。传统同步模式下，持久化操作极易引发分钟级算力闲置。虽然现有框架引入了异步保存机制，但边缘节点有限的系统资源与波动的存网状态会导致后台任务堆积，引发次生阻塞。为此，本章提出适配边缘环境的检查点分级写入技术，利用状态感知分配实现检查点的分级持久化与就近放置。

5.2 问题分析与研究思路

在云边端跨域分布式训练中，检查点保存与恢复性能瓶颈源于 PyTorch 检查点机制的执行路径与边缘异构存网条件之间的错配。DCP 将保存流程拆分为 GPU 到 CPU 的转存和 CPU 到持久化存储的写入两个阶段，其中前者已可借助 `async_save` 较好隐藏，因此本章关注的核心是后续的分发、持久化与恢复路径优化。

这一矛盾在边缘环境中更加突出。现有检查点优化工作多以高性能本地存储和稳定互联为前提，难以直接适应云边端场景；而对于边缘训练而言，即便模型规模较小，完整检查点仍包含模型权重、优化器状态等辅助数据，在有限带宽、受限容量和动态波动的节点状态下，保存与恢复开销依然突出。

另一方面，检查点内部不同内容对训练恢复的重要性并不相同。全局分片映射、模型权重和优化器状态在恢复路径上的关键程度存在明显差异，但默认写入方式通常采用统一的放置和恢复策略；再叠加节点容量和链路状态的持续变化，单一策略很难同时兼顾阻塞时间、可靠性与恢复速度。

基于上述分析，本章聚焦三个核心问题：①写入目标选择缺乏智能性，默认本地写入难以利用域内其他节点的空闲存储资源；②检查点内容缺乏差异化处理，不同重要性的数据未被区分副本策略和写入方式；③副本放置与故障恢复缺乏环境感知，难以根据实时容量、带宽和延迟状态动态调整路径。下文围绕动态资源模型、分级副本放置与异步写入、分层恢复三个方面展开设计。

5.3 详细方案设计与实现

5.3.1 符号描述

本章所描述的基于状态感知分配的检查点分级写入技术涉及对云边端跨域环境中检查点写入与恢复过程的建模和分析。为简化后续模块设计和算法描述中的参数推导过程，本章方法所涉及数据特征及其描述如表 5.1 所示。

表 5.1 基于状态感知分配的检查点分级写入技术相关符号描述表

符号	定义
K	检查点中包含的数据分片集合
k_i	第 i 个检查点分片, $k_i \in K$
$cr(k_i)$	分片 k_i 的重要性等级, $cr \in \{\text{Critical}, \text{High}, \text{Medium}, \text{Low}\}$
N_e	边缘节点集合, $N_e = \{n_1, n_2, \dots, n_m\}$
N_c	云侧节点集合
$z(n)$	节点 n 所属的区域标识
$\tau(n)$	节点 n 的层级类型, $\tau \in \{\text{Edge}, \text{Cloud}\}$
$cap(n_j)$	节点 n_j 的可用存储容量
$cap_ratio(n_j)$	节点 n_j 的存储容量使用率
$bw(n_j)$	节点 n_j 的当前可用网络带宽
$dist(n_s, n_t)$	节点 n_s 到 n_t 的拓扑距离等级, $dist \in \{0, 1, 2, 3\}$
$R_{cloud(cr)}$	重要性等级为 cr 的分片的云侧目标副本数
$R_{edge(cr)}$	重要性等级为 cr 的分片的边缘侧目标副本数
θ_{skip}	节点跳过容量使用率阈值, 超过此值的节点被排除
θ_{low}	低级降级容量使用率阈值, 超过此值时 Low 级别副本数降为 1
θ_{med}	中级降级容量使用率阈值, 超过此值时 Medium 级别的 Cloud 副本将取消
θ_{crit}	高级降级容量使用率阈值, 超过此值时仅保留 Critical 级别的完整副本策略
BW_{high}	高带宽阈值, 超过此值正常写入
BW_{med}	中带宽阈值, 低于此值限速写入
BW_{low}	低带宽阈值, 低于此值延迟非关键数据
$L(n_s)$	延迟追踪器记录的节点 n_s 的加权平均访问延迟
$w_{recency}$	延迟追踪的近期权重系数

5.3.2 整体方案设计

基于状态感知分配的检查点分级写入技术的整体架构如图 5.1 所示, 自上而下分为五层: Python SDK 接入端负责对接 PyTorch DCP 接口; 分级策略层通过重要性分级引擎和副本策略引擎确定各分片的副本数量与目标节点; 写入执行层按重要性分叉为同步、本域同步+跨域异步、异步三条写入路径并由 Worker 池并发执行; 流控层通过准入门控、

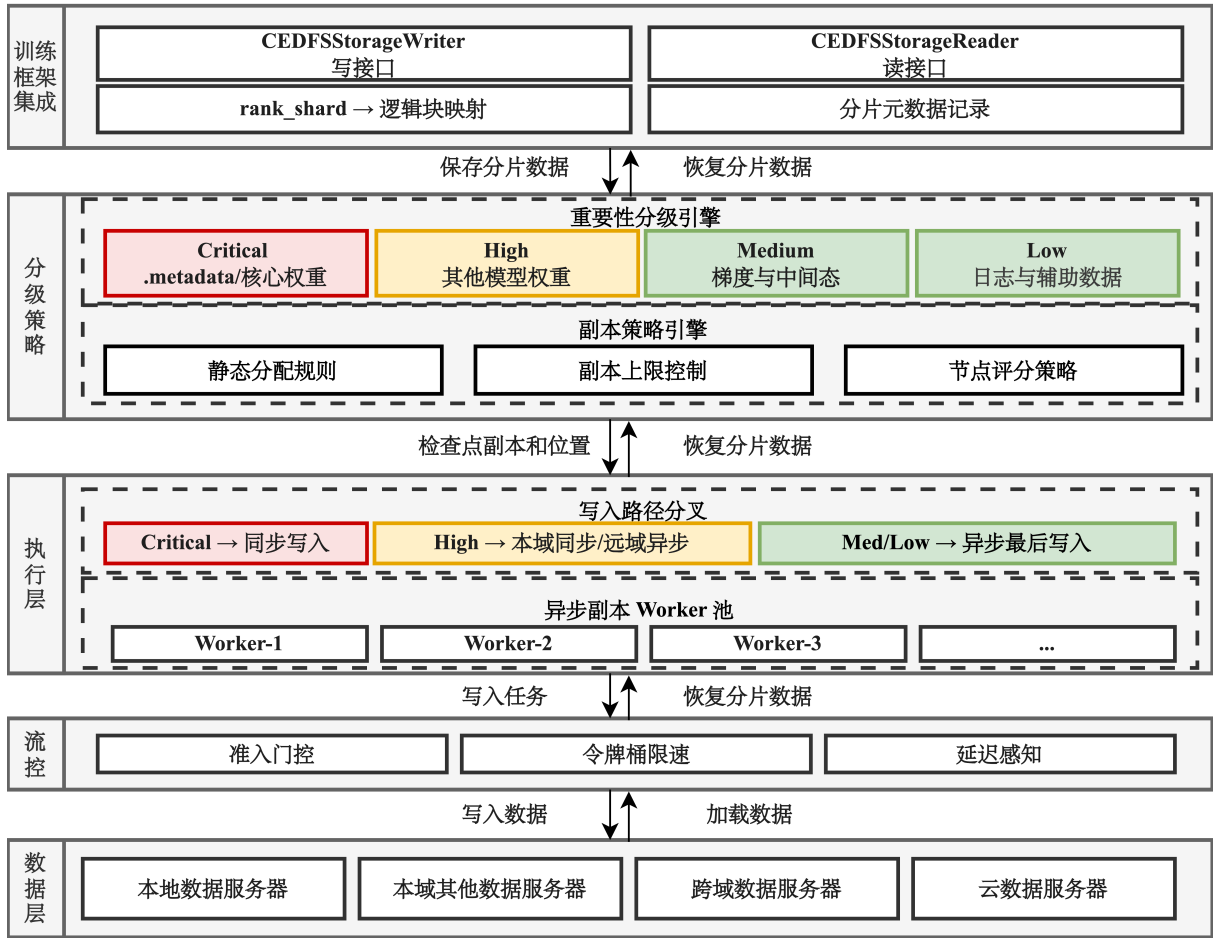


图 5.1 基于状态感知分配的检查点分级写入技术整体架构

令牌桶限速和延迟感知协同管控写入速率；数据层对应本地、本域、跨域和云端四级存储目标。

整体工作流程：如图 5.2 所示，检查点保存前，客户端先刷新集群状态，并据此确定各分片的优先级与目标节点；保存时，关键数据优先完成前台落盘，非关键副本转入后台异步创建；恢复时，系统按从近到远的分层阶梯选择数据源，并按需调整恢复顺序以缩短训练可恢复时间。更细的分配、写入与恢复机制见第 5.3.3 节至第 5.3.6 节。

客户端侧设计：在客户端侧，检查点写入功能通过五层架构实现。

①Python SDK 接入端以 Python SDK 形式提供，实现 PyTorch DCP 的 StorageWriter 和 StorageReader 抽象接口。CEDFSSStorageWriter 封装了完整的检查点保存逻辑，CEDFSSStorageReader 封装了检查点恢复逻辑，二者通过 PyO3 绑定调用 Rust 对应实现。用户在调用 dcp.save()或 dcp.load()时，只需将系统提供的 Writer/Reader 作为参数传入，即可透明启用全部优化功能，无需修改训练代码。

②重要性分级模块负责将检查点内容按其对训练恢复的关键程度分为 Critical、High、Medium 和 Low 四个等级，分级结果直接决定后续的副本数量、写入模式和恢复优先级，具体分级规则将在第 5.3.3 节详述。

③副本策略引擎 (ReplicationPolicy) 根据分片的重要性等级和当前集群状态计算目标节点列表。策略引擎执行五步决策流程：获取基础副本数、应用副本上限、容量过滤、带宽调整、节点选择，具体算法将在第 5.3.4 节详述。

④写入线程池 (WriteThreadPool) 提供异步任务执行能力，基于 Tokio 运行时的信号量机制控制并发数，通过 mpsc 通道接收写入任务，每个任务携带优先级 (Critical > High > Normal > Low)。

⑤流量控制器 (WriteFlowController) 在上述过程中实现准入控制、速率限制和背压反馈三层协同管控，以平衡多级副本的写入带宽。

数据服务器侧设计：在数据服务器侧，依托 CEDFS 三层部署中的云侧与边侧数据服务器（其总体架构将在第六章统一给出），为检查点写入提供以下两类支撑能力：①节点状态上报：数据服务器定期采集本节点的可用磁盘容量、存储使用率、内存使用率和当前读写吞吐等运行时指标，通过 gRPC 心跳接口向元数据服务器上报；元数据服务器汇

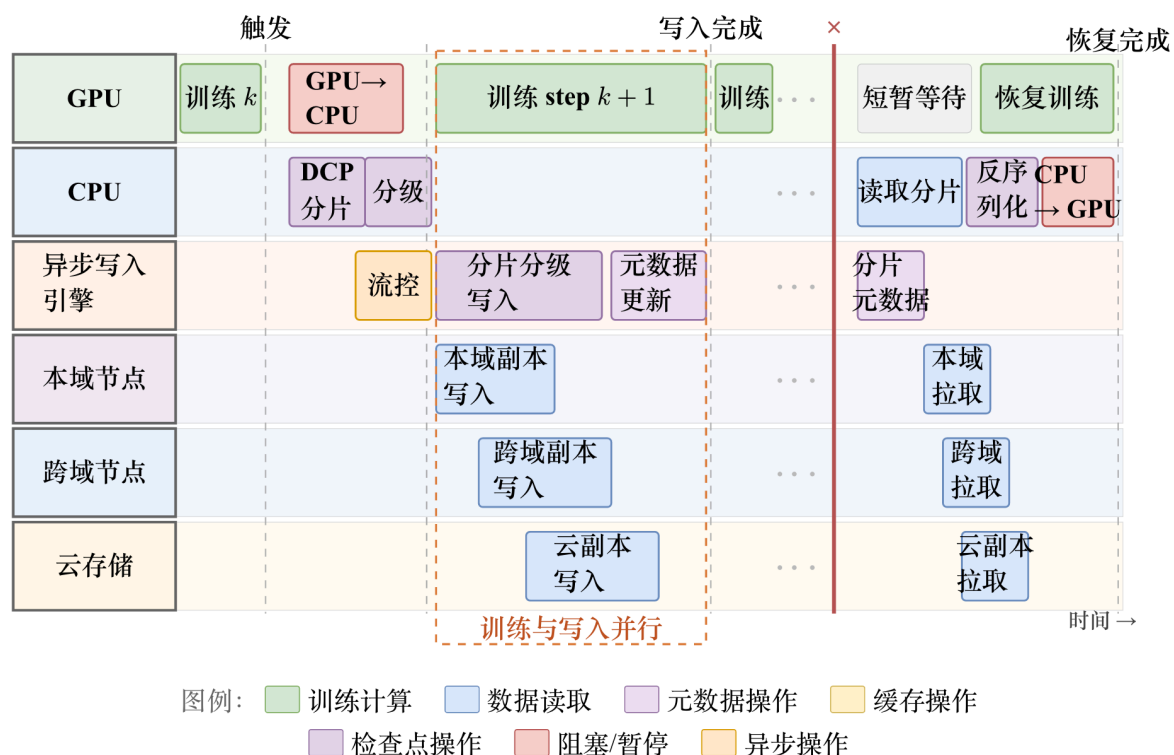


图 5.2 优化后检查点分级写入与恢复流程

聚所有节点的状态信息，在客户端调用 `fetch_cluster_state()` 时返回完整的集群状态快照。

②数据块读写与副本管理：数据服务器同时暴露 gRPC 和 QUIC 双传输接口，gRPC 用于控制平面的块分配和副本请求，QUIC 用于数据平面的高吞吐块传输。

5.3.3 动态资源模型构建

资源模型为检查点分片分配和副本放置提供实时的集群状态感知基础。模型的核心是将边缘集群中各节点的静态拓扑关系和动态运行状态统一建模为可供分配策略直接查询的结构化信息。

在拓扑距离建模方面，系统按 CEDFS 云—边—端三层部署组织节点（具体部署方案见第六章），并据此定义四级拓扑距离模型。设当前写入节点为 n_s ，目标节点为 n_t ， $z(n)$ 表示节点 n 所属的区域标识， $\tau(n)$ 表示节点 n 的层级类型（Edge 或 Cloud），则距离等级 $\text{dist}(n_s, n_t)$ 定义如下：

$$\text{dist}(n_s, n_t) = \begin{cases} 0 & n_t = n_s \\ 1 & n_t \neq n_s \wedge \tau(n_t) = \text{Edge} \wedge z(n_t) = z(n_s) \\ 2 & \tau(n_t) = \text{Edge} \wedge z(n_t) \neq z(n_s) \\ 3 & \tau(n_t) = \text{Cloud} \end{cases} \quad (5.1)$$

其中，距离等级 0 对应本地节点（Local），1 对应域内节点（IntraPeer），2 对应跨域节点（ExtraPeer），3 对应云端节点（Cloud）。

在动态状态采集方面，集群状态快照包含每个节点的以下运行时指标：可用存储容量 $\text{cap}(n_j)$ 、存储容量使用率 $\text{cap_ratio}(n_j)$ 、当前可用网络带宽 $\text{bw}(n_j)$ 、读写吞吐量和内存使用率。这些指标由数据服务器通过周期性的心跳机制上报至元数据服务器，CEDFSStorageWriter 在每次 `set_up_storage_writer()` 调用时通过 `fetch_cluster_state()` 获取最新快照。在实际部署中，状态快照的新鲜度取决于心跳间隔（默认 5 秒），对于绝大多数的边侧训练而言，该新鲜度满足实时性需求。

在延迟追踪方面，路由模块中的延迟追踪器（SourceLatencyTracker）以滑动窗口方式持续记录各存储源的访问延迟。每个源维护最近 w 条（默认 $w = 10$ ）延迟记录，实时延迟估计采用近期加权平均：

$$L(n_s) = \frac{\sum_{i=1}^w w_{\text{recency}}^{w-i} \cdot l_i}{\sum_{i=1}^w w_{\text{recency}}^{w-i}} \quad (5.2)$$

其中 l_i 为第 i 条延迟记录（按时间升序）， w_{recency} 赋予近期记录更高权重。每条记录携带时间戳，超过存活时间的记录在查询时自动过期。该延迟追踪信息主要服务于故障恢复阶段的源选择（第 5.3.6 节）。

5.3.4 基于重要性分级的检查点分片与动态分配

检查点分片与动态分配是本方法的核心决策环节，负责根据检查点内容的重要性和集群的实时状态，为每个分片确定副本数量、写入模式和目标节点。该环节由重要性分级、副本策略决策和节点选择三个步骤组成。

重要性分级：检查点内容按其对训练恢复的关键程度分为 Critical、High、Medium 和 Low 四个等级，每个等级对应差异化的基础副本数量和写入模式。表 5.2 给出了一组示例配置，具体参数值将根据实验环境在第 5.4 节中确定。

表 5.2 检查点内容重要性分级与基础副本默认配置

重要性等级	内容类型	Cloud 副本	Edge 副本	Cloud 上限	Edge 上限
Critical	全局分片映射文件（.metadata）	2	3	5	10
High	模型权重（weight/bias）	1	2	2	3
Medium	通用参数	1	1	1	2
Low	优化器状态	0	1	0	2

分级通过该机制实现：①优先进行路径匹配：文件路径后缀为 .metadata 的全局分片映射文件自动归为 Critical 级，因其记录全局分片到存储位置的映射关系，是恢复的必要前提。②FQN 匹配：当分片携带 PyTorch DCP 提供的完全限定名列表时，扫描所有 FQN 关键词，包含 weight 或 bias 的归为 High 级（模型权重决定训练精度），仅包含 exp_avg、exp_avg_sq、momentum、step 等优化器关键词的归为 Low 级（优化器状态可在恢复后重建），其余归为 Medium 级。

副本策略决策：副本策略引擎（ReplicationPolicy）接收分片的重要性等级和当前集群状态，通过五步决策流程输出副本分配方案（ReplicaDecision），包含目标 Cloud 节点列表、目标 Edge 节点列表、同步/异步写入模式和节流速率：

①获取基础副本数：从表 5.2 中查询重要性等级对应的 Cloud 基础副本数 $R_{\text{cloud}(\text{cr})}$ 和 Edge 基础副本数 $R_{\text{edge}(\text{cr})}$ 。例如，High 级别的模型权重默认需要 1 个 Cloud 副本和 2 个 Edge 副本。

②副本上限裁剪：取基础副本数与配置的最大副本数取小值，防止在大规模集群中副本数爆炸式增长。

③容量降级：根据集群中各节点的容量使用率进行降级调整：容量使用率超过 $\theta_{\text{skip}} = 80\%$ 的节点被直接排除出候选列表；当集群整体容量使用率超过 $\theta_{\text{low}} = 70\%$ 时，Low 级别分片的副本数强制降为 1；当集群整体容量使用率超过 $\theta_{\text{med}} = 85\%$ 时，Medium 级别分片的 Cloud 副本取消；当集群整体容量使用率超过 $\theta_{\text{crit}} = 90\%$ 时，仅保留 Critical 级别的完整副本策略，其他级别大幅缩减。

④带宽调整：根据当前网络带宽状态分五级调整写入策略：带宽高于 BW_{high} 时正常写入所有副本；带宽在 BW_{med} 至 BW_{high} 之间时限制非 Critical 副本在 80% 带宽下写入；带宽在 BW_{low} 至 BW_{med} 之间时延迟 Medium 和 Low 级别的副本写入；带宽低于 BW_{low} 时仅写入 Critical 级别数据；极端情况下带宽降至 0 时触发熔断，仅尝试本地写入。

⑤节点选择：当 Advisory 服务可用时，在每个距离层级内，客户端首先尝试通过 Advisory 服务的 GetDataLocationHints 接口获取各候选节点的综合评分，该评分融合了节点的实时存储容量、当前写入负载和网络可达性，候选节点按综合评分降序排列；当 Advisory 服务不可用时，回退为按可用容量降序排列。上述五步决策流程如图 5.3 所示。

5.3.5 基于异步传输通道的检查点写入与副本创建

检查点写入与副本创建的整体流程按数据流方向分为四个阶段：任务提交、线程池调度、流量控制和异步副本传输。分片的实际写入根据重要性等级采用差异化的执行策略，并通过异步传输通道将副本写入过程与前台训练解耦，在不影响训练性能的前提下完成跨域数据冗余。

①在任务提交阶段，不同重要性等级的分片进入不同的写入通路。Critical 级别的全局分片映射文件（.metadata）在 finish() 阶段以同步方式写入所有目标边缘节点和云端节点，确保恢复元信息的最高可靠性，不经过线程池排队。High 级别的模型权重以同步方式写入最近的边缘节点，写入成功后生成异步副本任务进入后续的副本传输阶段。Medium 和 Low 级别的数据则直接作为异步任务提交至写入线程池。

②在线程池调度阶段，写入线程池（WriteThreadPool）基于 Rust Tokio 异步运行时实现，以信号量控制并发写入数，通过带优先级的 mpsc 通道实现任务调度。每个写入任务携带优先级标签（Critical > High > Normal > Low），线程池优先执行高优先级任务。任务去重机制通过去重键防止同一分片被重复提交：提交任务时检查去重集合，若存在相同键的进行中任务则直接跳过，避免前台重试导致的冗余写入。

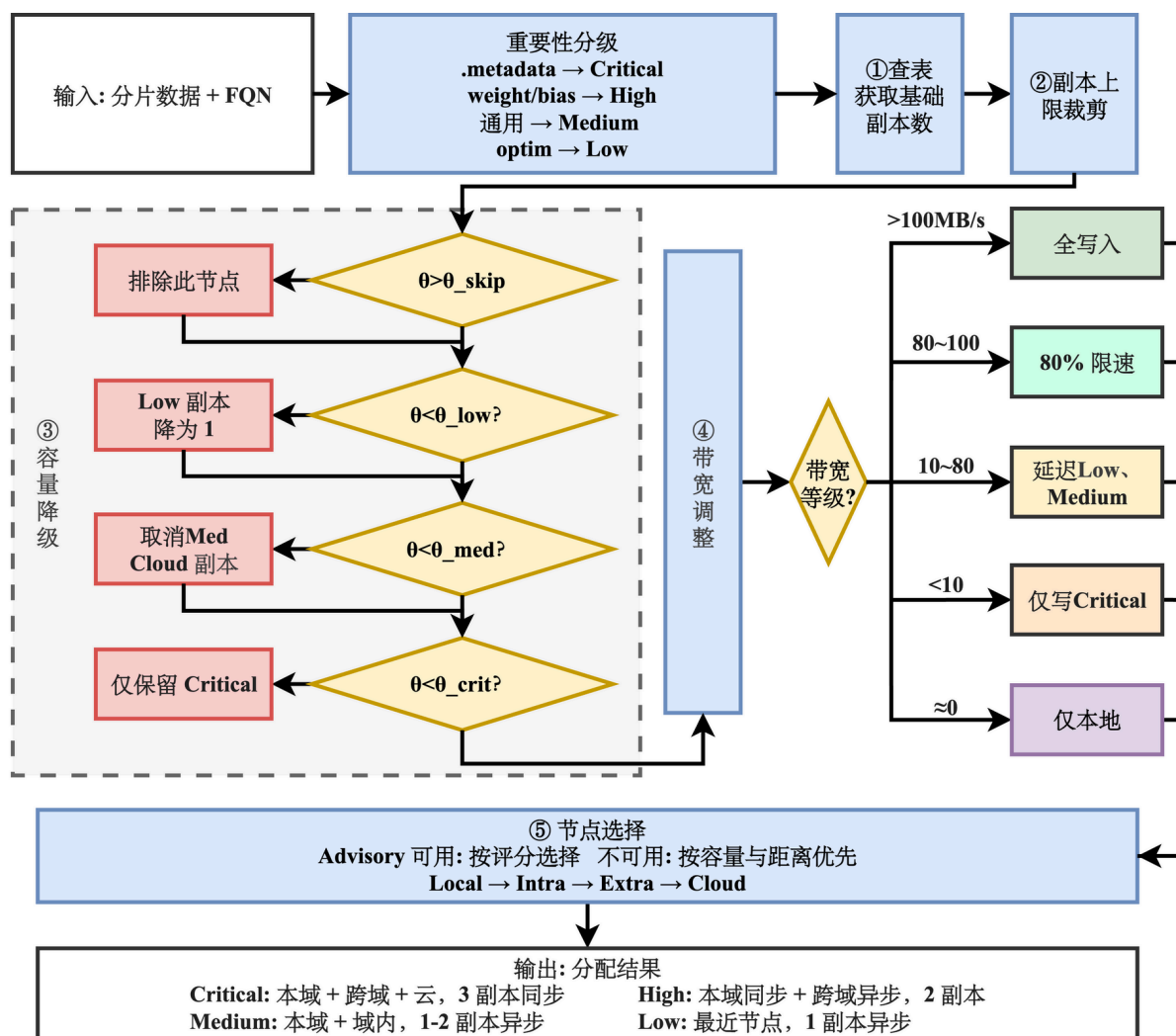


图 5.3 检查点分片动态分配决策流程

③在流量控制阶段，流量控制器（WriteFlowController）实现三层协同的流控架构。准入控制层通过信号量控制同时进行的写入操作数，超过并发上限时阻塞新任务的提交。速率限制层采用令牌桶算法，以配置的最大写入带宽为速率生成令牌，每次写入消耗与数据量对应的令牌数，令牌不足时写入任务排队等待。背压层监控任务队列的排队时间，当平均排队时间超过阈值时向上层反馈拥塞信号，触发低优先级任务的延迟或丢弃策略。三层机制确保：Critical 级别数据始终获得写入通路、High 级别数据在合理时间内完成写入、Medium 和 Low 级别数据在系统负载允许时异步处理。对于 Low 级别数据，在极端拥塞场景下，流量控制器可选择丢弃以保护关键数据的写入通路。

④在异步副本传输阶段，异步副本工作线程（AsyncReplicationWorker）作为独立的后台任务运行，通过 tokio mpsc（多生产者单消费者）通道接收副本任务。每个任务包含数据的字节内容、目标节点地址、重要性等级和超时配置。工作线程从通道中逐一取出

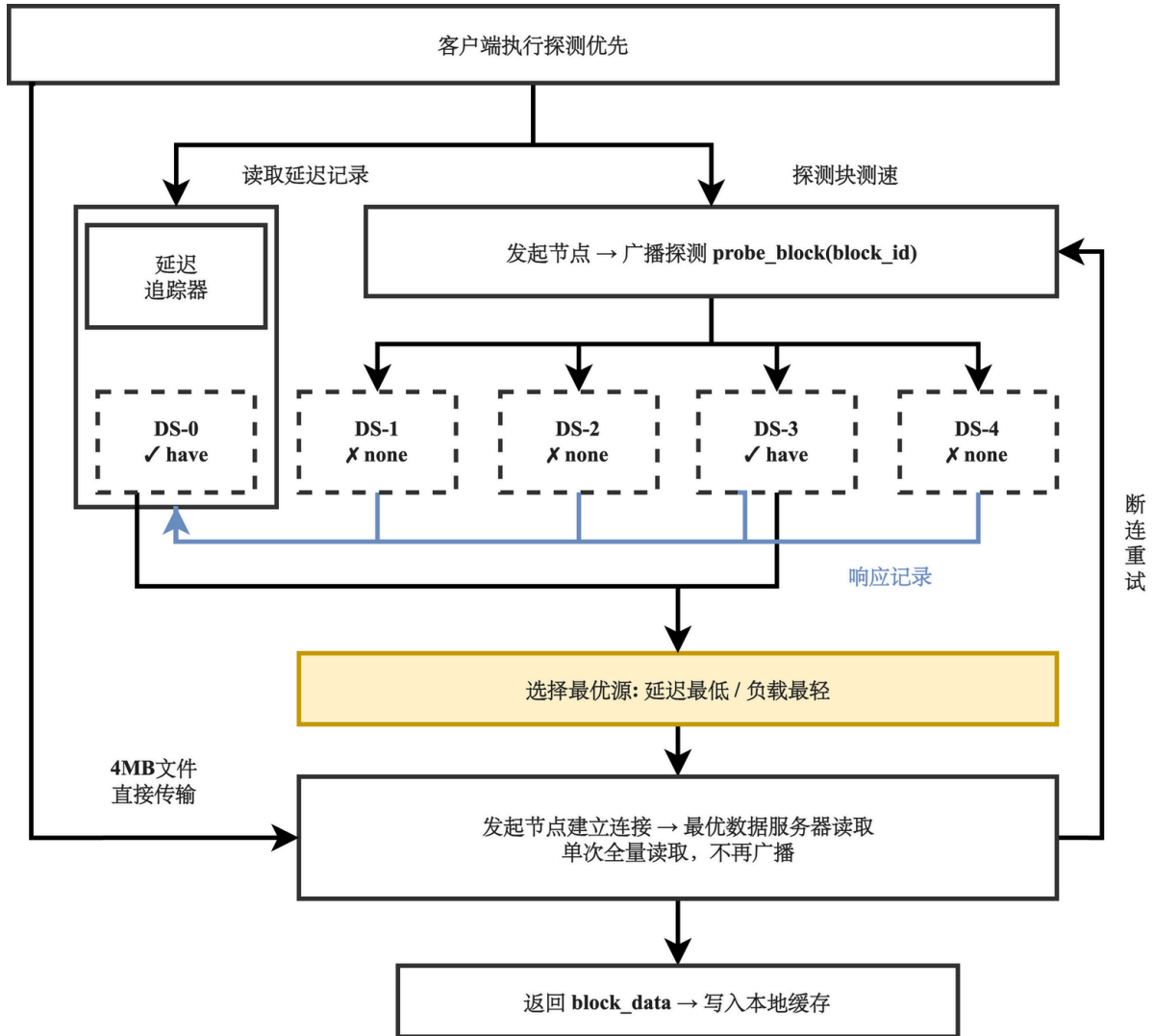


图 5.4 探测优先策略示意图

任务，通过 QUIC 协议将数据传输至目标节点。传输过程中，工作线程对每个任务采用指数退避策略进行重试。

5.3.6 基于分层恢复阶梯的检查点故障恢复

检查点故障恢复是确保分布式训练容错性的关键环节。在云边端环境中，检查点副本分布于多个层级的存储节点上（本地边缘、域内节点、跨域节点、云端），各源的可用性和实时延迟差异显著。恢复管理器（RecoveryManager）通过分层恢复阶梯和探测优先策略，实现快速、高效的检查点恢复。

五级恢复阶梯：恢复管理器按可达性和延迟从优到劣依次尝试。

①本地重试：先尝试从本地读取检查点分片，本地读取具有最低延迟，是最优恢复路径。

②域内节点恢复：若本地读取失败，尝试从同区域的其他边侧节点读取。系统查询资源模型获取同区域的可用边侧节点列表。对于大文件（超过 4 MB），采用探测优先策略选择响应最快的域内节点。

③跨域节点恢复：若域内节点均不可用，扩展至其他域的边侧节点。同样利用探测优先策略，在多个跨域候选源中选择延迟最优的节点进行全量读取。

④云端兜底：若所有边侧节点均不可用，降级至云端存储读取。云端通常具有最高的数据可靠性但延迟较大，作为最后的完整数据恢复保障。

⑤部分恢复：若所有源均不可用且分片的重要性等级为 Low，允许通过阅读器直接重建默认结构（零值张量或空缓冲）代替真实磁盘读取，避免框架因文件缺失触发异常。该策略仅对优化器状态等可重建数据生效，Critical 和 High 级别的数据必须完整恢复，否则恢复失败。

探测优先 (Probe-then-Read) 策略：对于大体积检查点文件的恢复，探测优先策略通过两阶段读取降低带宽开销，其过程如图 5.4 所示。

第一阶段为探测阶段。向多个候选源并发发送小规模探测块的读取请求。探测块大小根据文件总大小动态确定：①4-64 MB 的文件使用 1 MB 探测块，②64 MB-1 GB 的文件使用 2 MB 探测块，③超过 1 GB 的文件使用 4 MB 探测块。所有候选源的探测请求并发执行，记录每个源的响应延迟（包括失败源）。探测阶段结束后，保留响应最快的源返回的探测数据，其他源的数据丢弃。

第二阶段为全量读取阶段。使用探测阶段确定的最快源读取文件的剩余部分，仅使用单源传输。对于小文件（4 MB 以下），由于探测开销与文件大小相当，退化为传统的单源直接读取。

延迟追踪器的缓存复用进一步优化了探测开销：若延迟追踪器中存在目标源的未过期延迟记录，直接使用加权平均延迟选择最优源，跳过探测阶段进入全量读取。

多 rank 协调恢复：在分布式训练中，当多个 rank 同时触发检查点恢复时，各 rank 独立发起的恢复请求可能集中访问相同的存储源节点，导致带宽竞争和恢复延迟显著增加。为此，系统引入恢复协调器 (RecoveryCoordinator)，通过 Advisory 服务实现多 rank 恢复请求的全局协调。

恢复协调的工作流程如下：①各 rank 在开始恢复前，将自身的恢复计划（包含待读取的分片列表、各分片的候选源节点和预估数据量）通过 gRPC 接口提交至元数据服务器的 Advisory 服务的 CoordinateRecovery 端点；②Advisory 服务可用时，在 5 秒超时窗口内

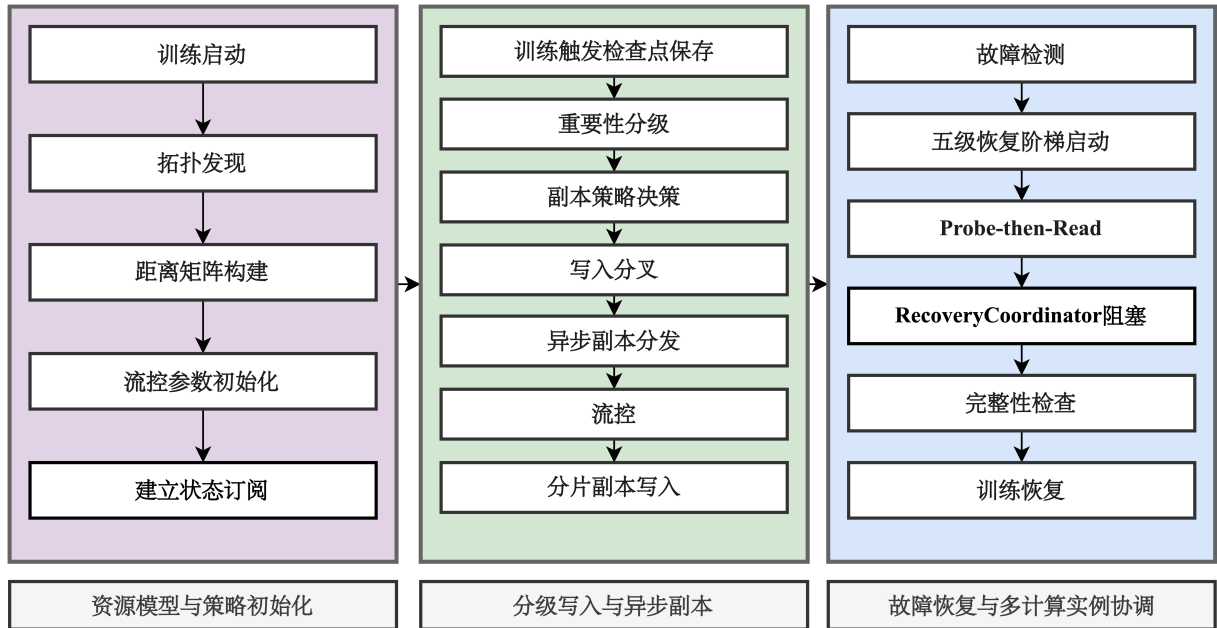


图 5.5 基于状态感知分配的检查点分级写入技术端到端执行流程

收集所有 rank 的恢复计划，基于全局视图进行协调——计算各存储源节点的总负载，为每个源分配带宽上限 (per_source_bandwidth_limit)，并为每个 rank 返回建议的源选择顺序，使不同 rank 尽量分散到不同的存储源节点上，避免热点源被多个 rank 同时集中访问；③当 Advisory 服务不可用时，各 rank 回退为独立恢复模式，按五级阶梯自主选择源节点，功能不受影响，仅在多 rank 并发恢复时可能因带宽竞争导致恢复时间略有增加。

恢复执行流程：①首先，恢复管理器读取全局分片映射文件 (.metadata)，获取检查点的全局分片映射；②随后，若 Advisory 服务可用，各 rank 将恢复计划提交至恢复协调器获取全局协调后的源选择建议和带宽分配；③接下来，对每个分片按五级阶梯依次尝试读取，每级结合延迟追踪信息、恢复协调建议和探测优先策略选择最优源；④最后，读取成功后将数据返回至 PyTorch DCP 框架完成状态恢复。各 rank 的恢复过程在协调完成后并行执行。恢复管理器在恢复过程中持续更新延迟追踪器：每次读取操作的延迟均被记录，供后续源选择使用；连续多次失败的源会积累较高的延迟值，自然被优先级排序降低，实现隐式的故障源回避。

5.3.7 整体执行流程分析

综合第 5.3.2 节至第 5.3.6 节的设计，本方法的端到端执行流程可分为三个阶段，如图 5.5 所示。

第一阶段为资源模型构建与策略初始化。每次检查点保存前，CEDFSSStorageWriter 通过 `fetch_cluster_state()` 获取最新的集群状态快照，计算各节点到本地节点的拓扑距离并更新副本策略引擎的节点信息（第 5.3.3 节）。流量控制器和写入线程池在首次调用时懒初始化，后续保存操作复用已有资源。

第二阶段为分级写入与异步副本创建。PyTorch DCP 将检查点数据传入 `write_data()` 方法后，系统对每个分片进行重要性分级（第 5.3.4 节），查询副本策略引擎获取分配方案，通过流量控制器获取写入令牌后提交至写入线程池执行。Critical 和 High 级别数据的同步写入在返回前完成，异步副本通过后台工作线程传输至远端节点（第 5.3.5 节）。写入流程中，流量控制器保障关键数据的写入通路不被低优先级数据阻塞。

第三阶段为故障恢复。训练因故障中断恢复时，CEDFSSStorageReader 通过恢复管理器执行恢复流程：若 Advisory 服务可用，各 rank 先将恢复计划提交至恢复协调器进行全局协调以避免带宽竞争；随后按五级阶梯依次尝试读取检查点数据（第 5.3.6 节）。大文件采用探测优先策略节省恢复带宽，延迟追踪器的缓存复用进一步加速源选择。恢复过程支持按重要性分级处理，关键数据必须完整恢复，低优先级数据允许端侧读拦截并按需重建默认缓冲以应对极端故障场景。

5.4 实验与验证

本节通过多组实验验证检查点分级写入与多级恢复方案的有效性。

5.4.1 实验环境与参数配置

本章实验在阿里云弹性计算服务（ECS）集群上进行，集群由 10 台虚拟机组成，节点按云端、边缘域 A、边缘域 B 三域部署：云端 1 台节点承担云端元数据根服务器与数据服务器，边缘域 A 部署 4 台节点（含边缘元数据子服务器、数据服务器与训练客户端），边缘域 B 部署 5 台节点（结构同边缘域 A）。两个 VPC 通过对等连接互通，基础跨 VPC 延迟约 0.6 ms，同 VPC 延迟约 0.17 ms；跨域链路上额外使用 `tc/netem` 注入与本章场景对应的延迟、抖动与带宽约束。

检查点数据集：本章实验采用 ResNet-50 模型的完整训练检查点作为标准测试负载，分级规则沿用第 5.3.4 节，不再重复展开。写入性能实验使用 175 MB 规模的检查点，恢复性能实验使用 239 MB 规模的检查点；对后者而言，训练可恢复的关键路径数据约为 94 MB，其余约 145 MB 作为可延后处理部分。跨域上传敏感性实验额外使用 100 MB、500 MB 和 1000 MB 三种规模的合成检查点。

关键评测指标：本章实验涉及的关键评测指标及其含义如表 5.3 所示。

表 5.3 第五章关键评测指标

指标	含义
写入阻塞时间	检查点保存操作阻塞训练的时间，即从调用 <code>dcg.save()</code> 到训练可继续的耗时
总完成时间	检查点所有副本（含异步副本）全部写入完成的端到端时间
恢复时间 RTO	从发起检查点恢复到训练可继续的时间
训练可恢复时间	分级恢复中关键数据加载完成、训练可继续执行的时间
训练可恢复时间缩短比例	分级恢复相对于均匀下载策略所节省的训练可恢复时间占比
传输时间	检查点数据传输至目标节点的耗时

实验参数配置：影响实验结果的关键参数如表 5.4 所示。

表 5.4 第五章实验关键参数配置

参数	描述	默认值
<code>write_threads</code>	SDK 并行写入线程数	4
<code>importance_levels</code>	重要性分级数	4 (Critical/High/Medium/Low)
<code>recovery_levels</code>	恢复阶梯级数	5
<code>capacity_skip_threshold</code>	节点跳过容量使用率阈值 θ_{skip}	80%
<code>capacity_low_threshold</code>	Low 级降级容量使用率阈值 θ_{low}	70%
<code>capacity_med_threshold</code>	Medium 级降级容量使用率阈值 θ_{med}	85%
<code>capacity_crit_threshold</code>	Critical 级降级容量使用率阈值 θ_{crit}	90%
<code>probe_size</code>	探测式恢复探测块大小	1~4 MB（按文件大小动态调整）
<code>latency_window</code>	延迟追踪滑动窗口大小 w	10

5.4.2 端到端检查点写入与恢复方法对比

本组实验将本章方案与现有检查点保存方法进行端到端性能对比。

为了评估不同检查点保存方法的写入阻塞时间和恢复时间，选取四种方法进行对比：①`torch.save` 为 PyTorch 原生本地保存，仅写入本地，无远端备份；②PyTorch DCP 为 PyTorch 分布式检查点接口，分片写入本地；③CEDFS-bypass 为 CEDFS 同步网络传输模式，关闭本地缓存和异步机制，每次写入均通过同步传输至远端；④CEDFS-full 为本章完整方案。四种方法均使用 175 MB 的 ResNet-50 检查点，在 cloud-edge 网络配置下测试，如图 5.6 所示。

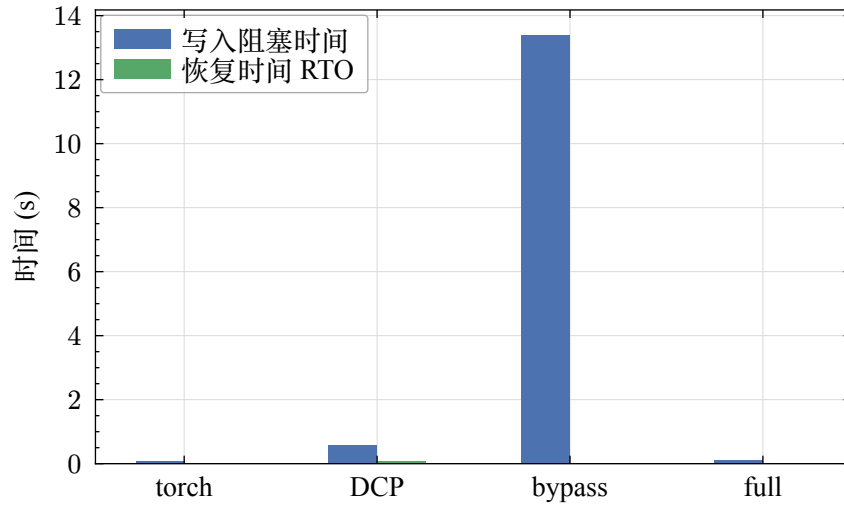


图 5.6 四种检查点保存方法的写入阻塞时间与恢复时间对比

图 5.6 直接展示了四种方法在写入阻塞和恢复两个维度上的性能差异。CEDFS-full 的写入阻塞时间仅为 0.105 秒，与 torch.save 的 0.092 秒处于同一量级，而 CEDFS-bypass 的阻塞时间高达 13.375 秒，是 CEDFS-full 的 127 倍。PyTorch DCP 的阻塞时间为 0.589 秒，略高于前两者，因其需完成分片序列化和本地持久化。在恢复时间方面，四种方法均在 0.03 秒左右，差异不显著，因为恢复均从本地 NVMe 读取。该图间接揭示了 CEDFS-full 的核心设计优势：通过异步本地优先写入策略，检查点数据先以 NVMe 写入速度快落盘，训练立即恢复计算，远端副本在后台线程中异步传输，写入阻塞时间与网络条件完全解耦；而 CEDFS-bypass 采用同步网络传输，175 MB 数据在 80 Mbps 带宽下的传输延迟直接叠加为训练阻塞时间。

CEDFS-full 在写入阻塞时间上与 PyTorch 原生本地保存相当，同时自动完成远端冗余副本创建，兼顾了写入效率与数据可靠性。需要指出的是，本组实验中四种方法的恢复时间均约 0.03 秒，是本地 NVMe 数据完好可用的理想情况。然而在实际边缘部署中，节点硬件故障、掉电或磁盘损坏将导致本地检查点数据丢失，此时恢复必须从远端跨域节点拉取完整检查点，网络延迟和带宽限制将使恢复时间从毫秒级跃升至数十秒乃至数百秒。

5.4.3 异步写入阻塞与检查点规模关系分析

本组实验分析异步本地优先写入策略的阻塞时间随检查点规模的变化趋势。

为了验证异步写入对不同规模检查点的适用性，在本地 NVMe 上测量 50 MB 至 1000 MB 七种规模检查点的写入阻塞时间，该时间仅包含本地 NVMe 写入耗时，不含后台异步网络传输部分，如图 5.7 所示。

图 5.7 显示写入阻塞时间与检查点大小呈近似线性关系：50 MB 检查点阻塞 27 ms，200 MB 为 162 ms，500 MB 为 261 ms，1000 MB 为 512 ms，所有规模均不超过 1 秒。该结果表明，异步本地优先写入策略将检查点保存的关键路径缩减为仅本地 NVMe 写入，网络传输被完全移至后台异步执行。对于边缘微调训练中常见的百 MB 至数百 MB 规模检查点，写入阻塞时间在 100~500 ms 范围内，相对于单步训练迭代时间可忽略。

异步本地优先写入策略的阻塞时间仅由本地 NVMe 写入速度决定，与网络条件无关，在各种检查点规模下均能保持亚秒级阻塞，验证了该策略对中小规模边缘模型的实用性。

5.4.4 跨域检查点异步上传性能分析

本组实验分析 CEDFS SDK 在不同带宽条件下的跨域检查点上传性能。

为了评估后台异步上传阶段的实际传输效率，在 5 至 100 Mbit/s 五组带宽配置下，分别测量 100 MB、500 MB 和 1000 MB 三种规模检查点的上传时间，并与理论传输时间（即检查点大小除以链路带宽）进行对比。SDK 采用 4 线程并行写入，目标为远端数据服务器，如图 5.8 所示。

图 5.8 将理论上传时间与 CEDFS SDK 实测时间进行了直接对比。在高带宽段（100 Mbit/s），100 MB 检查点的 SDK 实测时间为 11.95 秒，理论值为 8.5 秒，额外开销约为理论值的 1.4 倍；在低带宽段（5 Mbit/s），1000 MB 检查点的 SDK 实测为 2738.69 秒，理论值为 1639.3 秒，额外开销约为理论值的 1.67 倍。该图揭示了两个现象：其一，在高带宽

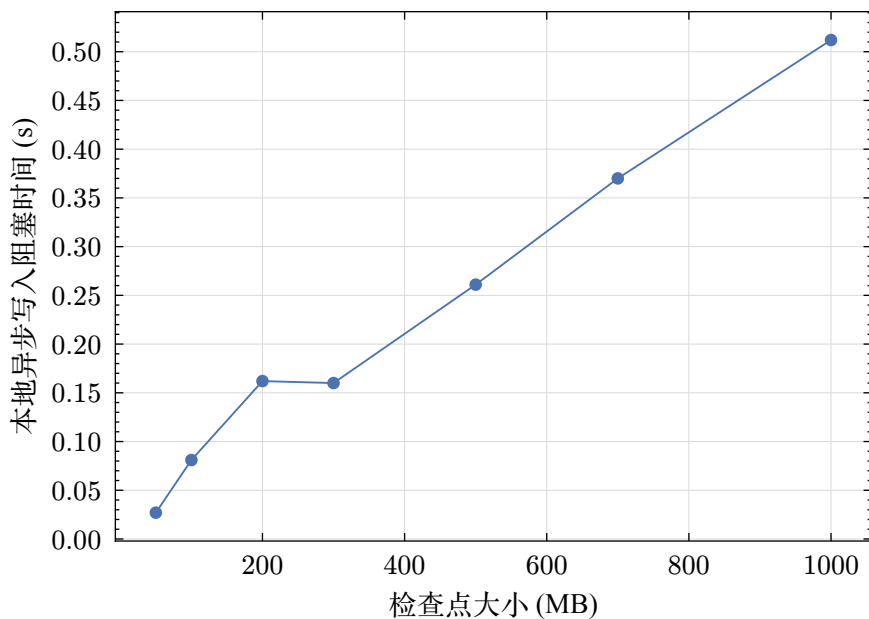


图 5.7 本地异步写入阻塞时间与检查点大小的关系

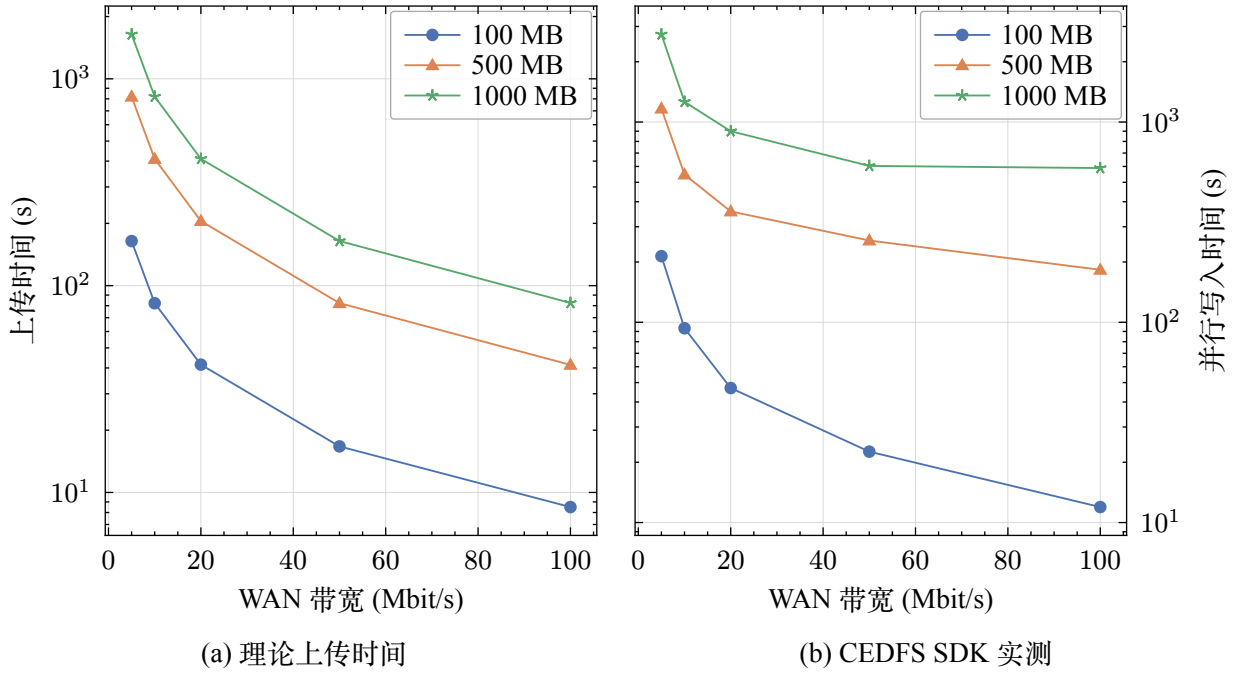


图 5.8 跨域检查点上传时间对比

段 (100 Mbit/s) 实测时间与理论值的差距最为明显, 额外开销达 40%, 说明此时网络已不再是瓶颈, CPU 处理能力成为制约因素——QUIC 协议要求对每个数据包执行加解密运算, 高带宽意味着单位时间内需处理的数据包数量急剧增加, 边缘节点有限的 CPU 资源在并行加密和数据搬运之间产生竞争, 从而拉大了实测与理论的差距; 其二, 在低带宽段 (5~10 Mbit/s), 网络传输时间占绝对主导, QUIC 加解密的 CPU 开销相对可忽略, 实测与理论的比值反而更稳定。

上述结果表明, CEDFS SDK 在各带宽段均保持可用的上传效率, 低带宽下实际瓶颈由网络带宽主导, 高带宽下则需关注 QUIC 加解密对边缘节点 CPU 的压力。对于本文面向的边缘微调场景, 检查点异步上传在后台执行、不阻塞训练前向计算, 因此即使高带宽下存在一定的 CPU 开销, 也不影响训练吞吐, 验证了 SDK 异步上传对云边端跨域环境的适用性。

5.4.5 存储容量感知降级实验分析

本组实验验证容量感知动态分配策略在存储空间不足时的自适应行为。

为了模拟边缘节点存储空间逐步紧张的场景, 人为将 Edge-A 的存储占用率设置为 40%、65%、85% 和 95% 四个水平, 分别对应容量降级阈值 θ_{skip} 至 θ_{crit} 的不同触发条件。使用 158 MB 检查点 (Critical 级 1 MB、High 级 105 MB、Low 级 52 MB) 进行写入, 观察分片在 Edge-A 和 Edge-B 之间的分发路径变化, 如图 5.9 所示。

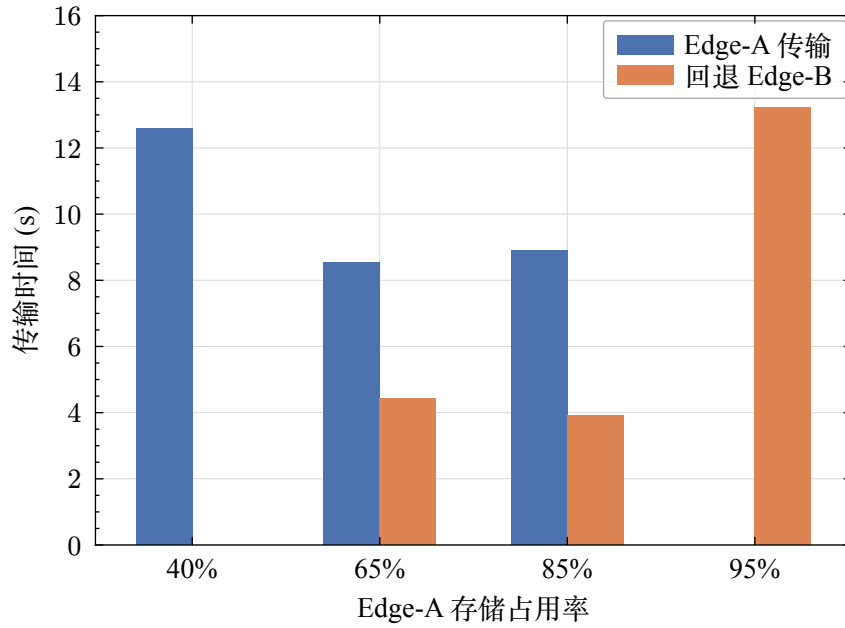


图 5.9 Edge-A 存储占用率对检查点分片分发路径的影响

图 5.9 直接展示了四个占用率水平下检查点分片的分发行为。占用率为 40% 时，所有分片均写入 Edge-A，传输时间 12.59 秒；占用率升至 65% 和 85% 时，High 级分片继续写入 Edge-A，Low 级分片回退至 Edge-B，Edge-A 传输时间缩短至 8.5~8.9 秒，Edge-B 承担 3.9~4.4 秒传输；占用率达 95% 时超过 θ_{crit} 阈值，Edge-A 被完全旁路，所有分片回退至 Edge-B，传输时间 13.22 秒。该图间接反映了容量感知策略的渐进降级机制：系统在存储压力增大时优先释放低优先级数据的本域写入资源，保障关键数据的就近写入；当本域节点完全不可用时，分配策略自动回退至跨域备选节点，确保检查点副本的完整创建。

容量感知动态分配策略根据存储水位自动调整分片的分发路径，高存储压力下低优先级数据优雅降级至远端副本，同时高优先级数据尽可能保留就近写入，验证了分配策略对边缘节点存储容量波动的自适应能力。

5.4.6 检查点分级恢复效果实验分析

本组实验验证分级恢复策略在不同网络延迟条件下的加速效果。

为了量化分级恢复相比全量均匀下载在训练可恢复时间上的优势，在 5 ms 至 200 ms 七组 RTT 配置下，分别测量三种恢复策略的训练可恢复时间：均匀下载策略读取完整的 239 MB 检查点后恢复训练；分级恢复策略优先加载 Critical 和 High 级数据约 94 MB 后即可恢复训练，剩余数据惰性加载；仅关键层策略只加载 Critical 级数据。每组配置重复 3 次取均值，如图 5.10 和图 5.11 所示。

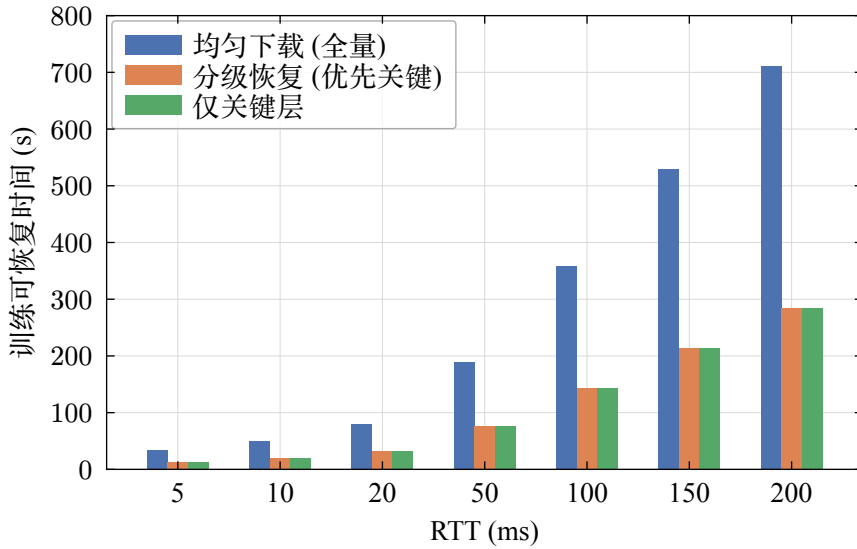


图 5.10 不同 RTT 下三种恢复策略的训练可恢复时间对比

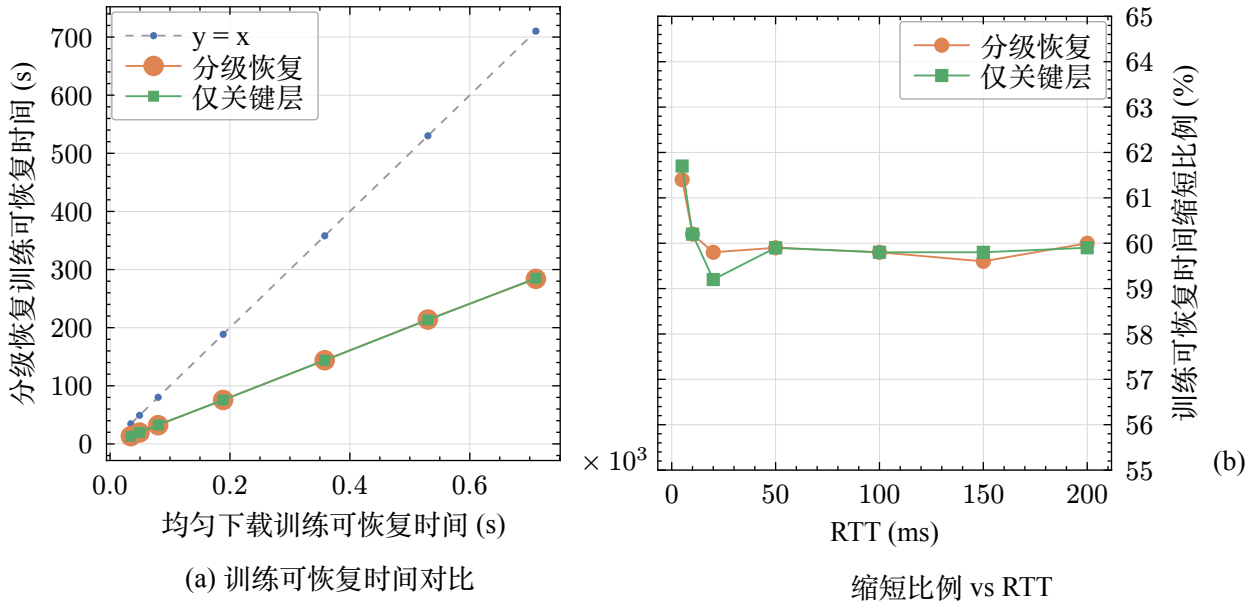


图 5.11 训练可恢复时间对比与缩短比例

图 5.10 直接展示了三种策略在各 RTT 下的训练可恢复时间。在 RTT=5 ms 时，均匀下载耗时 34.61 秒，分级恢复仅 13.35 秒；在 RTT=200 ms 时，均匀下载增至 710.28 秒，分级恢复为 284.03 秒。分级恢复与仅关键层策略的训练可恢复时间几乎一致，因为 Critical 级数据仅 7 KB，对训练可恢复时间的贡献可忽略，实际训练可恢复时间主要由 High 级模型权重约 94 MB 的加载决定。图 5.11 进一步从缩短比例的角度展示了分级恢复的稳定性：子图(a)中分级恢复的数据点落在 $y = x$ 对角线以下的同一水平，子图(b)显示训练可恢复时间缩短比例在所有 RTT 下均稳定在 59.6%~61.4%，几乎保持水平。该恒定比例与数据量裁剪比高度吻合：分级恢复仅需加载约 94 MB 的关键数据，跳过的数据量约占总检

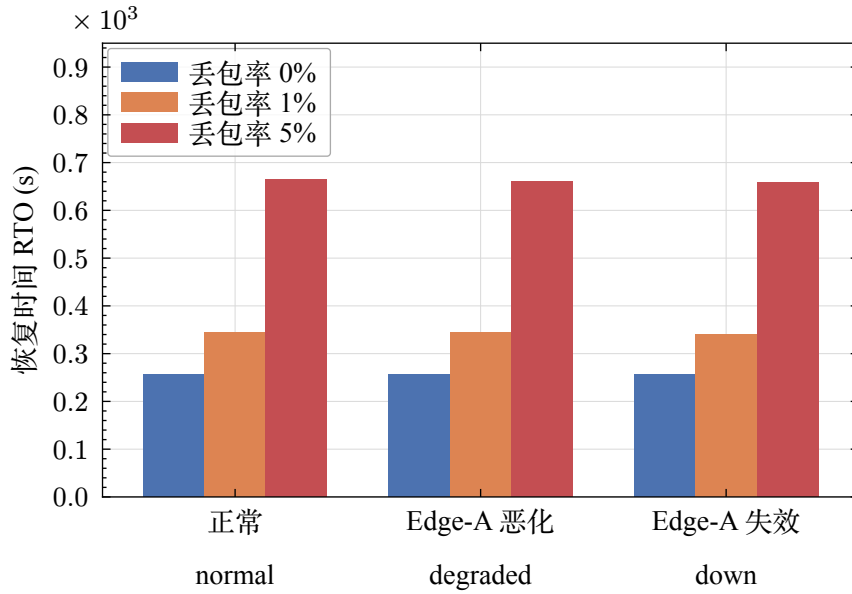


图 5.12 不同节点状态与丢包率下的检查点恢复时间

查点的 60.7% (即 239 MB 中约 145 MB 的低优先级数据), 与实测缩短比例一致, 表明分级恢复的收益由数据量裁剪比决定, 不受网络延迟影响。

分级恢复在所有 RTT 下均稳定缩短约 60% 的训练可恢复时间, 且该收益比例不随网络延迟变化, 验证了按重要性分级加载策略对云边端跨域高延迟环境的良好适应性。

5.4.7 自动路由恢复韧性实验分析

本组实验验证 CEDFS 在节点故障和网络恶化条件下的恢复稳健性。

为了评估自动路由恢复机制在不同故障场景下的表现, 设计三种 Edge-A 状态: 正常 (RTT=5 ms)、恶化 (RTT=80 ms 模拟网络拥塞) 和失效 (iptables 屏蔽模拟节点不可达), 同时叠加 0%、1% 和 5% 三种丢包率, 形成本实验配置。在每组配置下测量 239 MB 检查点的恢复时间, 检查点副本分布在 Edge-A、Edge-B (RTT=30 ms) 和 Cloud (RTT=70 ms) 三个节点上, 每组重复 3 次取均值, 如图 5.12 所示。

图 5.12 直接展示了自动路由机制在各场景下的恢复时间。在 0% 丢包条件下, 三种 Edge-A 状态的恢复时间分别为 257.07 秒、257.22 秒和 257.03 秒, 差异不超过 0.1%; 在 1% 丢包下均在 340~344 秒, 5% 丢包下均在 659~665 秒。该图揭示了两个关键结论: 其一, 同一丢包率下三种故障场景的恢复时间几乎完全一致, 表明 CEDFS 在 Edge-A 恶化或失效时能够透明切换至 Edge-B 或 Cloud, 用户无需人工干预; 其二, 丢包率是决定恢复时间的主导因素, 1% 丢包使恢复时间增加约 34%, 5% 丢包增加约 2.6 倍, 这是 TCP/QUIC 协议在丢包条件下触发重传和拥塞退避的必然结果。

自动路由恢复机制在节点恶化和失效场景下均能透明切换数据源，恢复时间不受单点故障影响，验证了 CEDFS 多路径传输对云边端环境中节点状态波动的良好适应性。

5.4.8 带宽条件对分级恢复的影响分析

本组实验分析网络带宽对分级恢复效果的影响。

为了验证分级恢复在不同带宽条件下的稳定性，固定 $RTT=50\text{ ms}$ ，在 5 至 100 Mbit/s 五组带宽配置下分别测量均匀下载和分级恢复两种策略的训练可恢复时间，如图 5.13 所示。

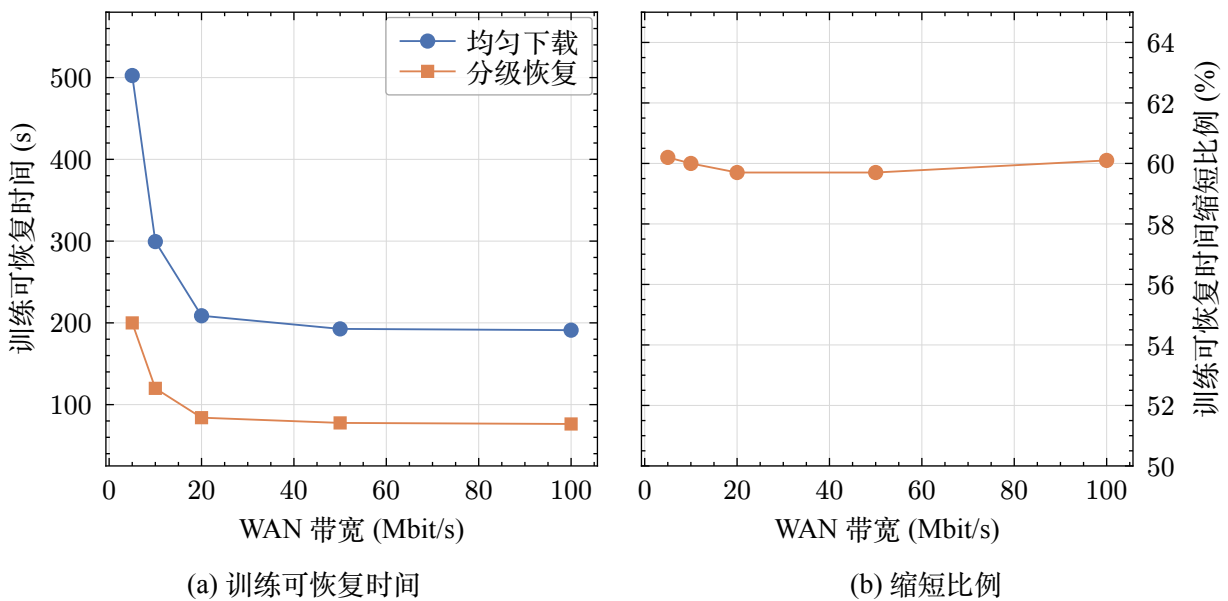


图 5.13 不同带宽下的训练可恢复时间及其缩短比例

图 5.13 的子图(a)展示了训练可恢复时间随带宽的变化趋势。均匀下载在 5 Mbit/s 时耗时 502.54 秒，100 Mbit/s 时降至 191.06 秒；分级恢复在 5 Mbit/s 时耗时 199.95 秒，100 Mbit/s 时降至 76.32 秒。两种策略的训练可恢复时间均随带宽增大而下降，但在 50 Mbit/s 以上趋于平缓，说明高带宽条件下协议处理和 I/O 开销逐渐成为训练可恢复时间的主要下限。子图(b)表明各带宽点上的训练可恢复时间缩短比例与第 5.4.5 节的 RTT 实验基本一致，说明分级恢复在不同带宽条件下都能保持稳定的相对收益。

分级恢复在各种带宽条件下均保持稳定优势，带宽变化主要改变绝对训练可恢复时间，而不改变其相对改进趋势。

5.5 本章小结

本章针对云边端跨域训练中检查点写入受节点状态波动影响大、故障恢复路径不稳定的问题,提出了基于状态感知分配的检查点分级写入技术。该技术通过动态资源模型、重要性分级、副本差异化放置以及分层恢复阶梯,在统一框架下协同优化保存与恢复两条路径。

综合本章实验可以看出,该技术一方面将检查点保存阻塞压缩到接近本地写入的水平,另一方面在跨域恢复场景下稳定缩短约 60% 的训练可恢复时间;同时,容量感知降级和自动路由机制使系统在存储压力增大或节点故障时仍能保持可用的恢复性能。

第六章 云边端跨域虚拟存储空间原型系统与集成验证

6.1 引言

第三章至第五章分别针对云边端跨域分布式训练在冷启动、迭代加载与检查点写入恢复三个阶段的存储瓶颈，提出了元数据批量预取与缓存、异步数据预热以及状态感知的检查点分级写入三项关键技术。三者面向训练全生命周期的不同 I/O 阶段，单独评估时已分别在各章给出有效性验证，但其工程价值最终需要在一个统一的存储系统中协同体现。

本章将上述三项技术整合到自主设计实现的云边端跨域虚拟存储空间系统软件（CEDFS, Cloud-Edge-Device File System）中，作为项目所用跨域分布式训练框架 SpanTrain^[85] 的底层存储支撑，并通过端到端实验验证集成系统在多种网络与负载条件下的整体表现。

6.2 云边端跨域存储空间架构

该系统采用分层架构实现云边端存储资源聚合与统一访问，自底向上分为物理资源层、系统支撑层、用户层，整体架构如图 6.1 所示。

6.2.1 物理资源层

物理资源层是系统的底层存储硬件与介质的物理整合，覆盖云边端三层拓扑节点的各类存储形态^[86]。

云侧节点具备大规模存储能力，同时提供本地文件系统作为数据块存储介质，满足海量数据的可靠存储需求。边侧节点配备中等规模的 SSD 或 HDD 存储设备，同样接入本地文件系统进行数据块存储。云侧和边侧的元数据服务器均可根据集群规模灵活选择存储后端：当部署节点较多时，采用分布式 KV 数据库 TiKV^[87] 实现元数据的分布式管理和自动同步；当节点较少时，采用单机嵌入式数据库 RocksDB^[88] 降低资源开销。端侧节点由 IoT 处理单元和本地文件系统构成，聚焦轻量化存储介质，不部署元数据服务器，通过客户端直接访问存储服务。

6.2.2 系统支撑层

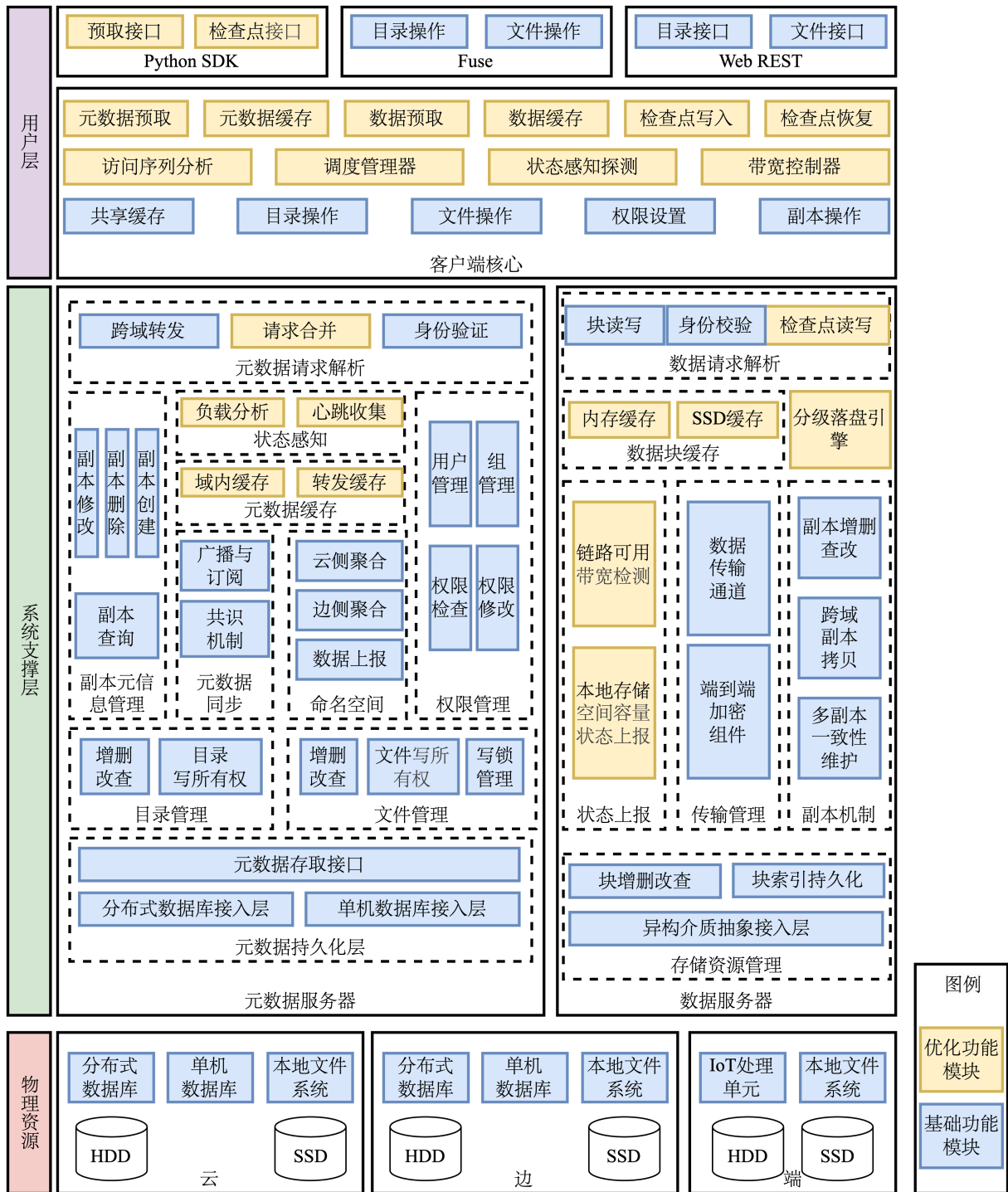


图 6.1 云边端跨域存储空间总体架构图

系统支撑层是架构的功能核心，通过元数据服务器与数据服务器两大子模块协同工作，实现元数据与数据的分离管控。控制平面采用 gRPC 协议处理元数据操作和节点管理请求，数据平面采用基于 QUIC 协议的传输通道完成数据块的高性能传输。

元数据服务器：承担元数据的解析、预取、同步、持久化等核心逻辑。

在元数据组织方面，采用 inode 结构存储文件属性，通过 KV 存储引擎以 inode 编号为键进行索引和持久化，同时维护目录项映射实现路径解析。

在命名空间管理方面，采用子树聚合方法构建层级聚合目录树：非根节点的元数据服务器在启动时向父节点注册自身管理的子树，形成树形命名空间结构；当客户端访问非本域管辖的路径时，元数据服务器通过转发机制将请求路由到目标域的元数据服务器，并通过转发缓存加速后续同路径的访问。

在一致性保障方面，域内元数据服务器通过 KV 存储引擎的事务保证元数据操作的原子性，确保文件创建、目录更新等复合操作的崩溃一致性；跨域元数据操作通过转发机制路由至目标域的元数据服务器执行，由目标域保证操作的本地原子性。

在面向训练的数据加速方面，元数据服务器提供批量元数据请求接口，支持将多条元数据查询合并，同时支持跨域预取请求，批量获取远程域的元数据。

数据服务器：数据服务器聚焦数据读写与存储资源监控。

在数据存储方面，采用固定大小的块结构对文件数据进行分割，每个块通过 Snowflake 算法生成全局唯一的块标识，块索引持久化于同节点部署的 KV 数据库中。

在写入优化方面，数据服务器维护块预分配池，后台异步补充预分配块，避免写入时因同步分配块而产生的延迟。同时支持块锁写入模式，允许客户端持锁直接写入数据服务器。

在副本管理方面，采用可配置的多副本策略，支持按数据重要性分级设定不同的副本策略。副本写入通过后台异步线程池完成，配有任务队列、并发控制和流量控制机制。

在面向训练的数据加速方面，数据服务器同时暴露 gRPC 和 QUIC 两种传输接口：gRPC 用于控制平面的块分配和元数据交互，QUIC 基于 UDP 多路复用用于数据平面的高吞吐块传输。

在状态感知方面，数据服务器维护各节点的可用内存、磁盘容量和使用率等运行时指标，并通过延迟追踪模块记录各存储源的访问延迟信息。

6.2.3 用户层

用户层实现了与底层元数据服务器和数据服务器的交互功能，封装了多种接口类型，为上层应用和分布式训练框架提供统一的访问入口。

FUSE 接口将 CEDFS 挂载为用户态文件系统，支持标准 POSIX 文件操作，使上层应用无需修改即可透明访问云边端跨域存储资源。此外，系统还提供 HTTPREST 接口用于 Web 端数据管理和运维监控。

Python SDK 基于 PyO3 绑定提供 Python 原生调用接口，是本文加速技术的主要承载层。在元数据预取方面；SDK 提供目录递归预取接口，客户端内置元数据缓存，能将预取结果长期驻留；在数据预热方面，SDK 深度集成 PyTorch DataLoader，内置智能预取引擎；在检查点写入与恢复方面，SDK 实现了 PyTorch 分布式检查点 (DCP) 的自定义存储接口，将检查点分片信息提交至客户端核心后，由客户端核心完成分级和异步写入。

在多进程协作方面，系统设计了独立的缓存服务器进程，通过共享内存 IPC 机制，实现 FUSE 进程和 Python SDK 进程共享缓存。

6.3 云边端跨域虚拟存储空间部署与工作机制

CEDFS 采用三层部署模型，各层节点通过路由表维护拓扑关系，每个节点记录域内同层节点、跨域同层节点、上层节点和下层节点四类邻居信息，以支撑灵活的跨域数据访问路由。

6.3.1 各层部署方案

云侧部署元数据服务器根节点和大容量数据服务器，前者根据集群规模接入 TiKV 或 RocksDB 实现元数据持久化，后者提供数据块的可靠归档存储。边侧部署元数据服务器子节点和数据服务器，元数据服务器启动时向云侧注册管辖子树，同样根据集群规模选择 TiKV 或 RocksDB 作为存储后端；数据服务器暴露 gRPC 与 QUIC 双传输接口。端侧仅部署客户端程序，通过 FUSE 挂载或 Python SDK 直接调用存储服务；多训练进程场景下可部署缓存服务器进程，通过共享内存 IPC 实现跨进程缓存共享。各层部署方案如图 6.2 所示。

值得注意的是，CEDFS 基于底层位置透明的通信模型实现了模块物理拓扑解耦，客户端具备强部署适应性与跨域双向数据流 (Push/Pull) 支持。其实际工程不仅可部署于边缘执行微调，也可无缝部署于云端或异构集群发起广义联邦计算；同时，除边缘主动推送 (Push) 外，云端等大容量节点也可以自身为客户端利用下行或空闲带宽反向主动拉取 (Pull) 边缘生成的检查点或样本。

6.3.2 训练过程中的基础访问链路

在未启用加速技术的基础模式下，分布式训练各阶段的数据访问链路如下：

(1) 元数据访问链路：训练节点客户端对每个样本文件逐条向边侧元数据服务器发起 gRPC 查询。元数据服务器在本地 KV 存储中检索 inode 属性；若目标元数据归属其他

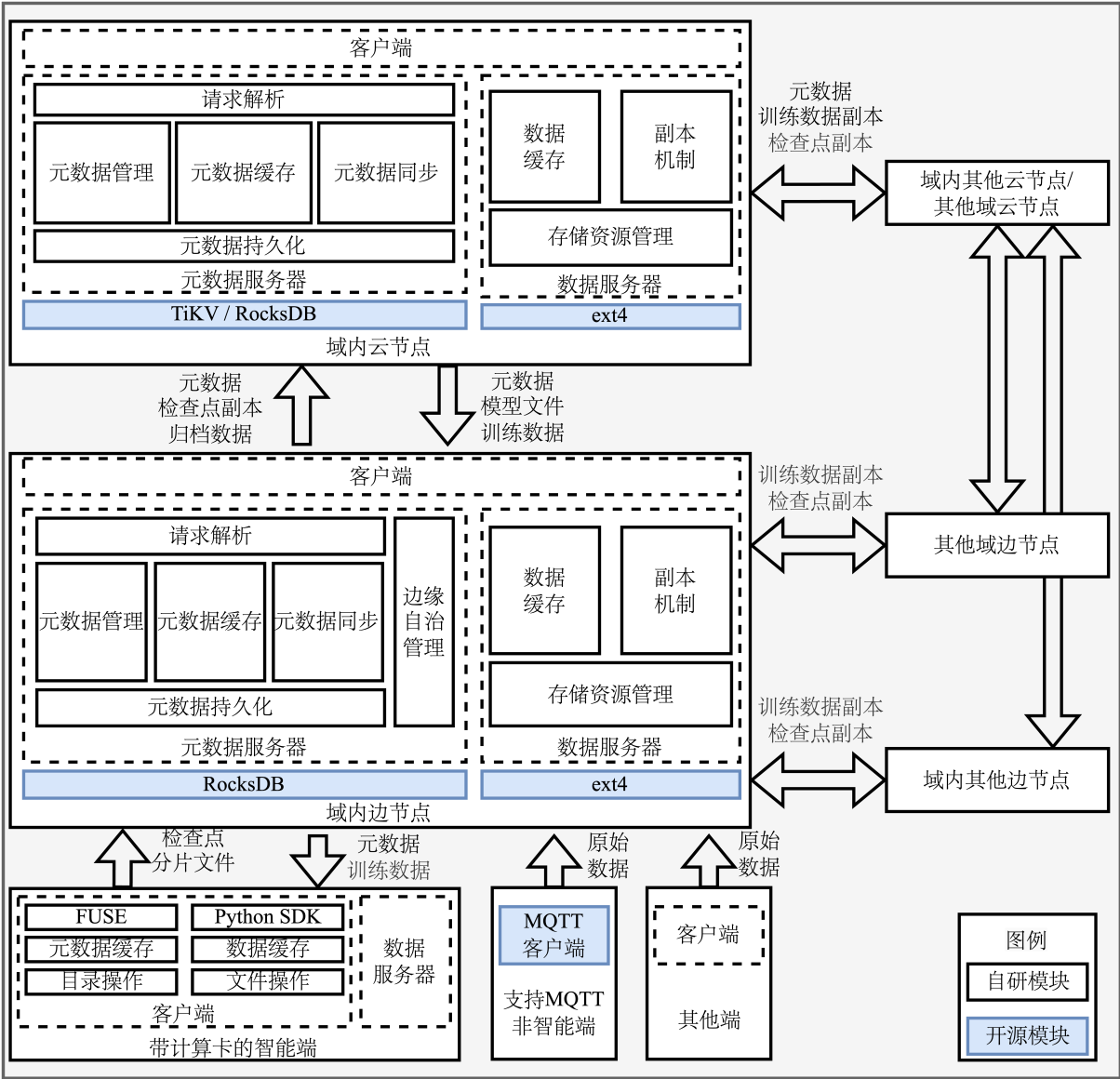


图 6.2 云边端跨域存储空间部署图

域，则通过转发机制路由至目标域的元数据服务器，必要时进一步回溯至云侧根节点获取路由信息。每个文件的属性查询构成一次端到边（或端到边到云）的独立 RPC 往返。

(2) 数据加载链路：训练节点客户端先通过 gRPC 向边侧元数据服务器查询样本文件的块位置映射，获取目标数据服务器地址，再通过 QUIC 协议逐块读取文件数据。数据加载与训练计算串行执行，GPU 需等待当前 batch 的全部数据块加载完成后方可开始计算。图 6.3 展示了基础模式下训练数据加载训练的时序流程。GPU 在初始化和每个 batch 的数据加载阶段均处于阻塞等待状态，数据加载与训练计算完全串行。

(3) 检查点写入链路：训练进程将模型状态从 GPU 显存拷贝至 CPU 内存后，客户端向元数据服务器请求块分配，元数据服务器按默认策略指定写入目标节点，客户端再

经 QUIC 协议将数据块写入该节点。所有检查点数据经过统一的分配和写入路径，写入期间训练进程阻塞等待。



图 6.3 常规数据加载训练流程

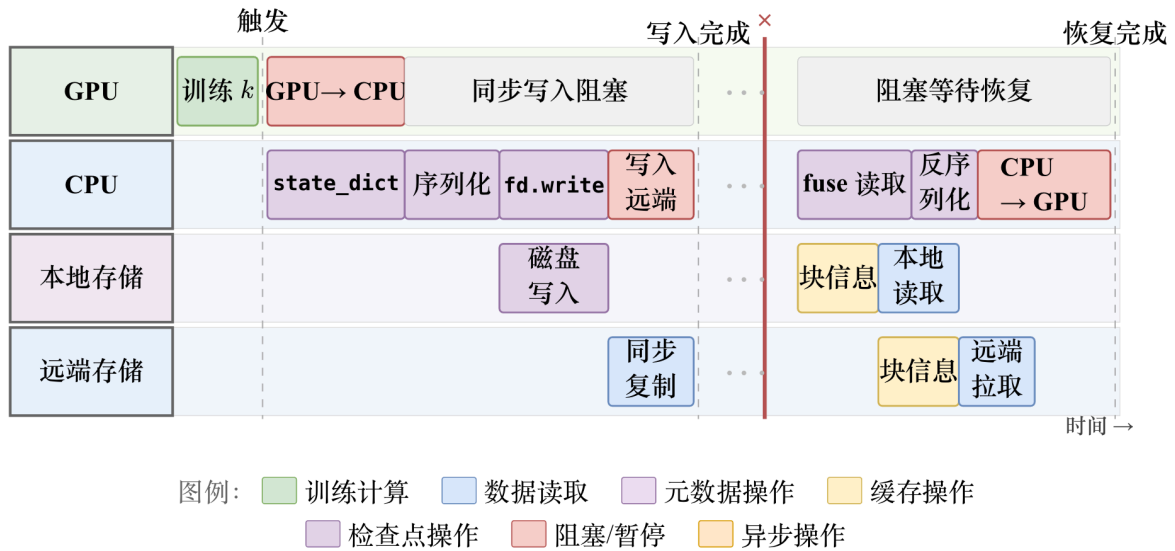


图 6.4 常规检查点写入与恢复流程

(4) 故障恢复链路：训练节点客户端按固定优先级依次尝试本地边缘节点、域内节点、跨域节点、云端节点读取检查点文件，当某一源超时后切换至下一源。图 6.4 展示了基础模式下检查点写入与恢复的时序流程，检查点写入期间 GPU 全程阻塞，故障恢复时同样需要串行完成读取和反序列化，GPU 持续闲置。

6.3.3 加速技术优化后的访问链路

启用第三章至第五章提出的三项加速技术后，CEDFS 在元数据访问、数据加载和检查点写入与恢复三个链路上同时获得优化。在元数据访问链路上，预取引擎在数据集初始化前异步完成目录树批量遍历并写入分层缓存，训练框架后续的属性查询直接命中本地或共享内存缓存，避免逐条跨域 RPC；在数据加载链路上，预取调度器按确定性访问序列提前将远端样本搬运至本地两级缓存，训练数据读取转化为本地命中，数据加载与计算在时间维度上形成稳定重叠；在检查点写入与恢复链路上，分级写入将关键分片以本地优先方式快速落盘并将远端副本异步搬运，恢复阶段则通过分层路由和延迟感知的源选择，按重要性分级加载所需数据。其中，检查点写入与恢复阶段的优化时序如图 5.2 所示，可与上文图 6.4 形成对照；元数据预取与数据预热两条链路的内部机制已分别在第三章和第四章详细阐述，此处不再重复展开。

总体上，三项技术围绕同一基础访问链路协同：元数据预取消除冷启动阶段的跨域往返，数据预热消除迭代阶段的 I/O 阻塞，检查点分级写入与恢复消除模型保存与恢复阶段的长时阻塞，三者共同将云边端跨域训练的端到端阻塞控制在较低水平。

6.4 实验与验证

本节通过系统的端到端实验验证 CEDFS 云边端跨域虚拟存储空间的整体性能。

需要说明的是，本文整体研究视角以边缘集群就近训练、跨域拉取远端数据为核心场景，而本章实验部署中训练任务运行于云端节点、训练数据存储于边缘节点，数据流方向为云端从边缘拉取。这一拓扑选择不影响实验结论的适用性：云边链路两端施加同等的网络状态约束，底层传输链路本质等价，同时，各存储方案所经历的网络条件和软件栈访问路径基本一致，更便于直接进行性能比较。

6.4.1 实验环境与参数配置

硬件与网络部署：本章实验沿用第四章所述的天数智芯三节点物理集群与跨域网络模拟配置（详见表 4.2、表 4.3 和表 4.4）。该配置在云一边链路上同时引入带宽受限与高 RTT，能够同时放大三类瓶颈：链路带宽决定数据预热（第四章）能否及时供给样本，RTT 决定元数据预取（第三章）对冷启动延迟的压缩幅度，二者共同影响检查点分级写入与恢复（第五章）的端到端阻塞时间。

对比存储方案：本章共比较五种存储方案，如表 6.1 所示。CEDFS-full 启用全部三项加速能力，CEDFS-direct 关闭缓存与预热（仍保留 CEDFS 自身架构开销），二者对照可分离出第三至第五章技术的整体收益；NFS 与 JuiceFS 作为传统网络文件系统基线，反映未做训练专用优化时的访问代价；JuiceFS-cached 启用本地数据缓存，用于与 CEDFS 中由数据预热（第四章）主导的缓存命中收益形成对照，避免将命中收益单纯归因于“是否使用了本地缓存”。

表 6.1 第六章实验的对比存储方案

方案	访问路径与说明
CEDFS-full	CEDFS 客户端访问；启用各级缓存与预热功能
CEDFS-direct	CEDFS 客户端访问；关闭各级缓存与预热功能
NFS	NFS v4.2 挂载；依赖内核页缓存
JuiceFS	FUSE 挂载；元数据后端为 Redis、对象后端为 MinIO；关闭本地数据缓存
JuiceFS-cached	FUSE 挂载；元数据后端为 Redis、对象后端为 MinIO；启用本地数据缓存

数据集说明：本章使用两组真实图像数据集，名称、规模与用途如表 6.2 所示。

表 6.2 第六章实验使用的数据集

数据集	类别数	训练样本数	规模
Imagenette2-320	10	9,469	367 MB
Imagenette2	10	9,469	1.5 GB

Imagenette2-320 与 Imagenette2 为两个不同的分辨率版本，分别用于缓存容量宽松与受限的两类实验场景情形。

关键评测指标：本章实验的关键指标如表 6.3 所示。

实验参数配置：为便于说明第六章各组实验共享的默认设定与关键后端配置，主要配置如表 6.4 所示。

6.4.2 带宽敏感性实验分析

本组实验分析弱带宽条件下各存储方案的训练吞吐与冷启动开销。为了隔离带宽因素，固定 RTT 约 4 ms，扫描多组带宽，比较各方案的稳态吞吐和首个 Epoch 加载时间。该组实验中，CEDFS-full 配置 256 MiB 内存加 256 MiB 磁盘的本地缓存，JuiceFS-cached 配置 256 MiB 本地数据缓存并配 256 MiB 用户态缓冲；先选用仅约 367 MB 的

Imagenette2-320,让数据集规模明显小于本地缓存预算,从而优先观察访问路径和序列预热本身带来的收益,避免由缓存容量带来干扰。

表 6.3 第六章实验的关键评测指标

指标	含义
稳态吞吐量	Epoch 1~4 的平均训练吞吐量 (img/s)
冷启动加载时间	Epoch 0 的数据加载耗时 (s)
data_ratio	数据加载时间占总步时的比例,反映 I/O 是否成为主瓶颈
P99 / P50	数据加载尾延迟与中位延迟之比,反映抖动放大程度
P99 延迟	每批数据的尾延迟 (s),反映弱网下最差读取开销
训练总时间	完整训练任务的端到端耗时 (s)
阻塞率	检查点写入阻塞时间占总训练时间的比例

表 6.4 第六章实验的关键配置

参数	描述	默认值
训练框架与默认并行度	默认训练配置方式	PyTorch DDP
world_size	训练进程总数	2
batch_size	训练批大小	64
训练轮数	每组实验的训练轮数	5
训练模型	训练任务使用的模型	ResNet-50/MobileNetV3
训练数据集	训练任务的数据集	Imagenette2-320/Imagenette2
训练优化器	训练任务统一设置	SGD
seed	训练任务的随机种子	42
CEDFS-full 本地缓存预算	默认缓存配置	256 MiB 内存/256 MiB 磁盘
JuiceFS-cached 缓存设置	JuiceFS 用户态缓存	256 MiB 内存/256 MiB 磁盘

图 6.5 显示, CEDFS-full 的稳态吞吐曲线在 10~500 Mbps 范围内始终位于最上方,对应数值由 118.8 img/s 提升到 214.1 img/s。低带宽区间的相对优势最为明显: 10 Mbps 时, CEDFS-full 分别达到 CEDFS-direct、NFS 和 JuiceFS-cached 的 3.2 倍、3.8 倍和 1.28 倍; 到 40 Mbps 时, 仍保持 2.7 倍、3.8 倍和 1.69 倍的领先。进一步看曲线形态, 当带宽提升到 40 Mbps 以上后, CEDFS-full 基本进入平台区, 稳态吞吐稳定在 191.8~214.1 img/s 之间, 波动仅 11%, 说明此时训练关键路径已不再主要受链路带宽限制。相比之下, 其余远端访问方案的曲线更早变平: CEDFS-direct 与 JuiceFS 大致在 100 img/s 附近趋于饱和, NFS 则更早停留在约 60 img/s, 带宽利用率降低。

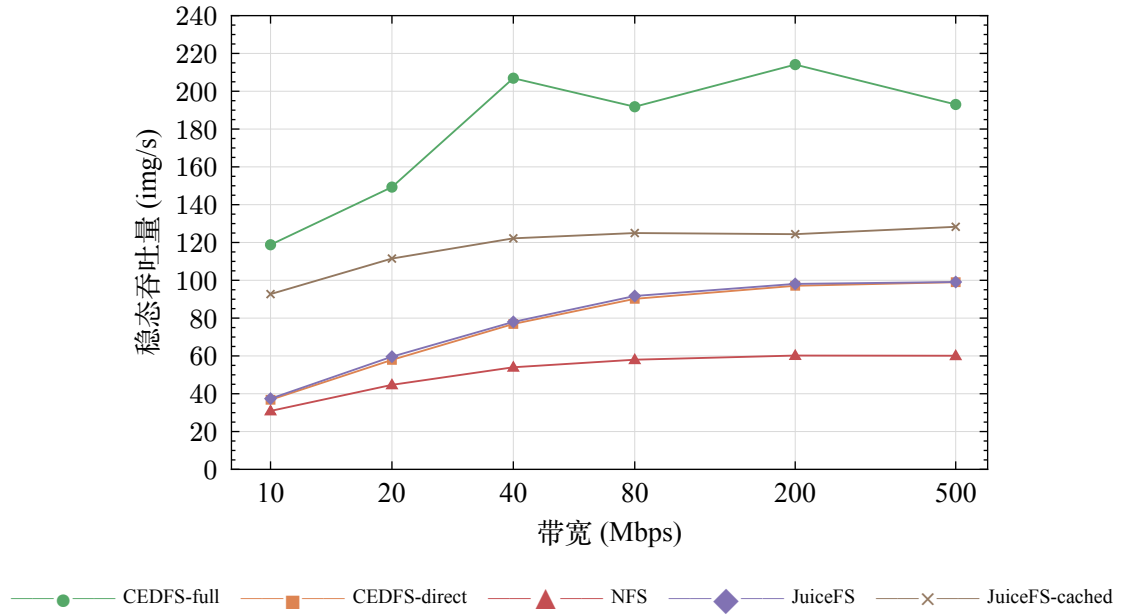


图 6.5 各存储方案稳态吞吐量随带宽变化

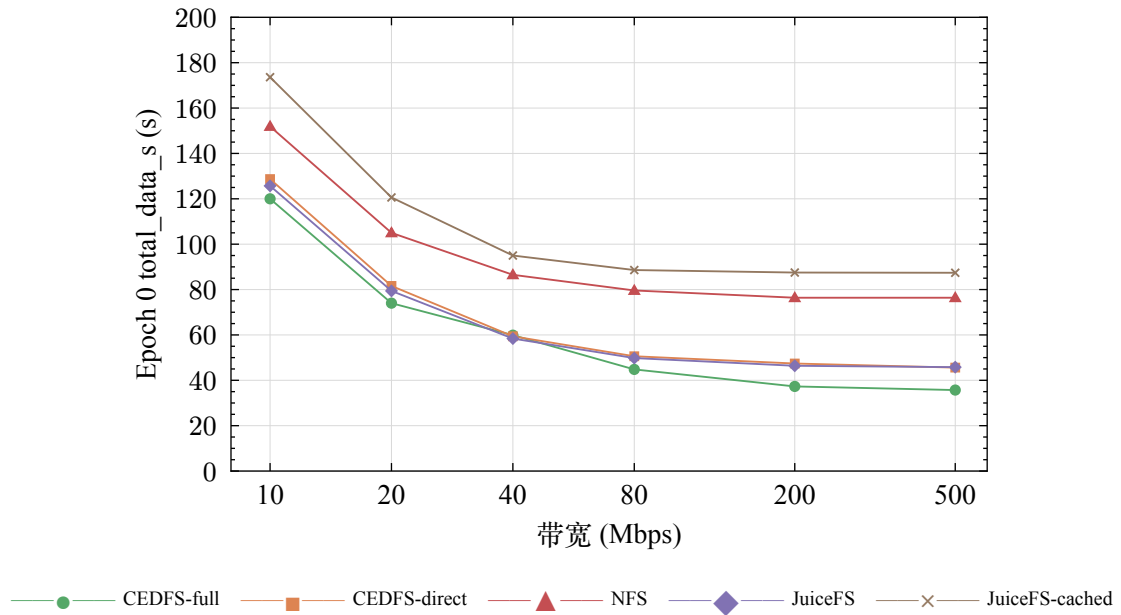


图 6.6 首个 Epoch 总数据加载时间随带宽变化

图 6.6 给出了首个 epoch 的总数据加载时间。CEDFS-full 随带宽从 10 Mbps 提升到 500 Mbps 时，冷启动耗时由 120.0 秒降至 35.7 秒；CEDFS-direct 对应为 128.7~45.6 秒，NFS 为 151.9~76.4 秒，JuiceFS-cached 为 173.6~87.4 秒。这一结果表明，CEDFS-full 在首轮确实承担了跨域数据拉取和本地缓存建立的额外工作，但该额外工作并未转化为明显失控的冷启动开销，其总耗时始终位于最优或近最优区间。

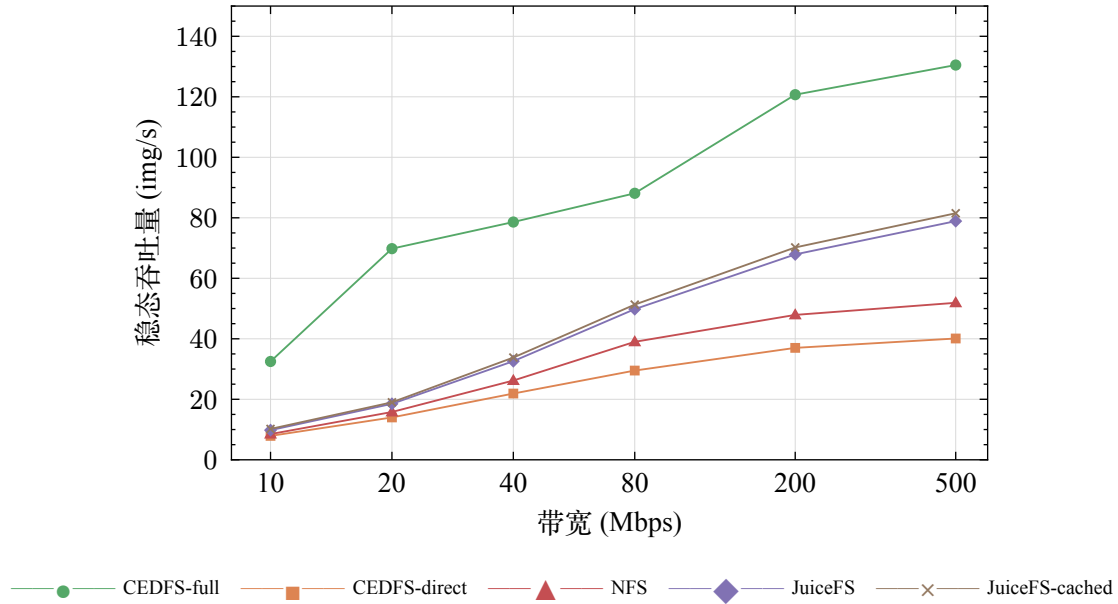


图 6.7 缓存有限下稳态吞吐量随带宽变化

随着链路带宽提高，各方案的稳态吞吐都逐渐偏离理想的线性增长，说明在高带宽条件下，软件栈开销会变高，吞噬训练的有效带宽。相比之下，CEDFS-full 能让新增带宽能够更有效地转化为训练吞吐，因此能在高带宽场景下仍保持最高稳态吞吐。

6.4.3 缓存受限带宽敏感性实验分析

本组实验分析数据规模接近缓存上限时的带宽敏感性。为此将数据集替换为 Imagenette2 全分辨率版本，在其余配置不变的条件下重新扫描 10~500 Mbps 带宽，以观察缓存收益受限后的退化行为。

图 6.7 显示，在缓存受限条件下，CEDFS-full 在 10/20/40 Mbps 时分别达到 32.5、69.8 和 78.6 img/s，领先 NFS 约 3.8 倍、4.4 倍和 3.0 倍，领先 JuiceFS-cached 约 3.2 倍、3.7 倍和 2.3 倍；即使在 500 Mbps 下，CEDFS-full 仍有 130.5 img/s，明显高于 NFS 的 51.9 img/s 和 JuiceFS-cached 的 81.5 img/s。与标准缓存场景相比，本组所有方案的绝对吞吐都明显下降，说明数据集规模一旦逼近或超过缓存预算，链路搬运开销会变大；但 CEDFS-full 的领先幅度依然存在，能将有限缓存预算优先分配，改善训练吞吐。

图 6.8 表明，在全分辨率数据集下，所有方案均会有分钟级冷启动：10 Mbps 时的范围为 566.5~618.1 秒，500 Mbps 时仍有 65.0~131.1 秒。CEDFS-full 在 Epoch 0 冷启动时间并不占优，10 Mbps 时略高于 JuiceFS-cached 和 NFS；不过，即便在 10 Mbps 这一最不利点上，CEDFS-full 相对 JuiceFS-cached 和 NFS 的冷启动耗时也仅高出约 7.3% 和 8.6%。

综合稳态吞吐与冷启动结果可以看出，缓存受限并没有抹平各方案之间的差异。CEDFS-full 虽在冷启动时会带来少量额外开销，但在后继轮次能更高效地利用有限的缓存，更具针对性地提升训练吞吐。

6.4.4 RTT 敏感性实验分析

本组实验分析跨域时延是否会直接进入训练关键路径。为了隔离时延因素，固定带宽 80 Mbps，仅扫描 0.14~100 ms RTT，并同时在 ResNet-50 和 MobileNetV3 上观测稳态吞吐与 data_ratio 变化。

图 6.9 显示，CEDFS-full 在全部 RTT 区间内几乎保持恒定：ResNet-50 为 168~189 img/s，MobileNetV3 为 171~222 img/s。与之相比，对照组在 RTT 超过 4 ms 后快速退化，RTT=100 ms 时 CEDFS-direct、NFS 和 JuiceFS 分别只剩 4.8、3.0 和 8.1 img/s。在训练过程中发起远端同步读取，RTT 就会线性进入关键路径；而 CEDFS-full 通过本地命中和异步预取，吸收了时延扩散。

图 6.10 进一步表明，CEDFS-full 的 data_ratio 在所有 RTT 下都近似稳定，ResNet-50 约 76%，MobileNetV3 约 89%；其余方案则随 RTT 单调恶化，在 100 ms 时均超过 99%。其余方案则随着 RTT 增大，使数据等待时间在总步时中的占比持续上升；CEDFS-full 并未消除 I/O 开销，而是能将其稳定在与 RTT 近似解耦的范围。

因此，本组实验可以看出，CEDFS-full 对 RTT 的敏感性显著更低。当面临云边端跨域环境中较高 RTT 时，CEDFS-full 能避免让 RTT 成为训练的阻塞。

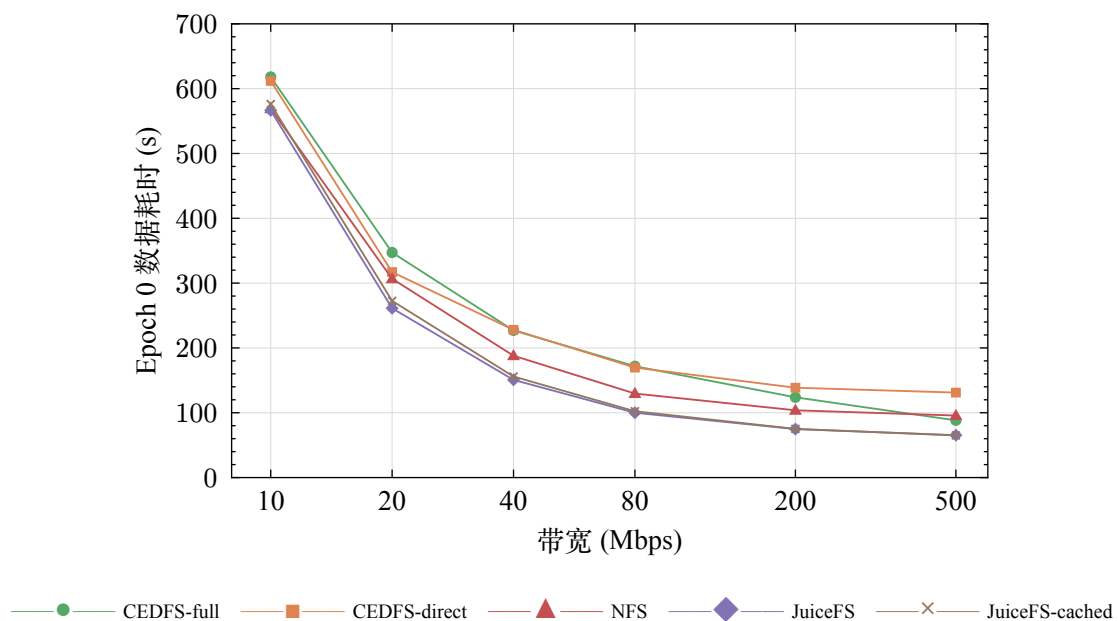


图 6.8 缓存有限下首个 Epoch 数据加载耗时随带宽变化

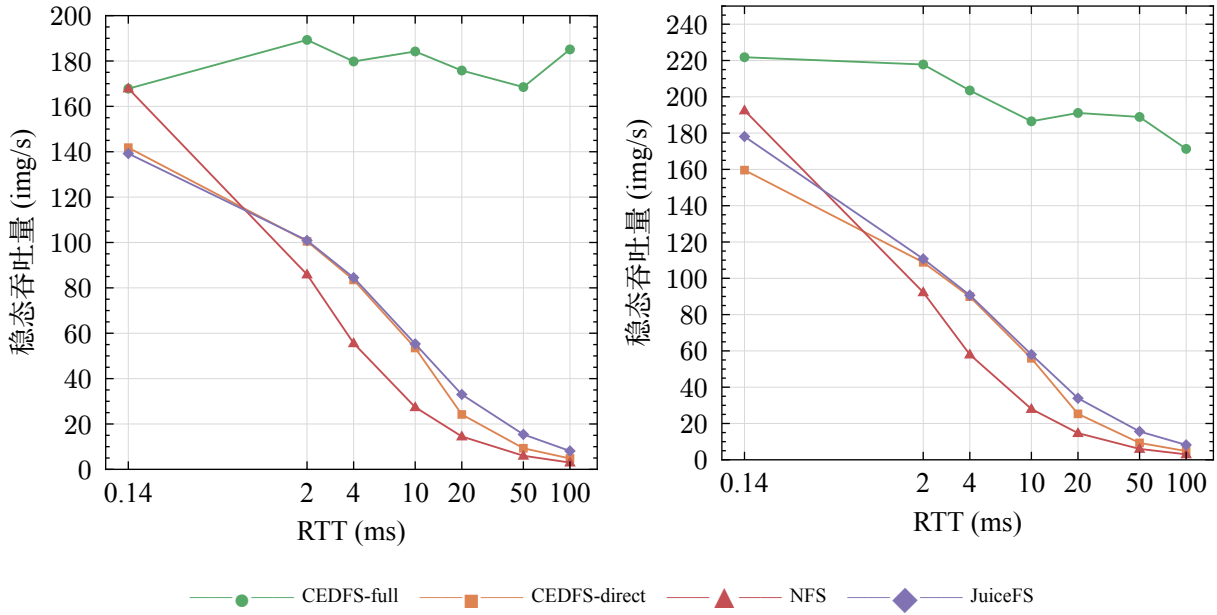


图 6.9 稳态吞吐量随 RTT 变化

6.4.5 检查点保存频率实验分析

本组实验分析检查点保存频率对训练主循环的影响。为了隔离写入后端差异，固定训练数据路径和模型配置，仅改变检查点保存间隔，比较不同写入方案对训练总时间和阻塞率的影响。

图 6.11 显示，本地 NVMe 与 CEDFS-async 的训练总时间基本处于同一量级；当每 1 个 epoch 保存一次时，CEDFS-sync、NFS、JuiceFS 和 JuiceFS-cached 分别达到 968.3、

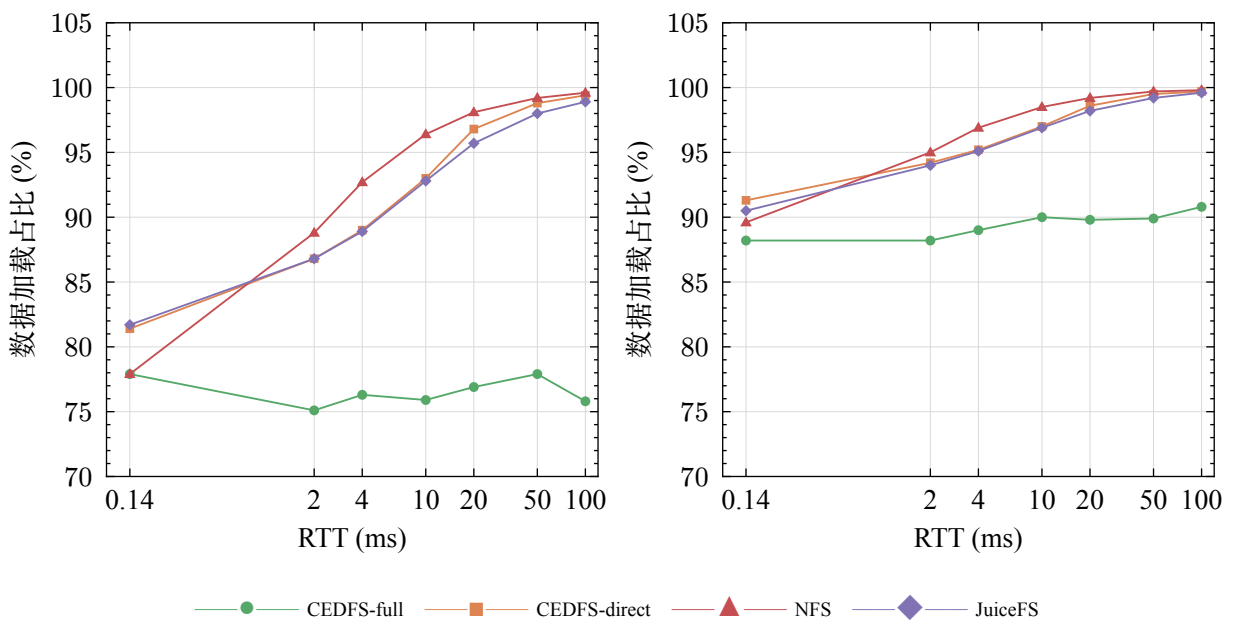


图 6.10 数据加载占比随 RTT 变化

1039.2、946.4 和 895.6 秒，显著高于 CEDFS-async 的 593.3 秒，增幅约为 51%~75%。而当检查点间隔放宽到 10 个 epoch 时，所有方案都回落到 508~560 秒范围，说明该组差异确实主要来自检查点写入本身。换句话说，检查点越频繁，同步远端写的劣势越会被放大。

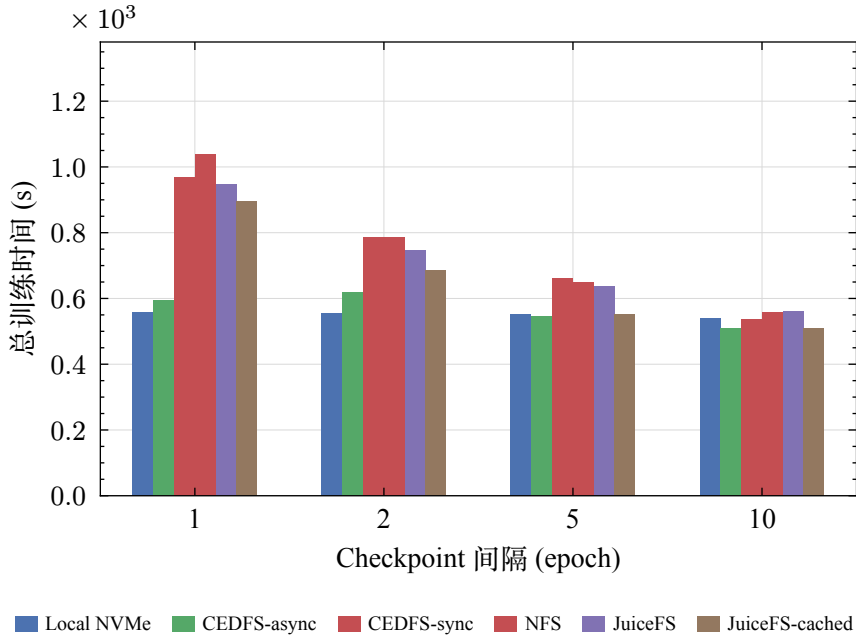


图 6.11 训练总时间随检查点间隔变化

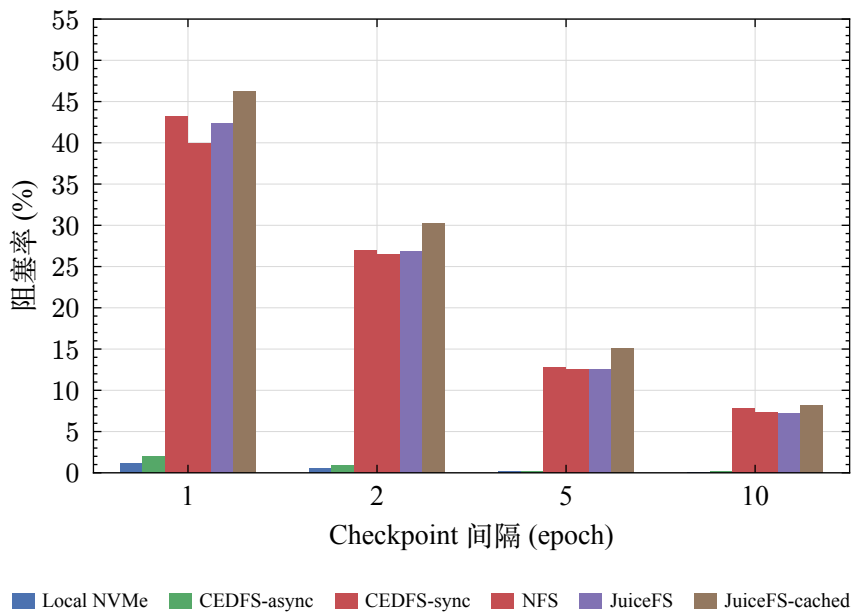


图 6.12 训练阻塞率随检查点间隔变化

图 6.12 给出了更直接的阻塞视角。本地 NVMe 与 CEDFS-async 的阻塞率仅为 0.12~1.95%，而 CEDFS-sync、NFS、JuiceFS 和 JuiceFS-cached 为 7.17~46.3%。在最激进的每 epoch 保存条件下，CEDFS-async 仅 1.95%，其余同步路径为 39.91~46.3%，相当于前者的 20~24 倍。同步路径会在前台产生阻塞，把网络波动直接暴露给训练进程。

本组实验表明，CEDFS-full 在启用框架异步写入后，检查点写入对训练前台造成的停顿已被压缩到极低水平，同时后台写入流量也未对预取流量造成显著干扰。

6.4.6 稳定性实验分析

本组实验以随机丢包为代表考察跨域链路稳定性下降对训练吞吐与尾延迟的影响。为了隔离丢包因素，固定 RTT 约 4 ms 和带宽 80 Mbps，扫描 0~5% 随机丢包率。

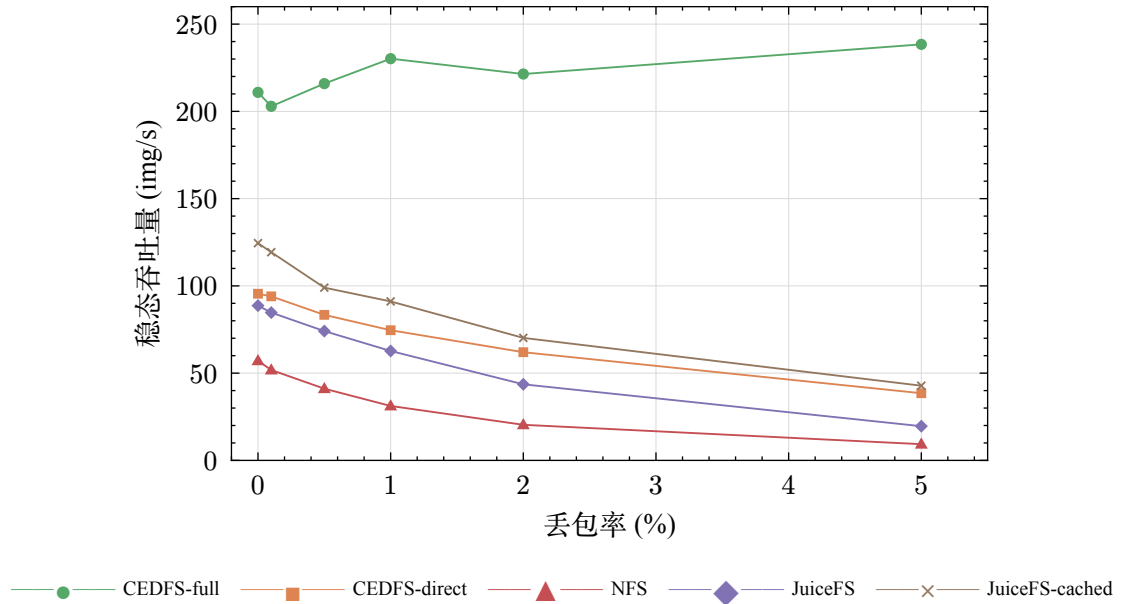


图 6.13 稳态吞吐量随丢包率变化

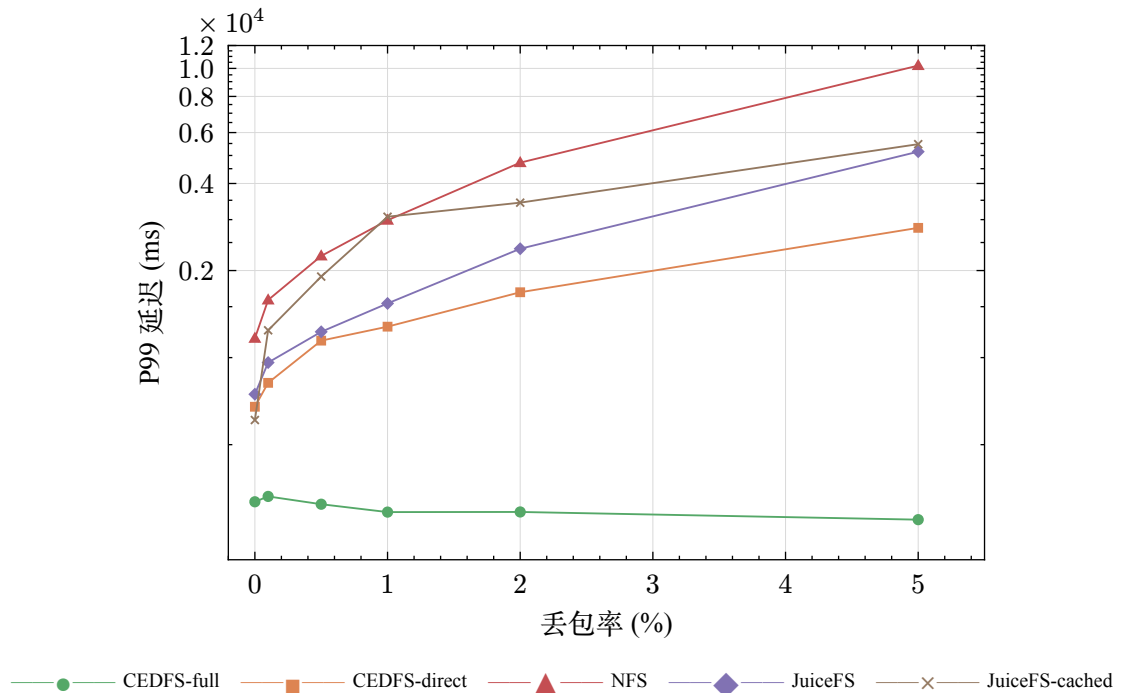


图 6.14 数据加载 P99 延迟随丢包率变化

图 6.13 显示, CEDFS-full 在全部丢包率下保持 202.9~238.4 img/s, 未观察到随丢包率增加而恶化的趋势, 变化基本处于正常波动范围内; 而 CEDFS-direct、NFS、JuiceFS 和 JuiceFS-cached 在 5% 丢包时分别降至 38.5、9.3、19.6 和 42.8 img/s, 均呈现出明显退化的趋势, 这表明仅有本地缓存并不足以保证健壮性。

图 6.14 显示, CEDFS-full 的 P99 始终位于 275~331 ms, 相对稳定, 最坏批次的数据等待时间能控制在约 300 ms 量级。

在链路稳定性下降时, CEDFS-full 削弱了不稳定网络对训练过程的直接扰动。对于边侧长时间训练任务而言, 这种稳定性收益较为关键。

6.5 本章小结

本章针对云边端跨域分布式训练中存储资源分散异构、跨域网络环境复杂以及训练各阶段数据访问需求多样等问题, 设计了云边端跨域虚拟存储空间的总体方案。

首先, 分析了云边端存储资源的三项核心挑战, 即存储资源地域分散且异构、跨域网络环境复杂多变、分布式训练各阶段对存储系统提出差异化访问需求, 并明确了分层架构设计、控制与数据平面分离、基于扩展接口实现训练加速模块三条研究思路。

其次, 设计了物理资源层、系统支撑层和用户层三层架构。物理资源层整合云边端各层存储硬件与介质, 支持 TiKV 和 RocksDB 两种可选的元数据存储后端; 系统支撑层通过元数据服务器和数据服务器实现元数据与数据的分离管控, 元数据服务器采用层级聚合目录树组织命名空间并提供批量请求与预取接口, 数据服务器采用块结构存储数据并提供 gRPC 与 QUIC 双传输通道和状态感知能力; 用户层通过 FUSE 接口、Python SDK 和共享内存缓存服务器向上层提供统一访问入口和训练加速集成能力。

最后, 阐述了 CEDFS 的三层部署模型和工作机制, 给出了基础模式下元数据访问、数据加载、检查点写入和故障恢复四条访问链路, 并在统一架构下集成了前三章分别提出的元数据批量预取、数据预热与检查点分级写入三项关键技术。

端到端实验进一步验证了该集成方案的有效性: 在前述各类场景下, 该架构均能保持较为稳定的训练吞吐, 并将前台阻塞控制在较低水平, 说明本文提出的整体方案能够将三项关键技术协同纳入同一系统, 并在云边端跨域训练场景中获得稳定的整体收益。

总结与展望

论文工作总结

本论文面向云边端跨域分布式训练中元数据访问、数据加载、检查点写入与恢复等 I/O 性能瓶颈,开展面向跨域训练的元数据批量预取与缓存方法、基于异步预取调度的数据预热技术和基于状态感知分配的检查点分级写入技术的研究,优化跨域数据访问通路中的三条关键路径,提升云边端跨域分布式训练效率。本文的主要工作和成果总结如下:

(1) 跨域元数据请求需频繁穿越广域链路,目录遍历和文件属性查询的延迟成倍放大,严重拖慢训练冷启动。针对以上问题,本文设计并实现了面向跨域训练的元数据批量预取与缓存方法。该方法在数据集初始化阶段通过异步递归遍历目录树并将同层多个目录聚合为批量 RPC 请求,在单轮内完成全量元数据的拉取,大幅减少跨域通信次数;构建内存、磁盘、IPC 共享内存三级缓存架构,实现预取结果的高效驻存与跨进程复用,消除重复的跨域请求。实验结果表明,该方法在多种操作类型上均显著降低了元数据访问延迟,且加速优势随网络延迟的增大进一步增强;多级缓存机制使后续训练轮次能够持续命中本地缓存,大幅消除重复的跨域请求;批量聚合有效减少了协议流量开销,并在网络丢包条件下展现出良好的性能稳定性。

(2) 训练样本从远端加载到本地的速度跟不上 GPU 消耗节奏,计算单元长时间等待数据就绪而空转。针对以上问题,本文设计并实现了基于异步预取调度的数据预热技术。该技术利用训练框架随机种子固定所带来的访问序列可预知性,在每轮训练启动前预计算出完整的文件访问顺序并注入预取引擎;预取引擎根据多因子优先级策略生成预取任务,在带宽配额约束下将数据异步搬运至本地缓存;训练执行阶段预取引擎切换为后台运行模式,以预取窗口始终领先于训练消费进度的方式持续搬运,将原本串行的数据加载转化为计算与加载的流水化并行形式。实验结果表明,随着本地缓存预算的增加,该技术能够将有限的缓存更稳定地转化为训练吞吐收益,并在缓存进入有效工作区间后稳定超过对比方案;该技术的主要优势集中在小到中等并行度范围,表明其更适合边侧多卡训练与小规模协同训练环境。

(3) 检查点大体积状态文件的同步写入会阻塞训练迭代数十秒乃至数分钟,且边缘节点异构的带宽与存储条件也使故障后的快速恢复难以保障。针对以上问题,本文设计并实现了基于状态感知分配的检查点分级写入技术。该技术构建覆盖边缘节点存储容量、实时带宽和拓扑距离等维度的动态资源模型;按重要性将检查点内容分为四个优先

等级并配置差异化副本策略，通过五步决策流程实现自适应的分片到节点映射；采用异步本地优先写入将检查点快速落盘、训练随即恢复，远端副本由后台线程自动完成；故障恢复时依据五级恢复阶梯与探测优先策略就近取副本，以最低传输代价恢复训练状态。实验结果表明，异步本地优先写入使保存阻塞时间接近原生本地写入水平；分级恢复稳定缩短约 60% 的恢复时间，且收益比例不受网络延迟影响；容量感知策略在高存储压力下实现优雅降级；自动路由机制在节点故障时透明切换数据源，恢复性能不受单点故障影响。

下一步工作展望

本文解决了云边端跨域分布式训练存储加速中的三类关键 I/O 性能问题，优化了跨域数据访问效率，但这些方法仍具有一定的改进空间，其中主要包含以下三方面：

(1) 针对元数据批量预取与缓存方法，本文采用了静态批量聚合策略将同层目录请求合并为单次 RPC，但是在目录树深度不均衡的场景下，固定的聚合粒度和遍历策略可能无法始终保持最优的预取效率。后续的研究工作可以引入自适应聚合策略，依据训练访问模式动态调整批量大小与遍历优先级，进一步提升元数据预取在异构目录结构下的适应能力。

(2) 针对基于异步预取调度的数据预热技术，本文利用训练随机种子可确定的特性预计算访问序列并进行前瞻式预取，但该方法依赖于训练框架层面的种子确定性，当训练流程引入在线数据增强或动态采样策略时，访问序列的可预测性将受到影响。后续的研究工作可以结合在线访问模式检测与轻量级序列预测机制，在访问序列不完全可知的条件下仍能维持较高的预取命中率，以拓宽该技术在更多训练范式下的适用性。

(3) 针对基于状态感知分配的检查点分级写入技术，本文验证了分级写入能够有效降低保存阻塞并加快故障恢复，但当前检查点粒度划分与副本放置策略仍以通用规则为主，尚未针对不同模型规模、训练阶段和节点条件进行更细致的适配。后续研究可围绕不同训练任务的状态特征，建立更精细的分级标准与资源分配策略，使检查点系统在更广泛部署条件下获得稳定收益。

参考文献

- [1] 董裕民, 张静, 谢昌佐, 等. 云边端架构下边缘智能计算关键问题综述: 计算优化与计算卸载[J]. 电子与信息学报, 2024, 46(3): 765-776.
- [2] ZHENG Y, ZHAO W, LUO J. GTALight: An IoT-Enhanced End – Edge – Cloud Collaborative Framework for Scalable Regional Traffic Signal Control[J/OL]. IEEE Internet of Things Journal, 2026, 13(4): 6863-6876. DOI:10.1109/JIOT.2025.3637902.
- [3] LIU L, ZHANG J, SONG S H, et al. Client-Edge-Cloud Hierarchical Federated Learning[C/OL]//ICC 2020 – 2020 IEEE International Conference on Communications (ICC). 2020: 1-6. DOI:10.1109/ICC40277.2020.9148862.
- [4] 施巍松, 张星洲, 王一帆, 等. 边缘计算: 万物互联时代新型计算模型[J]. 计算机研究与发展, 2017, 54(5): 907-924.
- [5] QIAO A, CHOE S K, SUBRAMANYA S J, et al. Pollux: Co-adaptive Cluster Scheduling for Goodput-Optimized Deep Learning[C]//Proceedings of the 15th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2021. USENIX Association, 2021.
- [6] JIANG Z, LIN H, ZHONG Y, et al. MegaScale: Scaling Large Language Model Training to More Than 10,000 GPUs[C]//Proceedings of the 21st USENIX Symposium on Networked Systems Design and Implementation. USENIX Association, 2024: 745-760.
- [7] 谢超, 王磊, 过敏意, 等. 分布式深度学习训练系统综述[J]. 软件学报, 2023, 34(7): 3083-3108.
- [8] 赵世琳. 智能建造安防视频监控系统云边端协同计算框架设计[J/OL]. 四川建材, 2025, 51(11): 49-51. DOI:10.3969/j.issn.1672-4011.2025.11.013.
- [9] ARE S, KRISHNA N G, THATI J, et al. Federated multi scale vision transformer with adaptive client aggregation for industrial defect detection[J/OL]. Scientific Reports, 2025, 15(1): 34149. DOI:10.1038/s41598-025-12390-z.
- [10] WANG T, GOOI H B. Distribution-Balanced Federated Learning for Fault Identification of Power Lines[J/OL]. IEEE Transactions on Power Systems, 2024, 39(1): 1209-1223. DOI:10.1109/TPWRS.2023.3267463.
- [11] HE K, ZHANG X, REN S, et al. Deep Residual Learning for Image Recognition[C/OL]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). 2016: 770-778. DOI:10.1109/CVPR.2016.90.
- [12] SILVA F M d, TAVARES J M R S, LOPES A M, et al. Cross-Dataset Benchmarking of Deep Learning Models for Surface Defect Classification in Metal Parts[J/OL]. Applied Sciences, 2026, 16(6): 3022. DOI:10.3390/app16063022.
- [13] SU Y, FAN W, MENG Q. Joint Adaptive Aggregation and Resource Allocation for Hierarchical Federated Learning Systems Based on Edge-Cloud Collaboration[J/OL]. IEEE Transactions on Cloud Computing, 2025, 13(1): 369-382. DOI:10.1109/TCC.2025.3530681.
- [14] YANG X, LIWANG M, WANG X. HEART: Achieving Timely Multi-Model Training for Vehicle-Edge-Cloud-Integrated Hierarchical Federated Learning[Z/OL]. (2025). DOI:10.48550/arXiv.2501.09934.

-
- [15] CHEN L, LI G, LV Y, et al. Explicit Data Correlations-Directed Metadata Prefetching Method in Distributed File Systems[J/OL]. IEEE Transactions on Parallel and Distributed Systems, 2019, 30(12): 2692-2705. DOI:10.1109/TPDS.2019.2921760.
- [16] WEI X, XIAO L, SONG W, et al. A self-tuning client-side metadata prefetching scheme for wide area network file systems[J/OL]. Science China Information Sciences, 2022, 65(3): 132101. DOI:10.1007/s11432-019-2833-1.
- [17] 刘炆, 郑帆, 刘彬, 等. 边缘智能: 现状与展望[J]. 中国科学: 信息科学, 2020, 50(11): 1611-1631.
- [18] LIU M, PAN L, LIU S. Collaborative Storage for Tiered Cloud and Edge: A Perspective of Optimizing Cost and Latency[J/OL]. IEEE Transactions on Mobile Computing, 2024, 23(12): 10885-10902. DOI:10.1109/TMC.2024.3384264.
- [19] CHEN J, WANG Y, YE M, et al. A Secure Cloud-Edge Collaborative Fault-Tolerant Storage Scheme and Its Data Writing Optimization[J/OL]. IEEE Access, 2023, 11: 66506-66521. DOI:10.1109/ACCESS.2023.3291452.
- [20] ROY S, SARKAR S, SAHU A. Profit Maximization Using Collaborative Storage Management in Multi-Tier Edge-Cloud System[C/OL]//Proceedings of the 30th IEEE International Conference on High Performance Computing, Data, and Analytics. IEEE, 2023: 309-318. DOI:10.1109/HiPC58850.2023.00048.
- [21] DE LIMAMELO V D, THIESSEN M, PANAS A. PathFS: A File System for the Hierarchical Edge[C]//Proceedings of the 7th International Workshop on Edge Systems, Analytics and Networking. New York, NY, USA: Association for Computing Machinery, 2024: 55-60.
- [22] ZHANG Y, LI G, LU K. PhatKV: Towards an Efficient Metadata Engine for KV-based File Systems on Modern SSD[C]//Proceedings of the 24th USENIX Conference on File and Storage Technologies. 2024.
- [23] HE Q, TAN S, CHEN F. EDIndex: Enabling Fast Data Queries in Edge Storage Systems[C]//Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval. Taipei, Taiwan: ACM, 2023: 675-685.
- [24] XIE J, QIAN C, GUO D. Efficient Indexing Mechanism for Unstructured Data Sharing Systems in Edge Computing[C]//IEEE International Conference on Computer Communications (INFOCOM). Paris, France: IEEE, 2019: 820-828.
- [25] WU G, DENG Y, QIN X. Using Provenance to Boost the Metadata Prefetching in Distributed Storage Systems[C/OL]//2016 IEEE 34th International Conference on Computer Design (ICCD). 2016: 80-87. DOI:10.1109/ICCD.2016.7753264.
- [26] VANGOOR B K R, TARASOV V, ZADOK E. To FUSE or Not to FUSE: Performance of User-Space File Systems[C]//Proceedings of the 15th USENIX Conference on File and Storage Technologies (FAST). USENIX Association, 2017: 59-72.
- [27] DAI H, WANG Y, KENT K B. The State of the Art of Metadata Managements in Large-Scale Distributed File Systems - Scalability, Performance and Availability[J]. IEEE Transactions on Parallel and Distributed Systems, 2022, 33(12): 3850-3869.
- [28] PAWLOWSKI B, JUSZCZAK C, STAUBACH P, et al. NFS version 3: design and implementation[C]//USENIX Summer. Boston, MA, 1994: 137-152.

-
- [29] MORRIS J H, SATYANARAYANAN M, CONNER M H, et al. Andrew: a distributed personal computing environment[J]. *Communications of the ACM*, 1986, 29(3): 184-201.
- [30] NELSON M N, WELCH B B, OUSTERHOUT J K. Caching in the sprite network file system[J]. *ACM Transactions on Computer Systems (TOCS)*, 1988, 6(1): 134-154.
- [31] YI C, JIN H, JIA Y. Metadata management in global distributed storage system[C]//*Current Trends in High Performance Computing and Its Applications*. Springer, 2005: 175-184.
- [32] TATEBE O, HIRAGA K, SODA N. Gfarm grid file system[J]. *New Generation Computing*, 2010, 28(3): 257-275.
- [33] PAN S, STAVRINOS T, ZHANG Y, et al. Facebook's Tectonic Filesystem: Efficiency from Exascale[C]//*Proceedings of the 19th USENIX Conference on File and Storage Technologies (FAST)*. USENIX Association, 2021: 217-231.
- [34] WANG L, YE S, YANG B, et al. DIESEL: A Dataset-Based Distributed Storage and Caching System for Large-Scale Deep Learning Training[C/OL]//*49th International Conference on Parallel Processing (ICPP)*. ACM, 2020: 20:1-20:11. DOI:10.1145/3404397.3404472.
- [35] GHEMAWAT S, GOBIOFF H, LEUNG S T. The Google File System[C]//*Proceedings of the 19th ACM Symposium on Operating Systems Principles (SOSP)*. Bolton Landing, NY, USA: ACM, 2003: 29-43.
- [36] SCHMUCK F, HASKIN R. GPFS: A Shared-Disk File System for Large Computing Clusters[C]//*Proceedings of the 1st USENIX Conference on File and Storage Technologies (FAST)*. USENIX Association, 2002: 231-244.
- [37] WEIL S A, BRANDT S A, MILLER E L, et al. Ceph: a scalable, high-performance distributed file system[C]//*Proceedings of the 7th Symposium on Operating Systems Design and Implementation*. 2006: 307-320.
- [38] WANG Y, LI C, SHAO X, et al. Lunule: an agile and judicious metadata load balancer for CephFS[C]//*Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 2021.
- [39] REN K, GIBSON G A. IndexFS: Scaling File System Metadata Performance with Stateless Caching and Bulk Insertion[C/OL]//*Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*. 2014: 237-248. DOI:10.1109/SC.2014.25.
- [40] XIE W, CHEN Z. LocoFS: A Loosely-Coupled Metadata Service for Distributed File Systems[C/OL]//*Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*. 2017: 1-13. DOI:10.1145/3126908.3126928.
- [41] NIAZI S, ISMAIL M, HARIDI S, et al. HopsFS: Scaling Hierarchical File System Metadata Using NewSQL Databases[C]//*Proceedings of the 15th USENIX Conference on File and Storage Technologies (FAST)*. USENIX Association, 2017: 89-104.
- [42] LV W, LU Y, ZHANG Y. InfiniFS: An Efficient Metadata Service for Large-Scale Distributed Filesystems[C]//*Proceedings of the 20th USENIX Conference on File and Storage Technologies (FAST)*. USENIX Association, 2022: 313-328.
- [43] WANG Y. CFS: Scaling Metadata Service for Distributed File System via Pruned Scope of Critical Sections[C/OL]//*Proceedings of the 18th European Conference on Computer Systems (EuroSys)*. ACM, 2023: 331-346. DOI:10.1145/3552326.3587443.

-
- [44] CAI M, SHEN J, TANG B. Exploiting Flat Namespace to Improve File System Metadata Performance on Ultra-fast, Byte-addressable NVMs[J/OL]. *ACM Transactions on Storage*, 2024, 20(1): 2:1-2:47. DOI:10.1145/3620673.
- [45] WANG Y, LIU Y, CHEN Z. PancakeFS: A Write Efficiently and Read Optimized Filesystem[C]//2024 4th International Conference on Consumer Electronics and Computer Engineering (ICCECE). 2024: 196-199.
- [46] GUO H, LU Y, LV W, et al. SingularFS: A Billion-Scale Distributed File System Using a Single Metadata Server[C]//*Proceedings of the USENIX Annual Technical Conference (ATC)*. USENIX Association, 2023: 915-928.
- [47] MOHAN J, PHANISHAYEE A, RANIWALA A, et al. Analyzing and Mitigating Data Stalls in DNN Training[J]. *Proceedings of the VLDB Endowment*, 2021, 14(5): 771-784.
- [48] ERBENA, MAYER R, JACOBSEN H A. How Can We Train Deep Learning Models Across Clouds and Continents? An Experimental Study[J]. *Proceedings of the VLDB Endowment*, 2024, 17(6): 1214-1226.
- [49] SUN B, YU X, ZHANG C. SOLAR: A Highly Optimized Data Loading Framework for Distributed Training of CNN-based Scientific Surrogates[Z]. *arXiv*, 2022.
- [50] LI S, ZHAO Y, VARMA R, et al. PyTorch Distributed: Experiences on Accelerating Data Parallel Training[J]. *Proceedings of the VLDB Endowment*, 2020, 13(12): 3005-3018.
- [51] KHAN R I S, YAZDANI A H, FU Y, et al. SHADE: Enable Fundamental Cacheability for Distributed Deep Learning Training[C]//*Proceedings of the 21st USENIX Conference on File and Storage Technologies (FAST)*. USENIX Association, 2023: 135-152.
- [52] LIU J, NICOLAE B, LI D. Lobster: Load Balance-Aware I/O for Distributed DNN Training[C]//*Proceedings of the 51st International Conference on Parallel Processing*. Bordeaux, France: ACM, 2022: 1-11.
- [53] MURRAY D G, SIMSA J, KLIMOVIC A, et al. tf.data: A Machine Learning Data Processing Framework[J]. *Proceedings of the VLDB Endowment*, 2021, 14(12): 2945-2958.
- [54] GRAUR D, AYMOUN D, KLUSER D, et al. Cachew: Machine Learning Input Data Processing as a Service[C]//*Proceedings of the 2022 USENIX Annual Technical Conference, USENIX ATC 2022*. USENIX Association, 2022: 689-706.
- [55] GU R, ZHANG K, XU Z. Fluid: Dataset Abstraction and Elastic Acceleration for Cloud-native Deep Learning Training Jobs[C]//2022 IEEE 38th International Conference on Data Engineering (ICDE). Kuala Lumpur, Malaysia: IEEE, 2022: 2182-2195.
- [56] GARG S, ZHANG J, PITCHUMANI R. CrossPrefetch: Accelerating I/O Prefetching for Modern Storage[C]//*Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*. La Jolla CA, USA: ACM, 2024: 102-116.
- [57] DRYDEN N, BÖHRINGER R, BEN-NUN T. Clairvoyant Prefetching for Distributed Machine Learning I/O[C/OL]//*Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*. 2021: 1-15. DOI:10.1145/3458817.3476181.
- [58] SUBRAMANYA S J, ARFEEN D, LIN S, et al. Sia: Heterogeneity-aware, Goodput-optimized ML-Cluster Scheduling[C/OL]//*Proceedings of the 29th ACM Symposium on Operating Systems Principles, SOSP 2023*. ACM, 2023: 642-657. DOI:10.1145/3600006.3613175.

-
- [59] ROBROEK T, NIELSEN N K, TÖZÜN P. TensorSocket: Shared Data Loading for Deep Learning Training[J/OL]. *Proceedings of the ACM on Management of Data*, 2025, 3(4): 267:1-267:26. DOI:10.1145/3749185.
- [60] EISENMAN A, MATAM K K, INGRAM S, et al. Check-N-Run: a Checkpointing System for Training Deep Learning Recommendation Models[C]//*Proceedings of the 19th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2022*. USENIX Association, 2022.
- [61] DUAN J, SONG Z, MIAO X, et al. Parcae: Proactive, Liveput-Optimized DNN Training on Preemptible Instances[C]//*Proceedings of the 21st USENIX Symposium on Networked Systems Design and Implementation, NSDI 2024*. USENIX Association, 2024.
- [62] KHANDEPARKAR A, RIVIÈRE É, VOULGARIS S. Quiver: An Informed Storage Cache for Deep Learning[C]//*Proceedings of the 18th USENIX Conference on File and Storage Technologies (FAST)*. USENIX Association, 2020: 283-296.
- [63] LION R, THIBAUT S. From Tasks Graphs to Asynchronous Distributed Checkpointing with Local Restart[C/OL]//*2020 IEEE/ACM 10th Workshop on Fault Tolerance for HPC at eXtreme Scale (FTXS)*. 2020: 31-40. DOI:10.1109/FTXS51974.2020.00009.
- [64] WANG Z, JIA Z, ZHENG S. GEMINI: Fast Failure Recovery in Distributed Training with In-Memory Checkpoints[C]//*Proceedings of the 29th Symposium on Operating Systems Principles*. Koblenz, Germany: ACM, 2023: 364-381.
- [65] WANG G, RUWASE O, XIE B. FastPersist: Accelerating Model Checkpointing in Deep Learning[Z]. *arXiv*, 2024.
- [66] MAURYA A, UNDERWOOD R, RAFIQUE M M. DataStates-LLM: Lazy Asynchronous Checkpointing for Large Language Models[C]//*Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing*. Pisa, Italy: ACM, 2024: 227-239.
- [67] WAN B, HAN M, SHENG Y. ByteCheckpoint: A Unified Checkpointing System for Large Foundation Model Development[Z]. *arXiv*, 2024.
- [68] WANG Z, ZHU W, ZHANG Z, et al. Amber: Towards Fast and Space-Efficient Incremental Checkpointing in Large Language Model Training[C/OL]//*Proceedings of the 54th International Conference on Parallel Processing*. ACM, 2025: 678-688. DOI:10.1145/3754598.3754606.
- [69] MOHAN J, PHANISHAYEE A, RANIWALA A, et al. CheckFreq: Frequent, Fine-Grained DNN Checkpointing[C]//*Proceedings of the 19th USENIX Conference on File and Storage Technologies (FAST)*. USENIX Association, 2021: 203-216.
- [70] ZHANG K, CHEN Y, HU Z, et al. GoCkpt: Gradient-Assisted Multi-Step Overlapped Checkpointing for Efficient LLM Training[Z]. *arXiv*, 2025.
- [71] HUANG Z, NIE H, JIA H, et al. FlowCheck: Decoupling Checkpointing and Training of Large-Scale Models[C/OL]//*Proceedings of the 20th European Conference on Computer Systems*. ACM, 2025: 1334-1349. DOI:10.1145/3689031.3696088.
- [72] LIAN X, JACOBS S A, KURILENKO L, et al. Universal Checkpointing: A Flexible and Efficient Distributed Checkpointing System for Large-Scale DNN Training with Reconfigurable Parallelism[C]//*Proceedings of the 2025 USENIX Annual Technical Conference (ATC)*. Boston, MA, USA: USENIX Association, 2025: 1519-1534.

- [73] XU W, HUANG X, MENG S, et al. An Efficient Checkpointing System for Large Machine Learning Model Training[C/OL]//Proceedings of the SC '24 Workshops. IEEE, 2024: 896-900. DOI:10.1109/SCW63240.2024.00127.
- [74] LIU X, CHEN Y, XU J, et al. RapidScribe: Bandwidth-aware Parallel Checkpoint for Distributed Neural-Network Training[C/OL]//Proceedings of the 45th IEEE International Conference on Distributed Computing Systems (ICDCS). 2025: 824-834. DOI:10.1109/ICDCS63083.2025.00085.
- [75] QI H, LI C, WU Y, et al. ECCheck: Enhancing In-Memory Checkpoint with Erasure Coding in Distributed DNN Training[C/OL]//Proceedings of the 45th IEEE International Conference on Distributed Computing Systems (ICDCS). 2025: 253-263. DOI:10.1109/ICDCS63083.2025.00033.
- [76] JANG I, YANG Z, ZHANG Z, et al. Oobleck: Resilient Distributed Training of Large Models Using Pipeline Templates[C/OL]//Proceedings of the 29th ACM Symposium on Operating Systems Principles. ACM, 2023: 382-395. DOI:10.1145/3600006.3613152.
- [77] WANG Z, FU J, LI C, et al. Elastor: Elastic and Efficient Model Partitioning and Checkpointing for Fault-Tolerant Distributed Training[C/OL]//Proceedings of the 30th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP). 2026: 398-412. DOI:10.1145/3774934.3786445.
- [78] THORPE J, ZHAO P, EYOLFSON J, et al. Bamboo: Making Preemptible Instances Resilient for Affordable Training of Large DNNs[C]//Proceedings of the 20th USENIX Symposium on Networked Systems Design and Implementation. USENIX Association, 2023: 497-513.
- [79] LI W, CHEN X, SHU H. ExCP: Extreme LLM Checkpoint Compression via Weight-Momentum Joint Shrinking[C]//Proceedings of the 41st International Conference on Machine Learning (ICML): Vol. 235. PMLR, 2024: 27575-27588.
- [80] AGRAWAL A, REDDY S, BHATTAMISHRA S. Inshrinkerator: Compressing Deep Learning Training Checkpoints via Dynamic Quantization[C]//Proceedings of the ACM Symposium on Cloud Computing (SoCC). 2024: 1012-1031.
- [81] JIN H, WU D, ZHANG S. Design of a Quantization-Based DNN Delta Compression Framework for Model Snapshots and Federated Learning[J]. IEEE Transactions on Parallel and Distributed Systems, 2023, 34(3): 923-937.
- [82] YAO J, LIU X, HU R, et al. LowDiff: Efficient Frequent Checkpointing via Low-Cost Differential for High-Performance Distributed Training Systems[C/OL]//Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC). 2025: 1113-1126. DOI:10.1145/3712285.3759891.
- [83] HEMMINGER S. netem – Network Emulator[EB/OL]. (2011). <https://wiki.linuxfoundation.org/networking/netem>.
- [84] DEVERA M. HTB Linux queuing discipline manual[EB/OL]. (2002). <http://luxik.cdi.cz/~devik/qos/htb/manual/userg.htm>.
- [85] 王锦权, 刘旭昭, 廖晓坚, 等. SpanTrain:基于云边端异构设备的跨域分布式模型训练系统[J/OL]. 大数据, 2025, 11(3): 17-32. DOI:10.11959/j.issn.2096-0271.2025040.
- [86] 舒继武, 陈游旻, 胡庆达, 等. 非易失性内存系统软件研究进展[J/OL]. 中国科学: 信息科学, 2021, 51(6): 869-899. DOI:10.1360/SSI-2019-0128.

- [87] PINGCAP AND TIKV CONTRIBUTORS. TiKV[EB/OL]. (2025). <https://github.com/tikv/tikv>.
- [88] META OPEN SOURCE AND ROCKSDB CONTRIBUTORS. RocksDB[EB/OL]. (2025). <https://github.com/facebook/rocksdb>.

攻读硕士学位期间取得的成果

攻读硕士学位期间取得的创新成果

- [1] 肖利民, 李辉阳, 王锦权, 沈润楠, 陈旭光, 杨睿, **谭升阳**. 云边端环境中异构数据库的元数据抽取数据访问机制[P]. 中国专利: CN119576962A, 2025-03-07.
- [2] 肖利民, 王锦权, 沈润楠, 霍志胜, **谭升阳**, 张晨浩. 一种基于分区稀疏通信的云边端跨域训练高效通信技术[P]. 中国专利: CN119676235A, 2025-03-21.
- [3] 肖利民, 沈润楠, 王锦权, 霍志胜, **谭升阳**, 李云潼, 阮利. 面向云边端协同环境的数据布局方法[P]. 中国专利: CN119544721A, 2025-02-28.
- [4] 尚晶, 肖利民, 肖智文, 陈旭光, 武智晖, **谭升阳**, 张逸飞, 卢泽萍, 王冀彬. 分散集群的聚合访问方法、装置、设备、存储介质及产品[P]. 中国专利: CN119299523A, 2025-01-10.

攻读硕士学位期间参与的主要科研工作

- [1] 面向云边端协同的芯粒结构与系统软件（项目编号：2023YFB4503100）. 国家重点研发计划共性关键技术类专项. 2023.11—2026.10. 作为项目/课题骨干，参与项目的调研、申请、实施阶段，主要负责跨域虚拟存储空间 CEDFS 的整体设计与实现、面向云边端跨域分布式训练的存储加速技术方案设计与实现等技术研发工作。

致谢

在学位论文完成之际，谨向在研究生阶段给予我指导、帮助和支持的各位老师、同学及家人致以诚挚谢意。

首先，衷心感谢我的导师肖利民教授。肖老师学识渊博、治学严谨，从课题选题、系统架构设计到论文撰写与修改，始终给予我耐心细致的指导。老师严谨求实的学术态度和对问题的深刻洞察，使我在科研方法、工程实践和论文写作等方面都受益匪浅。

同时，感谢阮利老师、廖晓坚老师、霍志胜老师、王良老师和刘磊老师在科研工作和实验室学习过程中给予的帮助与指导。感谢王锦权、张晨浩、沈润楠，索咖顺、谢喜龙等师兄在研究思路、工程实践和论文写作方面提供的宝贵建议。感谢刘旭昭、李骏琳、苏晓、宋震、李云潼、王泽宇、张昊、孟令达、李辉阳、陈旭光、杨睿、张煜森、孙润泽、陈奕铭、杨辛晨、阮思凯、杨廷玉和胡菱艳等同学在学习和科研过程中给予的支持与陪伴。

此外，感谢我的家人长期以来的理解、信任与支持，使我能够安心完成学业与科研工作。家人的关心和鼓励始终是我不断前行的重要力量。

最后，感谢参与论文评审和答辩的各位专家教授。各位老师提出的宝贵意见和建议，对本文的进一步修改和完善具有重要帮助。